Certified Prime Lifting from Known Primes

Eighteen Additional Algorithms for Construction, Arithmetic, and Certificate Synthesis

Revised edition with the complete large-prime register

Keio

6 November 2026

Abstract

We study how certified primes can be reused to construct and prove larger primes, and how several incomplete certificates can be combined into a complete proof. The baseline investigation uses multiplicative orders, Lucas sequences, cyclotomic transfers, and elliptic curves; its inventory of 32 distinct certified integers and its complete verifier are retained. Five later certified integers extend the designated main inventory to 37 primes. Their shared ten-node certificate graph contains five entries with at least 18,000 decimal digits and reaches 18,886 digits. Two of the five were previously recorded probable-prime hits whose full certificates are completed in this register revision. The eighteen retained modules provide explicit algorithms with input contracts, correctness proofs, and executable demonstrations. They include coefficient-sensitive closures for ap^2+1, ap^3+1, and ap^2-1; digit-constrained generation; exact batch arithmetic; witness and elliptic-order fusion; and an intersection of residue classes with integer Hasse windows. In particular, a full Fermat congruence and short nonunity certify ap^2+1 for positive even $a < 4p-2$, while a short-index Lucas certificate applies to nonsquare positive even $a < 4p+2$ in ap^2-1. The construction prototypes certify outputs through 117 digits, and the positive quadratic criterion also recertifies the existing 18,882-digit chain member. Independent finite checks include exact trial division, direct recurrences, invalid-input controls, and exhaustive small optimization problems. All numerical constants and seven executable verification listings are embedded in this single source. These are additions and refinements developed for this manuscript, not established claims of historical priority, universal speed superiority, a new world record, or a resolution of the Collatz or Mills transcendence problems.

Keywords. Provable prime generation; Pocklington certificate; Lucas sequence; cyclotomic polynomial; elliptic primality certificate; complex multiplication; reproducible computation.

Scope. This is a computational research manuscript, not a claim of publication or peer review. It consolidates an interactive investigation and records which conclusions have explicit proofs and which remain proposals.

Reading the revision. The baseline constructions and their original verifier are preserved. Section 14 records the distinction between prime counts and algorithm counts. Section 14.1 now consolidates all five certified entries with at least 18,000 digits, including their status histories; Appendix G supplies the new standalone ten-node verifier. Sections 16, 17, and 18 contain A01–A18. Section 22 explains how to extract every complete program from this one TEX file.

Contents

1 Introduction and the precise problem **4**
1.1 What the computations establish . . . 4
1.2 What "immediate" can and cannot mean . . . 4

2 Three reusable certificate mechanisms **5**
2.1 Multiplicative orders from a factored part of $N-1$ . . . 5
2.2 Lucas ranks from a factored part of $N+1$ . . . 6
2.3 Elliptic points of a known order . . . 6

3 Arithmetic constructions and cubic extensions **7**
3.1 Linear, power, and mixed-product lifts . . . 7
3.2 A positive cubic specialization . . . 7
3.3 A negative cubic specialization . . . 8
3.4 An exact square-test closure throughout the cubic range . . . 9
3.5 Combining both signs by the Chinese remainder theorem . . . 9

4 Cyclotomic and repunit certificate transfer **10**

5 Reverse norm constructions and elliptic certificates **11**
5.1 Gaussian norms: discriminant -4 . . . 11
5.2 The norm x^2+3y^2: CM field discriminant -3 . . . 11
5.3 Extensions that have not been executed . . . 12

6 The numerical inventory and its provenance **12**
6.1 Foundational seeds . . . 12
6.2 First experiment: ten outputs . . . 12
6.3 The squaring chain through 9,439 digits . . . 13
6.4 Second experiment: nine principal and two auxiliary outputs . . . 14
6.5 Search and verification protocol . . . 14

7 An implementation improvement: shared exponents **15**

8 Record-scale inputs and stopped searches **15**
8.1 Mersenne coefficients inside Proth candidates . . . 15
8.2 What can be decided immediately at record scale . . . 16
8.3 Bounded experiments that did not produce a prime . . . 17

9 A constructive connection with Collatz prime trajectories **17**
9.1 Repository provenance and the usable construction . . . 17
9.2 An additional executed example: two 391-digit primes . . . 18
9.3 Why inverse ancestors of a fixed Mersenne seed are different . . . 18

10 The original Mills-constant motivation and its limits **19**

11 A catalogue of candidate designs **19**

12 Limitations and concrete further questions **20**

13 Reproducibility, provenance, and the complete verification record **21**

14 Revision ledger and the continuing computation **23**
14.1 Complete register of certified primes with at least 18,000 digits . . . 23
14.2 The interrupted 37,775-digit search . . . 25

15 A common contract for the eighteen algorithms **25**
15.1 Scope of the additional algorithms . . . 26
15.2 Catalogue of the eighteen additions . . . 26
15.3 A logical restriction on composition . . . 27

16 Six construction algorithms and their exact acceptance rules **28**
16.1 A01: parity-strengthened quadratic order closure . . . 28
16.2 A02: cubic closure by a bounded trace scan . . . 29
16.3 A03: simultaneous opposite-sign certificates . . . 30
16.4 A04: inverse construction with prescribed digit blocks . . . 31
16.5 A05: cyclotomic divisor mining with a certified size threshold . . . 32
16.6 A06: short-index Lucas closure for negative quadratic lifts . . . 33
16.7 Finite verification of the six constructions . . . 35

17 Six exact arithmetic and search modules **35**
17.1 A07: a certificate-constrained digit-language join . . . 36
17.2 A08: a quadratic residue compiler with a finite-difference audit . . . 37
17.3 A09: two-tree batch sieving with explicit factor routing . . . 38
17.4 A10: signed cofactor reduction with one correction . . . 39
17.5 A11: a CRT packet of common prime-power orders . . . 41
17.6 A12: an exact-cost finite-population screening planner . . . 42
17.7 Reproducibility and what these six experiments establish . . . 44

18 Certificate synthesis, hybrid bounds, and proof reuse **44**
18.1 A13: prime-power witness projection and fusion . . . 44
18.2 A14: batched unit checks with exact failure localization . . . 45
18.3 A15: elliptic order fusion by primary projection . . . 46
18.4 A16: compiling residue constraints with exact Hasse windows . . . 47
18.5 A17: exact proof-network planning with shared ancestors . . . 49
18.6 A18: inheriting a prime certificate through an extracted divisor . . . 50

19 Integrated experiments and independent checks **51**
19.1 What was actually executed . . . 51
19.2 An alternate certificate for the existing large chain . . . 52
19.3 Independent oracles and deliberately invalid inputs . . . 52

20 How the modules can be combined **53**
20.1 Three concrete workflows . . . 53
20.2 The meaningful sense of a new composition . . . 54

21 Efficiency, digit blocks, and the Lucas–Lehmer comparison **54**

22 Reproducing the expanded manuscript **55**
22.1 Extracting the exact programs . . . 55
22.2 Execution ledger and source identities . . . 56

References **58**

A Complete standalone certificate verifier **60**

B Construction algorithms A01–A06 **68**

C Arithmetic and search algorithms A07–A12 **75**

D Certificate algorithms A13–A18 **83**

E Exact verification of the continuing seven-node chain **91**

F Independent checks of the coefficient boundaries **93**

G Complete verification of the current large-prime register **95**

1 Introduction and the precise problem

The input is one or more integers whose primality is already proved. The aim is to construct a larger integer together with a short, independently checkable reason that it is prime. A useful construction must therefore specify three different objects: a family of candidates, a search within that family, and a certificate for each accepted candidate. Conflating these objects leads to incorrect conclusions, especially when a formula is short but its numerical evaluation involves millions of digits.

Definition 1.1. A *certified prime lift* is a tuple consisting of certified prime inputs, an explicitly defined output integer N larger than its designated input, and a finite certificate whose validity implies that N is prime. Different inputs may be used together; an input prime may occur to a high power.

This terminology describes a workflow, not a new primality theorem. Recursive generation of provable primes is well established, notably in Maurer's work [4]. The $N-1$ and $N+1$ methods used here belong to the classical Pocklington, Lucas, Lehmer, Morrison, and Brillhart–Lehmer–Selfridge framework [1, 2, 3]. The elliptic argument is the Goldwasser–Kilian certificate principle [5, 6]; constructing curves with prescribed order is related to reverse CM methods [7]. Our contribution is an explicit synthesis, elementary proofs for selected specializations, and a reproducible finite collection of certificates.

1.1 What the computations establish

The retained baseline investigation separates three cohorts.

(i) The first experiment starts from two Mersenne primes and one certified Proth prime and produces ten larger primes. The largest has 4,153 digits.

(ii) A previously reported cyclotomic seed and four successive squaring lifts are reconstructed and certified, ending at 9,439 digits. The coefficients of this chain came from the preceding conversation; their verification is not reported as a fresh search.

(iii) The second experiment produces nine principal outputs and two auxiliary cyclotomic outputs. These include three elliptic certificates, two pairs of cyclotomic transfers, a negative cubic Lucas certificate, and a linear lift from the 4,718-digit chain member.

An additional experiment in section 9 supplies two primes linked by an exact Collatz orbit segment. After duplicate seeds are removed, the combined baseline inventory contains 32 distinct primes. The five later large integers in Section 14.1 are additional and distinct, so the designated main inventory now contains 37 primes. The small, sometimes overlapping A01–A18 demonstrations are tracked separately. All integer identities and digit counts are exact. The certificate checks are deterministic integer computations, although search routines may use probable prime tests to reject candidates cheaply. No probable-prime pass is accepted as the final evidence of primality.

1.2 What "immediate" can and cannot mean

A formula such as $N = cP^2 + 1$ can be written immediately, and its factorization $N - 1 = cP^2$ immediately identifies potentially useful information. Neither fact proves N prime. For example, $P = 3$ and $c = 6$ give $N = 55$. Conversely, an already supplied certificate can often be checked much faster than a suitable candidate can be found. The numerical results below distinguish these two costs explicitly.

Large known primes can also be inserted into expressions that are immediately proved composite by small congruences. This is useful pruning, but is not a new giant-prime discovery. Likewise, an identity that rewrites an already known larger prime in terms of a smaller one reuses existing knowledge; it does not transfer primality by algebra alone.

2 Three reusable certificate mechanisms

We write $v_q(m)$ for the exponent of a prime q in m. A *fully factored* integer means that all of its prime factors, their multiplicities, and proofs of the primality of those factors are available. A proof may depend on other proofs, but the dependency graph must terminate in independently verified seeds or small primes proved by trial division.

2.1 Multiplicative orders from a factored part of $N-1$

Theorem 2.1 (Pocklington order principle)**.** *Let $N > 1$ be odd and let $F > 1$ divide $N-1$. Suppose that F is fully factored. For every distinct prime $q \mid F$, suppose that an integer a_q satisfies*

$$a_q^{N-1} \equiv 1 \pmod{N}, \qquad \gcd(a_q^{(N-1)/q} - 1, N) = 1. \tag{1}$$

Then every prime $\ell \mid N$ satisfies $F \mid \ell - 1$. Consequently each of the following conditions is sufficient for the primality of N:

$$(F+1)^2 > N; \qquad (2F+1)^2 > N \quad \text{if } F \text{ is odd.} \tag{2}$$

In particular, $F^2 > N$ is a convenient sufficient bound.

Proof. Fix a prime $\ell \mid N$ and a prime $q \mid F$. The first congruence makes a_q a unit modulo ℓ. Put $h = \operatorname{ord}_\ell(a_q)$. Then $h \mid N-1$, whereas the greatest-common-divisor condition implies $h \nmid (N-1)/q$. Thus $v_q(h) = v_q(N-1)$, so $q^{v_q(F)} \mid h \mid \ell - 1$. Applying this argument to every $q \mid F$ gives $F \mid \ell - 1$. Hence $\ell \geq F + 1$. If F is odd, then ℓ is odd and $\ell = uF + 1$ with a positive even u, so $\ell \geq 2F + 1$. A composite positive integer has a prime divisor at most its square root, contradicting either strict bound in (2). □

The gcd exponent is $(N-1)/q$, not $(N-1)/q^{v_q(F)}$. Repeated powers are already accounted for by the order argument. The formulation above does not require $\gcd(F, (N-1)/F) = 1$; some textbook formulations impose this condition to express the factored part using full prime powers. Different primes q may have different witnesses. When a common witness works, its full Fermat power need only be computed once.

Corollary 2.2 (Proth certificate)**.** *Let $N = k2^s + 1$, where k is positive and odd and $k < 2^s$. If $a^{(N-1)/2} \equiv -1 \pmod{N}$, then N is prime.*

Proof. Squaring gives the Fermat congruence. Since N is odd, the reduced power minus one has gcd 1 with N. Apply theorem 2.1 with $F = 2^s$; the inequality $(F+1)^2 > N$ follows from $k < F$. □

For the Mersenne seeds we also use the classical Lucas–Lehmer theorem: if r is an odd prime, $2^r - 1$ is prime precisely when the recurrence $s_0 = 4$, $s_{j+1} = s_j^2 - 2$ has $s_{r-2} \equiv 0 \pmod{2^r - 1}$ [8]. The small exponents used here are checked by trial division before this criterion is applied.

2.2 Lucas ranks from a factored part of $N+1$

Let $U_j(a,b)$ be defined by

$$U_0 = 0, \quad U_1 = 1, \quad U_{j+2} = aU_{j+1} - bU_j, \qquad D = a^2 - 4b.$$

Here a, b are Lucas parameters, not the capital-letter seed primes used later.

Theorem 2.3 (Lucas order principle)**.** *Let $N > 1$ be odd and let the fully factored integer $F > 1$ divide $N+1$. Fix a single pair of integers (a,b), put $D = a^2 - 4b$, and suppose that, for every distinct prime $q \mid F$,*

$$\gcd(N, 2bD) = 1, \qquad U_{N+1}(a,b) \equiv 0 \pmod{N}, \qquad \gcd(U_{(N+1)/q}(a,b), N) = 1 \quad (q \mid F). \tag{3}$$

Every prime $\ell \mid N$ then satisfies

$$F \mid \ell - \left(\frac{D}{\ell}\right), \qquad \ell \equiv 1 \text{ or } -1 \pmod{F}.$$

Thus $(F-1)^2 > N$ suffices for primality. If F is odd, the stronger sufficient bound is $(2F-1)^2 > N$.

Proof. For a prime $\ell \mid N$, let α, β be the distinct nonzero roots of $X^2 - aX + b$ in $\mathbb{F}_{\ell^2}$. The gcd hypothesis justifies $U_j = (\alpha^j - \beta^j)/(\alpha - \beta)$. If D is a square modulo ℓ, the ratio α/β lies in $\mathbb{F}_\ell^*$ and has order dividing $\ell - 1$. Otherwise Frobenius interchanges the roots, so this ratio has norm one and order dividing $\ell + 1$. In both cases, $U_j = 0$ modulo ℓ exactly when that order divides j. The same valuation argument as in theorem 2.1 shows that F divides this order. This proves the residue restriction. If F is odd, the coefficients in the possible forms $uF \pm 1$ must be positive and even, giving $\ell \geq 2F - 1$. The square-root divisor bound finishes the proof. □

A common discriminant matters when F has several distinct prime factors: the sign for a fixed divisor ℓ must be the same for all those factors. Using one pair (a,b) ensures this automatically. A Jacobi condition $(D/N) = -1$ is helpful in searching for parameters and is checked in the reported examples, but is not needed as an additional hypothesis in the particular order statement above. A generic Lucas probable-prime pass without the factor-by-factor gcd conditions is insufficient.

2.3 Elliptic points of a known order

Definition 2.4. A projective point modulo N is *strongly nonzero* if it reduces to a nonidentity point modulo every prime divisor of N. In a projective representation $(X : Y : Z)$, a sufficient condition is $\gcd(Z, N) = 1$. An affine point automatically has this property.

Theorem 2.5 (Known-order elliptic certificate)**.** *Let $N > 1$ satisfy $\gcd(N, 6) = 1$, and let $E : y^2 = x^3 + \alpha x + \beta$ have discriminant coprime to N. Let $F > 1$ be fully factored. Suppose that arithmetic valid modulo every prime divisor of N establishes*

$$[F]Q = O, \qquad [F/q]Q \text{ is strongly nonzero for every prime } q \mid F.$$

If $F > (N^{1/4} + 1)^2$, then N is prime. For a prime $F = P$, it is enough that Q be affine and $[P]Q = O$.

Proof. For every prime $\ell \mid N$, the order of the reduced point divides F but does not divide F/q for any $q \mid F$. Its order is therefore exactly F. Hasse's bound gives

$$F \le \#E(\mathbb{F}_\ell) \le (\sqrt{\ell}+1)^2.$$

If N were composite, choose $\ell \le \sqrt{N}$. The displayed inequality contradicts the stated lower bound on F. □

The numerical certificates use the stronger, entirely integral comparison

$$\bigl(\mathrm{isqrt}(P)-1\bigr)^4 > N, \qquad \mathrm{isqrt}(P) = \lfloor \sqrt{P} \rfloor. \tag{4}$$

Indeed, (4) implies $N^{1/4}+1 < \mathrm{isqrt}(P) \le \sqrt{P}$. Every nontrivial denominator in the affine point additions is required to be coprime to N. A failed inverse or an unresolved exceptional case rejects the certificate. These checks ensure that the operations reduce to genuine group operations over every $\mathbb{F}_\ell$; a nonzero-looking pair of residues alone does not provide an elliptic primality proof. The general fully factored-order version is classical; our three numerical elliptic outputs use prime orders.

3 Arithmetic constructions and cubic extensions

3.1 Linear, power, and mixed-product lifts

For a known prime P, the candidates $cP+1$ and cP^e+1 expose the factored parts P and P^e of $N-1$. When P is odd, c is taken even to make N odd. Taking several certified inputs gives

$$N = c\prod_{i=1}^{r} P_i^{e_i} + 1.$$

The primality of the product is unnecessary: what matters is its complete factorization. If that product is F, the elementary sufficient condition is $F^2 > N$, with the sharper bounds of theorem 2.1 available when useful. The analogous negative candidates expose a known factor of $N+1$ and use theorem 2.3.

For fixed small c, the decimal size of cP^e+1 is approximately e times that of P. This is a statement about size, not about the probability of primality. Increasing e also increases the cost of the modular computations. An arbitrarily high power can be specified without supplying a successful certificate or an efficient search.

Corollary 3.1 (Near-square lift)**.** *Let P be a known odd prime and $0 < h < P$. Set $N = 2P(P-h)+1$. If (1) holds for $q = P$, then N is prime.*

Proof. Every prime divisor is at least $2P+1$, whereas $N < 2P^2+1 < (2P+1)^2$. □

The known factor P can be smaller than $\sqrt{N}$ in this construction. Discarding parity information would miss a valid elementary certificate.

3.2 A positive cubic specialization

For an odd integer $F \ge 3$, define

$$\delta(F) = \begin{cases} 0, & F \equiv 3 \pmod 4, \\ 2, & F \equiv 1 \pmod 4. \end{cases} \tag{5}$$

Proposition 3.2 (Positive cubic lift)**.** *Let $F \geq 3$ be odd and fully factored. Let d be odd with $1 \leq d < F$, and set*

$$N_+(F,d) = F^3 - \delta(F)F^2 - dF + 1.$$

If (1) holds for every prime $q \mid F$, then $N_+(F,d)$ is prime.

Proof. The integer $N = N_+(F,d)$ is odd and greater than one. By theorem 2.1, each of its prime divisors has the form $uF+1$ with $u \geq 2$ even. Since $N < F^3 < (2F+1)^3$, a composite N would have exactly two prime factors, counted with multiplicity. Write

$$N = (uF+1)(vF+1), \qquad u, v \geq 2 \text{ even}.$$

The size bound yields $uv < F$. Also $u + v \leq uv < F$, because $(u-1)(v-1) \geq 1$. Hence

$$\frac{N-1}{F} = uvF + (u+v) = (F - \delta(F) - 1)F + (F - d).$$

Both displayed remainders lie strictly between 0 and F. Uniqueness of Euclidean division gives $uv = F - \delta(F) - 1 \equiv 2 \pmod 4$, contrary to $4 \mid uv$. □

This proof isolates a simple coefficient obstruction in the cubic range of classical $N-1$ criteria. It does not replace those criteria with a new fundamental test. The largest output of the first experiment uses a *composite* fully factored F, demonstrating that two input certificates can genuinely be merged in this specialization.

3.3 A negative cubic specialization

Proposition 3.3 (Negative cubic lift)**.** *Let $F \geq 3$ be odd and fully factored, let d be odd with $1 \leq d < F$, and put*

$$N_-(F,d) = F^3 - \delta(F)F^2 - dF - 1 > 1.$$

If the common-parameter Lucas conditions (3) hold for F, then $N_-(F,d)$ is prime.

Proof. Every prime divisor of $N = N_-(F,d)$ is 1 or -1 modulo F and is at least $2F-1$. Since $N < F^3 < (2F-1)^3$, a composite N has exactly two prime factors, with multiplicity. Because $N \equiv -1 \pmod F$ and $F \geq 3$, the signs must be opposite. Thus

$$N = (uF-1)(vF+1), \qquad u, v \geq 2 \text{ even}.$$

Write $T = uv$ and $w = u - v$. Then $4 \mid T$ and $|w| \leq T/2 - 2$. The latter follows, for instance when $u \geq v$, from $uv/2 - 2 - u + v = (v-2)(u/2+1) \geq 0$. Therefore

$$E := \frac{N+1}{F} = TF + w = F^2 - \delta(F)F - d < F^2.$$

If $T \geq F$, then $T \geq F+1$, since T is even and F odd. This gives $E \geq T(F-1/2)+2 > F^2$, a contradiction. Hence $T < F$ and $|w| < F$. The quotient and remainder of E upon division by F are $F - \delta(F) - 1$ and $F - d$. If $w \geq 0$, uniqueness gives $T = F - \delta(F) - 1 \equiv 2 \pmod 4$. If $w < 0$, it gives $T = F - \delta(F)$, an odd number. Both conclusions contradict $4 \mid T$. □

The two cubic constructions require different witnesses: ordinary multiplicative orders for N_+ and Lucas ranks for N_-. Their analogous coefficient patterns do not imply that the two numbers are simultaneously prime. We do not assert historical priority for either specialization.

3.4 An exact square-test closure throughout the cubic range

The coefficient patterns above are useful because a possible two-factor decomposition is ruled out by parity. More generally, that last obstruction can be decided by one integer square test. This gives a complete elementary closure of the argument after the order witnesses have been established.

Proposition 3.4 (Exact two-factor exclusion)**.** *Let $N > 1$ and $F \geq 3$ be odd, with $N < F^3$.*

(a) *Suppose $F \mid N-1$ and every prime divisor of N is 1 modulo F. Write $(N-1)/F = hF+r$, $0 \leq r < F$. Then N is composite if and only if $r^2 - 4h = z^2 \geq 0$ and $(r-z)/2, (r+z)/2$ are positive even integers.*

(b) *Suppose $F \mid N+1$ and every prime divisor of N is 1 or -1 modulo F. Use centered division*

$$(N+1)/F = TF + w, \qquad -F/2 < w < F/2.$$

Then N is composite if and only if $w^2 + 4T = z^2 \geq 0$ and $(z+w)/2, (z-w)/2$ are positive even integers.

The hypotheses on prime divisors can be supplied by theorems 2.1 and 2.3, respectively.

Proof. In case (a), the divisor lower bound and $N < F^3$ show that a composite has exactly two prime factors. As in proposition 3.2, write $N = (uF+1)(vF+1)$ with $u, v \geq 2$ even. Then $uv < F$ and $u+v < F$, so $uv = h$ and $u+v = r$. These are precisely the roots of $X^2 - rX + h$. Conversely, the stated positive even roots give this factorization into integers greater than one.

In case (b), the same size argument and the sign of N modulo F give $N = (uF-1)(vF+1)$ with $u, v \geq 2$ even. Put $T_0 = uv$ and $w_0 = u - v$. Since $|w_0| \leq T_0/2 - 2$ and $(N+1)/F \leq F^2$, the inequality $T_0 \geq F$ would imply

$$(N+1)/F \geq (F+1)(F-1/2) + 2 > F^2,$$

a contradiction. Hence $T_0 < F$ and $|w_0| < F/2$. Centered-division uniqueness gives $T = T_0$ and $w = w_0$. Thus $w^2 + 4T = (u+v)^2$, with the stated root formulas. The converse again reconstructs the two factors exactly. □

For odd F there is no tie in centered division: from ordinary division $E = hF + r$, take $(T, w) = (h, r)$ if $2r < F$ and $(h+1, r-F)$ otherwise. A square discriminant alone is not enough; positivity and evenness of the reconstructed roots are essential. The two cubic specializations force the potential product to be 2 modulo 4 or odd, immediately excluding these roots. This is a convenient BLS-style arithmetic consequence, without a claim of historical novelty.

3.5 Combining both signs by the Chinese remainder theorem

Proposition 3.5 (Two-sided residue certificate)**.** *Let $F, G > 1$ be coprime odd fully factored integers. Suppose that $F \mid N-1$, $G \mid N+1$, and that the witnesses of theorems 2.1 and 2.3 hold for these two parts. Define $L = 2FG$ and let r be the unique integer with*

$$1 < r < L, \qquad r \equiv 1 \pmod{2F}, \qquad r \equiv -1 \pmod{G}.$$

If $N < L^2$ and either $r^2 > N$ or $r \nmid N$, then N is prime.

Proof. Every prime divisor ℓ is 1 modulo $2F$ and ± 1 modulo G. Consequently $\ell \equiv 1$ or $r \pmod{L}$. A prime divisor at most $\sqrt{N}$ is smaller than L. The representative 1 is not prime, leaving only the integer r. The last hypothesis excludes it. □

This criterion combines information without claiming that both halves are always necessary. In the numerical example of section 6.2, the stronger one-sided Lucas bound already proves primality. Both sets of witnesses were nevertheless checked, so the example is a valid realization of the combined construction, not an example of its necessity.

4 Cyclotomic and repunit certificate transfer

Cyclotomic identities can expose a previously proved prime as a factor of the next candidate minus one. The logical benefit comes from that factorization, not from the mere occurrence of a cyclotomic polynomial.

Proposition 4.1 (A Φ_3 to Φ_7 transfer)**.** *Let $a \geq 2$ be fully factored. Set*

$$P = \Phi_3(a) = a^2 + a + 1, \qquad Q = \Phi_7(a) = a^6 + a^5 + a^4 + a^3 + a^2 + a + 1.$$

The witnesses (1) for $F = a$ prove P prime. Once P is proved, the witnesses for $F = aP$ prove Q prime.

Proof. For the first stage, $P - 1 = a(a + 1)$ and $(a + 1)^2 > P$, so theorem 2.1 applies. Note carefully that $a^2 < P$; the sharper integral lower bound on a prime divisor is essential. For the second stage,

$$Q - 1 = a(a + 1)\Phi_3(a)\Phi_6(a), \qquad (aP)^2 - Q = a^5 + 2a^4 + a^3 - a - 1 > 0.$$

Thus the known factor aP exceeds $\sqrt{Q}$. □

Proposition 4.2 (Repunit transfer)**.** *For integers $a, m \geq 2$, define $R_m(a) = (a^m - 1)/(a - 1)$. If both a and $R_m(a)$ are fully factored, then*

$$N = R_{2m+1}(a), \qquad F = aR_m(a)$$

satisfy $F \mid N - 1$ and $F^2 > N$. Consequently valid $N - 1$ witnesses for the prime factors of F certify N.

Proof. Direct factorization gives $N - 1 = aR_m(a)(a^m + 1)$. Moreover,

$$(a - 1)^2(F^2 - N) = a^{m+2}(a^{m-1} - 2) + a^2 + a - 1 > 0. \tag{6}$$

The first term is nonnegative for $a, m \geq 2$, and the remaining term is positive. Apply theorem 2.1. □

Neither proposition guarantees prime values. If $2m + 1 = uv$ with $u, v > 1$, then $R_u(a)$ is a proper divisor of $R_{uv}(a)$. Thus the tempting index iteration $3 \to 7 \to 15$ stops at 15. The longer pattern $5 \to 11 \to 23 \to 47 \to 95$ likewise stops at 95. Prime indices are necessary, not sufficient, for prime repunit values. A related valid but unexecuted transfer is

$$\Phi_5(a) - 1 = a(a + 1)\Phi_4(a), \qquad \Phi_4(a) = a^2 + 1.$$

If a is even and fully factored and $\Phi_4(a)$ is certified prime, then $F = a\Phi_4(a)$ is fully factored and $F^2 - \Phi_5(a) = a^6 + a^4 - a^3 - a - 1 > 0$ for $a \geq 2$.

5 Reverse norm constructions and elliptic certificates

Suppose that a known prime P is represented as $P = u^2 + Dv^2$. The identity

$$N = Pt^2 + 2ut + 1 = (ut + 1)^2 + D(vt)^2 \tag{7}$$

suggests a curve-order target. In the quadratic ring, put $\pi = 1 + t(u + v\sqrt{-D})$. Then

$$\mathrm{N}(\pi) = N, \qquad \mathrm{N}(\pi - 1) = Pt^2, \qquad N + 1 - \mathrm{Tr}(\pi) = Pt^2.$$

These relations motivate a CM search for a curve and a point R whose multiple $Q = [t^2]R$ has order P. They do not by themselves show that N is prime, that a particular twist has the predicted group order, or that Q is nonzero. The reported certificates finish with direct point arithmetic and theorem 2.5, independently of these predictions.

5.1 Gaussian norms: discriminant -4

Let

$$C = 401713 \cdot 2^{3300} + 1. \tag{8}$$

This 1,000-digit seed passes Proth's test with base 3. Cornacchia reduction provides positive integers u, v, with u even and v odd, such that $C = u^2 + v^2$. The complete integers are embedded in the verifier appendix. For $t \equiv 2 \pmod 4$, (7) gives $N \equiv 5 \pmod 8$, convenient for modular square-root extraction during the search. Four coefficient choices $y^2 = x^3 - dx$, $d \in \{1, 2, 4, 8\}$, were tried after a sieve and a Fermat prefilter.

Two successful parameters were

$$t = 3018, \qquad t = 2^{1650} + 94.$$

The outputs have 1,006 and 1,993 digits, respectively. In both cases the successful curve is

$$E : y^2 = x^3 - 4x,$$

and the supplied affine point Q satisfies $[C]Q = O$, the smoothness test, and (4). The original independent checks also verify the recorded construction $Q = [t^2]R$; this extra construction check is unnecessary for the final primality implication.

5.2 The norm $x^2 + 3y^2$: CM field discriminant -3

Let $A = 2^{1279} - 1$. A representation $A = s^2 + 3r^2$ is obtained with $s > 0$ even and $r > 0$ odd. With

$$t = 2^{638} + 794, \qquad N = At^2 + 2st + 1 = (st + 1)^2 + 3(rt)^2,$$

the resulting 770-digit integer is certified on $E : y^2 = x^3 + 1$. Its explicit affine point has order A; the curve discriminant is coprime to N and the strict Hasse bound holds. The search used affine arithmetic, whereas an independent verifier used Jacobian coordinates and a different scalar-multiplication order, with the necessary unit conditions checked. Here -3 denotes the discriminant of the CM field $\mathbb{Q}(\sqrt{-3})$; the displayed integral binary quadratic form $x^2 + 3y^2$ has discriminant -12.

5.3 Extensions that have not been executed

Other norm forms may be useful when a particular seed does not have the desired representation. For example, discriminant -8 uses $u^2 + 2v^2$; the curve $y^2 = x^3 - 30x - 56$ has $j = 8000$. The 9,439-digit seed below is 3 modulo 8, making this a relevant proposed direction. No prime based on this proposal was searched for or certified in the present work.

Likewise, the fully factored order $F = \prod P_i^{e_i}$ in theorem 2.5 allows several input certificates to be merged elliptically. This remains a classical theoretical option here, not a fourth computed elliptic example. Knowing a proposed curve order is insufficient: the required strongly nonzero multiples must actually be exhibited.

6 The numerical inventory and its provenance

6.1 Foundational seeds

Besides C from (8), write

$$H = 2^{521} - 1, \qquad A = 2^{1279} - 1, \qquad B = 2^{4423} - 1, \qquad p_0 = 14^{512} - 14^{256} + 1.$$

The first three are established Mersenne primes [9], but the verification does not rely on a database lookup: it repeats their Lucas–Lehmer tests. The complete list of foundational inputs is as follows.

Seed	Decimal digits	Independent certificate used here
H	157	Lucas–Lehmer, exponent 521
A	386	Lucas–Lehmer, exponent 1279
B	1,332	Lucas–Lehmer, exponent 4423
C	1,000	Proth, witness 3
p_0	587	$N - 1$ witnesses for $2^{256}7^{256}$

For p_0, put $w = 14^{256}$. Then $p_0 = w^2 - w + 1 = \Phi_{1536}(14)$, $p_0 - 1 = w(w - 1)$, and $w^2 > p_0$. The witnesses are $a_2 = 11$ and $a_7 = 2$. Both full power residues are 1, and both required gcds are 1. Base 2 alone is not the advertised certificate for the 2-part. This seed appeared in a PrimeGrid discussion on 27 December 2014, including subsequent reports of proof by existing software [10]. It is not presented as a newly discovered seed.

6.2 First experiment: ten outputs

Table 1 preserves the ten outputs behind the earlier statement that the largest constructed result had 4,153 digits. Every row is a proved prime. The notation T in row I5 is defined exactly by

$$\begin{aligned} L &= 2AC, \qquad r = 1 + 2A\big((-A^{-1}) \bmod C\big), \\ T &= r + \big(\lfloor C/A \rfloor + 964\big)L. \end{aligned} \tag{9}$$

The inverse and the least nonnegative residue are taken modulo C.

The ordinary $N - 1$ and positive cubic rows use base 2 for every distinct prime factor in their selected factored parts. Row I1 uses $F = A^2$ and Lucas parameters $(a, b) = (3, 1)$, so $D = 5$. Row I2 uses the Proth witness 3. Row I5 uses base 2 on the A-part of $T - 1$ and Lucas parameters $(5, 1)$, with $D = 21$, on the C-part of $T + 1$.

Table 1: The first ten certified outputs. Times are observed wall seconds; search includes its successful certificate, while verification is a separate standard-library Python execution.

ID	Exact integer	Digits	Search	Verify
I1	$776A^2 - 1$	773	0.384	0.540
I2	$A2^{2158} + 1$	1,035	1.158	0.084
I3	$A^3 - 1971A + 1$	1,156	1.501	0.202
I4	$2482A^4 + 1$	1,544	4.515	0.543
I5	T of (9)	1,999	5.112	6.837
I6	$722BC + 1$	2,334	3.434	1.757
I7	$2B(B - 868) + 1$	2,664	9.850	1.821
I8	$C^3 - 2C^2 - 1315C + 1$	2,998	9.157	2.975
I9	$B^3 - 15751B + 1$	3,995	211.692	7.122
I10	$(AC)^3 - 6529AC + 1$	4,153	88.080	13.183

For I10, set $F = AC \equiv 3 \pmod 4$. The complete nontrivial witness conditions are

$$2^{N-1} \equiv 1 \pmod N, \qquad \gcd(2^{(N-1)/A} - 1, N) = \gcd(2^{(N-1)/C} - 1, N) = 1.$$

Together with proposition 3.2, they certify the 4,153-digit output. The final successful certificate in the search took approximately 0.94 seconds; locating that candidate took 88.08 seconds. These costs answer different questions.

For I5, the calculations establish $L^2 > T$ and $r^2 > T$, as required by proposition 3.5. They also establish $(2C - 1)^2 > T$. Thus the stronger one-sided Lucas criterion already suffices, even though $A^2 < T$ and $C^2 < T$. The latter two inequalities alone must not be used to claim an essential advantage of the two-sided method.

6.3 The squaring chain through 9,439 digits

The earlier conversation supplied the following chain:

$$p_1 = 438p_0^2 + 1, \qquad p_2 = 13696p_1^2 + 1,$$
$$p_3 = 5494p_2^2 + 1, \qquad p_4 = 13258p_3^2 + 1.$$

The respective digit counts are

$$587 \longrightarrow 1177 \longrightarrow 2357 \longrightarrow 4718 \longrightarrow 9439.$$

For each step $N = cP^2 + 1$, the selected fully factored part is $F = P^2$; $P^4 > N$, and base 2 satisfies

$$2^{N-1} \equiv 1 \pmod N, \qquad \gcd(2^{cP} - 1, N) = 1. \tag{10}$$

The gcd exponent is cP, not c. Each P is proved before its child is accepted. The last integer has 31,354 bits. Its SHA-256 identifier, computed from its ASCII decimal expansion without a terminal newline, is

```
82ddc0fab24f40e0c8c777c2f2e2138fec8
d9bc505e21eb414858111ca910089
```

The printed line break is not part of the digest.

The original search times of these coefficients are unavailable, and no search-time inference is made from a later verification. The coefficients are therefore attributed to the prior conversation, followed by explicit independent reconstruction. In particular, the 9,439-digit result is not

accepted merely because it was reported in a chat message. It is included in the full standalone verification in section 13.

Absence from a selected prime database would not establish first discovery. The present manuscript makes no global novelty claim for p_3, p_4, or the other specific integers. The relevant achievement is a verifiable certificate, not the historical priority of a decimal string.

6.4 Second experiment: nine principal and two auxiliary outputs

Let $a_H = 78299H$ and $a_A = 48909A$. The multipliers factor as $78299 = 13 \cdot 19 \cdot 317$ and $48909 = 3 \cdot 7 \cdot 17 \cdot 137$. The second experiment gives the nine principal primes in table 2. The symbols u, s refer to the explicit norm representations in section 5.

Table 2: Principal outputs of the second experiment. A time spanning two rows is one shared search, not two independent timings.

ID	Exact integer or parameter	Digits	Search seconds
II1	$\Phi_3(a_H)$	324	1.352 (pair)
II2	$\Phi_7(a_H)$	971	same run
II3	$\Phi_3(a_A)$	780	6.151 (pair)
II4	$\Phi_7(a_A)$	2,339	same run
II5	$Ct^2 + 2ut + 1$, $t = 3018$	1,006	2.734
II6	$Ct^2 + 2ut + 1$, $t = 2^{1650} + 94$	1,993	4.351
II7	$At^2 + 2st + 1$, $t = 2^{638} + 794$	770	0.394
II8	$C^3 - 2C^2 - 9053C - 1$	2,998	71.453
II9	$2214p_3 + 1$	4,721	57.640

Two additional first-stage primes arose during the cyclotomic searches:

$$\Phi_3(260H) \quad (319 \text{ digits}), \qquad \Phi_3(4683A) \quad (778 \text{ digits}).$$

They are retained as auxiliary outputs, making eleven rather than nine certified integers in this experiment. Their searches are part of the shared runs in table 2; those times must not be multiplied by the number of outputs.

For II1–II2 all factor witnesses are base 2. For II3 every witness is base 2; in II4 the factor 3 uses base 5, while the remaining factors use base 2. In the first auxiliary, the factor 5 uses base 3; in the second auxiliary, the factor 7 uses base 5. All other auxiliary witnesses use base 2.

The negative cubic II8 uses $(a, b) = (9, 1)$ and $D = 77$. Its verified conditions are

$$\gcd(154, N) = 1, \quad (77/N) = -1, \quad U_{N+1}(9, 1) \equiv 0 \pmod{N}, \quad \gcd(U_{(N+1)/C}(9, 1), N) = 1.$$

The additional Lucas proof on the successful candidate took about 0.715 seconds. Independent generic-matrix Lucas verification took 27.469 seconds. This output is different from the 2,998-digit positive cubic I8.

For II9, $p_3^2 > N$ and the witness is base 2: $2^{N-1} \equiv 1 \pmod{N}$ and $\gcd(2^{2214} - 1, N) = 1$. This is a modest increase over p_3, not an improvement on the size of p_4. The two CM(-4) searches tested 91 and 3 candidates surviving their sieves, respectively; the CM(-3) search tested 10. These favorable finite runs do not imply constant expected search time.

6.5 Search and verification protocol

The principal search arithmetic used `gmpy2 2.3.2` with `GMP 6.3.0`. Independent verifiers used Python 3.12.14 standard-library integers, modular `pow`, and `math.gcd`. Lucas search code

used specialized doubling identities; independent verification used generic 2×2 matrix powers. The CM(-3) check also changed the point-coordinate system and scalar-multiplication order. These are independent implementations, not a claim of formal verification of Python, GMP, hardware, or the operating system.

The search workflow is: specify a finite parameter window; sieve against small primes; apply a cheap rejection test to survivors; attempt the full certificate; then verify that certificate with a separate implementation. The search does not promote a sieve survivor or a Fermat probable prime to the status of a prime. Different routines include different setup costs in their timers, and concurrent jobs shared resources. The reported wall times are observations from individual runs, not a controlled benchmark of the mathematical methods.

For perspective, independent checks of the three elliptic outputs took 2.936, 22.350, and 0.707 seconds, including their specified ancestor and construction checks. The first packet's complete independent verification took 35.327 seconds. Summing every per-output time from different scripts would double count some seed and chain checks.

7 An implementation improvement: shared exponents

The proof obligations in (10) need not be evaluated as two unrelated exponentiations. More generally, for $N = cP^e + 1$, define

$$z = a^{cP^{e-1}} \bmod N.$$

Then $a^{N-1} \equiv z^P \pmod N$. The two exact checks are therefore

$$z^P \equiv 1 \pmod N, \qquad \gcd(z-1, N) = 1. \tag{11}$$

This is algebraically identical to the original certificate. It is intermediate-result reuse, not a weaker test or a new primality theorem. A ladder $a^c, (a^c)^P, \ldots$ supplies another implementation of the same relations.

On the fixed 9,439-digit candidate p_4 and base 2, one GMP measurement gave

Implementation	Seconds	Sum of exponent bit lengths
Two separate powers	4.363427	47,038
Shared power of (11)	3.330762	31,354

The measured reduction is approximately 23.7%, or a factor of 1.310 in elapsed time. There was one repetition per method. Exponent bit-length sums are a rough workload indicator, not an exact operation count; windows, multiplication algorithms, and modular reduction affect runtime. In particular, this measurement does not establish a 23.7% reduction in total search cost or a universal speedup on other hardware.

A complementary practical improvement is to retain a proof dependency graph and cache each validated ancestor. The same prime should not be reproved for every child in a batch. Multiple witnesses using one base can also share its full Fermat residue. These optimizations reduce duplicated work while leaving every mathematical proof obligation explicit.

8 Record-scale inputs and stopped searches

8.1 Mersenne coefficients inside Proth candidates

Let $M_r = 2^r - 1$ and consider $N = M_r 2^s + 1$. If $s \geq r$, this is a Proth number since its odd coefficient is smaller than 2^s. If $r \geq 3$ is odd, odd s makes $3 \mid N$. For $r \geq 3$ odd and $s \geq r$

even, one has $N \equiv 5 \pmod{12}$. Consequently, when N is prime, quadratic reciprocity gives $(3/N) = -1$; For $r \geq 3$ odd and $s \geq r$ even, Euler's criterion and corollary 2.2 show that

$$N \text{ is prime} \iff 3^{(N-1)/2} \equiv -1 \pmod{N}.$$

Thus the witness base is fixed for this family. The modular exponentiation still has to be carried out.

If M_r is already certified prime and $1 \leq s < r$, then $M_r^2 > N$. The same candidate can instead be certified using its known M_r factor in $N - 1$. This gives a clean choice of proof mechanism on both sides of the parameter boundary, not automatic primality at either side.

Proposition 8.1 (A Fermat-number obstruction)**.** *Let $r > 1$ be odd and set $h = v_2(r-1)$. For any positive s with $j = v_2(s) < h$,*

$$2^{2^j} + 1 \mid (2^r - 1)2^s + 1.$$

This divisor is proper, so the candidate is composite.

Proof. Modulo $F_j = 2^{2^j} + 1$, one has $2^{2^j} \equiv -1$ and $2^{2^{j+1}} \equiv 1$. Since $r \equiv 1 \pmod{2^{j+1}}$, we obtain $2^r - 1 \equiv 1$. Since $s/2^j$ is odd, $2^s \equiv -1$. The product plus one vanishes. Finally $2^j < r$ and the candidate exceeds 2^r, so the displayed divisor is smaller than the candidate. The argument does not assume that the Fermat number itself is prime. □

For $r = 136279841$, one has $h = 5$. Therefore every exponent s not divisible by 32 is excluded by one of $3, 5, 17, 257, 65537$. The first exponent satisfying both $s \geq r$ and $32 \mid s$ is 136279872. Surviving this exact filter does not imply primality.

8.2 What can be decided immediately at record scale

The source snapshot consulted on 5 November 2026 records

$$M = 2^{136279841} - 1 \quad (41{,}024{,}320 \text{ digits}), \qquad R = 10223 \cdot 2^{31172165} + 1 \quad (9{,}383{,}761 \text{ digits})$$

as a known Mersenne record and a known Proth record, respectively [9, 11]. They are contextual external inputs, not additional nodes in the locally reverified inventory. Small modular arithmetic proves the following transformed candidates composite.

Candidate	Proper divisor	Candidate	Proper divisor
$2M + 1$	3	$2R + 1$	5
$4M + 1$	5	$2R - 1$	3
$M^2 - M + 1$	7	$(R-1)^2 + 1$	397
$2M(M-1) + 1$	2953	$R^3 - R + 1$	4483

The modular computations do not require writing the enormous decimal integers. For example, $M \bmod \ell$ can be computed as $2^{136279841} \bmod \ell - 1$.

The previously known prime $7 \cdot 2^{20267500} + 1$ [12] fits the combined form $(2^3 - 1)2^{20267500} + 1$ and has 6,101,127 digits. Such an existing example demonstrates that the family reaches large sizes, not that the construction has produced a new record. Similarly,

$$2^{53689908}((2^{82589933} - 1) + 1) - 1 = 2^{136279841} - 1$$

is an exact re-expression of an already known output prime.

8.3 Bounded experiments that did not produce a prime

A separate Proth search considered $N_k = k2^{32000000} + 1$ for odd $1 \le k < 2^{24}$. Among the 8,388,608 initial parameters, sieving by primes at most 10^9 eliminated 7,933,515 and retained 455,093. No record-sized primality proof finished. A short PRST [13] run estimated approximately 12.17 hours for one full test under its measured settings; that was an extrapolation, not an observed completed test. The attempted computation was stopped after 61,440 of roughly 32 million iterations. It supplies no prime certificate. The sieve bound also makes no statement about larger possible factors.

At the much smaller 9,439-digit seed p_4, bounded searches of $2kp_4 + 1$ and $2kp_4 - 1$ tested 15 and 13 sieve survivors in approximately 57.30 and 41.86 seconds, respectively, without obtaining a certified output. These are finite unsuccessful attempts. They neither prove compositeness of every member of either family nor prove that the chain cannot be continued. All reported searches are completed or stopped; no ongoing background search is implied.

9 A constructive connection with Collatz prime trajectories

9.1 Repository provenance and the usable construction

The *Verifiable Mathematical Reasoning Corpus* was consulted at the following commit, dated 5 November 2026, 13:06:15 UTC:

`5936298ed4bacb7fb1423b889f02278fa521ea16`

Its manuscript numbered [072], `unified_collatz_recurrences.tex`, contains a useful affine array and a discussion of primality-certificate obstacles in inverse Mersenne orbits [15]. The item numbered [048] has the same Git blob and is a duplicate source, not independent confirmation. No theorem elsewhere in the repository is needed to justify any numerical primality certificate in this paper.

Define the standard Collatz map by $\mathcal{C}(n) = 3n + 1$ for odd n and $\mathcal{C}(n) = n/2$ for even n. The corpus array can be specialized so that its orbit relation and a useful known factor of $N - 1$ are designed together.

Proposition 9.1 (Certificate-compatible Collatz array)**.** *Let $P > 3$ be a certified prime, let s, K, r be positive integers, and put $F = P^s$. Suppose that $2r4^K < F$. For $0 \le i \le K$, define*

$$N_i = 2rF\,4^i 3^{K-i} + 1.$$

Then $F \mid N_i - 1$, $N_i < F^2$, and

$$3N_i + 1 = 4N_{i-1} \quad (1 \le i \le K). \tag{12}$$

For any index i, the $N - 1$ witnesses for the single prime P certify N_i prime. If several indices pass, the corresponding primes lie on one genuine Collatz trajectory in decreasing index order, with exactly three standard steps per index difference.

Proof. Write $k_i = 2r4^i 3^{K-i}$. Then $k_i \le 2r4^K < F$ and $N_i = k_i F + 1 \le (F-1)F + 1 < F^2$. Apply theorem 2.1 to $F = P^s$. Identity (12) follows by direct expansion. Each N_i is odd, so the right side has exactly two factors of 2. Thus the steps are $N_i \mapsto 4N_{i-1} \mapsto 2N_{i-1} \mapsto N_{i-1}$, with no unverified valuation assumption. □

This construction reuses P as a certificate factor; it does not assert that the trajectory ends at P. It also does not assert that all the N_i are prime. Choosing s larger makes the finite certifiable range larger, while increasing the size and cost of the candidates.

9.2 An additional executed example: two 391-digit primes

The specialization $P = A = 2^{1279} - 1$, $s = K = 1$ was searched while preparing this manuscript. At $r = 98723$ it produced

$$\boxed{Q_0 = 592338A + 1, \qquad Q_1 = 789784A + 1.} \tag{13}$$

Both integers have 391 decimal digits and both are prime. For each $j \in \{0, 1\}$, the verified certificate is

$$A^2 > Q_j, \qquad 2^{Q_j - 1} \equiv 1 \pmod{Q_j}, \qquad \gcd(2^{(Q_j-1)/A} - 1, Q_j) = 1.$$

The reduced exponents are the small integers 592338 and 789784. Their precise orbit relation is

$$Q_1 \longmapsto 4Q_0 \longmapsto 2Q_0 \longmapsto Q_0.$$

The bounded search took approximately 1.1151 seconds and examined 1,484 parameters surviving the joint sieve before success. The first candidate's Fermat condition passed at 42 of those parameters; one pair passed both. A separate standard-library Python implementation rechecked the seed by Lucas–Lehmer, both exact certificates, hashes, sizes, and the orbit relation in approximately 0.0148 seconds. These are measured finite computations, not expected-time estimates or claims of the smallest possible multiplier.

The baseline inventory is therefore

$$\underbrace{5}_{\text{foundational seeds}} + \underbrace{4}_{\text{prior chain lifts}} + \underbrace{10}_{\text{first experiment}} + \underbrace{11}_{\text{second experiment}} + \underbrace{2}_{\text{Collatz pair}} = 32.$$

The pair supplies a concrete use of the suggested corpus. Its importance is the simultaneous orbit relation and checkable primality, not a new largest prime or a result on the eventual behavior of all Collatz orbits.

9.3 Why inverse ancestors of a fixed Mersenne seed are different

Proposition 9.2 (A one-step obstruction)**.** *Let $r \geq 3$ be an odd integer, put $M_r = 2^r - 1$, and let $a \geq 1$ be an integer. The odd integer $N = (4^a M_r - 1)/3$ has accelerated Collatz successor M_r, but*

$$M_r \nmid N - 1, \qquad M_r \nmid N + 1, \qquad v_2(N - 1) = \begin{cases} 3, & a = 1, \\ 2, & a \geq 2. \end{cases}$$

Proof. Since $M_r \equiv 1 \pmod 3$, N is an odd integer and $3N + 1 = 4^a M_r$ has exactly $2a$ factors of 2. Moreover,

$$3(N - 1) = 4^a M_r - 4, \qquad 3(N + 1) = 4^a M_r + 2.$$

Divisibility by $M_r > 4$ would force $M_r \mid 4$ or $M_r \mid 2$, respectively, which is impossible. Finally $v_2(N - 1) = 2 + v_2(4^{a-1} M_r - 1)$. This is 2 for $a \geq 2$ and 3 for $a = 1$, since $v_2(M_r - 1) = 1$. □

The proposition explains a structural limitation of this particular inverse orbit construction: neither the large seed factor nor a large power of two is automatically available in $N - 1$, and the seed factor is absent from $N + 1$ as well. It does not preclude other factorizations or an elliptic certificate for the same candidate.

The corpus also discusses existence of long finite sequences of prime visits using prime-distribution results for admissible linear forms, related to Maynard's work [14]. Those statements allow the starting and ending primes to vary. They do not supply an arbitrary prescribed giant prime as the endpoint, a search bound within $N < F^2$, or an infinite decreasing sequence of positive integers. The finite construction and proof above do not depend on such an existence theorem, nor do they address the Collatz conjecture.

10 The original Mills-constant motivation and its limits

The investigation began with prime-representing constants. For an exponent sequence (C_k), write $\xi(C_k)$ for the least real $\xi > 1$ such that $\lfloor \xi^{C_k} \rfloor$ is prime for every relevant k, whenever that minimum exists. Saito proved that Mills' constant $\xi(3^k)$ is irrational [16]. The version-v3 preprint on variants proves its transcendence under the Density Hypothesis and gives unconditional arithmetic results for several other exponent families [17]. These results concern constants, not a numerical primality oracle.

An expression $3^k + n$ or $3^k - n$ is an integer exponent and is therefore algebraic. A transcendence question in this context concerns the associated *real constant* $\xi(3^k \pm n)$, with positive indices and a well-defined minimum, rather than the integer $3^k \pm n$. Different shifts normally define different minimization problems. Neither the certificates in this paper nor the act of combining four shifted constants establishes a new transcendence result for any previously unresolved shift.

Even if a prime-representing constant is known abstractly, computing an individual integer part requires adequate certified precision. For a rational enclosure $L < \xi < U$, the equality $\lfloor L^E \rfloor = \lfloor U^E \rfloor$ identifies that integer part. An irrationality or transcendence theorem alone does not supply such an enclosure or the computational resources to raise it to a huge exponent.

Nor is primality preserved by arbitrary combinations of prime outputs. Products of two primes are composite; sums of two odd primes are even and exceed two; a nontrivial power of a prime is composite. Towers built from odd prime bases remain odd, so adding or subtracting one produces an even integer larger than two in the nontrivial cases. Useful combinations must instead carry a structural certificate, such as the mixed factored part AC, the cyclotomic transfer, or a point of known elliptic order. That is the precise connection between the original motivation and the methods developed here.

11 A catalogue of candidate designs

Table 3 collects the diverse designs considered during the investigation. It is a catalogue of constructions, not a count of independent new primality theorems. Here P and the P_i are certified primes, $F = \prod P_i^{e_i}$ is fully factored, and all candidates must be odd and greater than one. Every successful row requires actual witnesses and its applicable bound or exclusion argument.

Table 3: Candidate designs and the status of their use in this work.

Design	Certificate information	Status here
$cP + 1$	Known factor $P \mid N - 1$	Executed: II9 and Collatz pair
$cP^e + 1$	Known prime power in $N - 1$	Executed: chain and I4
$c\prod P_i^{e_i} + 1$	Merge several certified factors	Executed: I6
$P^e 2^s + 1$, $P^e < 2^s$	Proth certificate; coefficient need not be prime	Executed special case I2
$(aP)^{2^r} + 1$, $2 \le a < P$ even, $r \ge 0$ integral	Use $P^{2^r} \mid N - 1$	Valid proposal; not searched
$2P(P - h) + 1$, $0 < h < P$	Parity-strengthened divisor bound	Executed: I7
$F^3 - \delta F^2 - dF + 1$	$N - 1$ orders and cubic exclusion	Executed: I3, I8–I10

Design	Certificate information	Status here
$cP^e - 1$	Known factor of $N + 1$	Executed: I1
$c\prod P_i^{e_i} - 1$	Common-discriminant Lucas ranks	Valid proposal; not searched
$P^e 2^s - 1$	Appropriate Lucas/LLR conditions and bounds	Family considered; no new output
$F^3 - \delta F^2 - dF - 1$	Lucas ranks and cubic exclusion	Executed: II8
$\Phi_3(a)$, $\Phi_6(a)$ with factored a	Factor a in $N - 1$; exact divisor bound	Φ_3 executed; Φ_6 seed
$R_m(a) \to R_{2m+1}(a)$	Transfer factor $aR_m(a)$	Executed: $\Phi_3 \to \Phi_7$
$\Phi_4(a) \to \Phi_5(a)$	Transfer factor $a\Phi_4(a)$	Proved setup; not searched
$N \equiv 1 \pmod F$, $N \equiv -1 \pmod G$	Combine two residue restrictions	Executed: I5; one side also suffices
$Pt^2 + 2ut + 1$, $P = u^2 + v^2$	Prime-order point on a $j = 1728$ curve	Executed: II5–II6
$Pt^2 + 2ut + 1$, $P = u^2 + 3v^2$	Prime-order point on a $j = 0$ curve	Executed: II7
Other CM norm forms, e.g. $u^2 + 2v^2$	Appropriate curve, twists, and actual point	Proposed CM field discriminant -8
Elliptic fully factored order F	Strongly nonzero $[F/q]Q$ for every q	Classical theorem; not executed
$2rP^s 4^i 3^{K-i} + 1$	$N - 1$ certificate and Collatz relation	Executed: two additional primes
Factor extraction from $2^P - 1$	A prime divisor has order P	Larger factors guaranteed; extraction not executed

For the generalized Fermat-shaped row, take an integer $r \geq 0$, an even integer $2 \leq a < P$, and put $e = 2^r$ and $F = P^e$. Then $F^2 > (aP)^e + 1$, so the factored-part setup is valid. For the factor-extraction row, a prime divisor ℓ of $2^P - 1$ has $\mathrm{ord}_\ell(2) = P$, because P is prime and order 1 is impossible. Thus $\ell \equiv 1 \pmod{2P}$ for odd P, and in particular $\ell > P$. This guarantees larger prime divisors, but finding and certifying such a divisor may be much harder than specifying the enormous input to be factored.

12 Limitations and concrete further questions

The finite results leave several substantive questions open. First, which construction minimizes the cost per accepted prime at a prescribed output size? Answering this requires repeated experiments with matched hardware, parameter windows, sieve policies, and verification rules. The present single-run timings cannot rank all methods universally.

Second, the norm approach could use other discriminants or products of known prime orders. The difficulty is constructive: one must find an appropriate candidate and an actual point with the required order. The Hasse inequality alone does not create that point. Similarly, the cyclotomic transfer graph is constrained by algebraic compositeness of repunit indices; a formal iteration must be checked before launching an expensive search.

Third, one can search for several certified prime entries in the same Collatz-compatible array, perhaps with prescribed spacing. The certificate theorem remains exact, but simultaneous primality in a particular bounded array is not guaranteed. The existence of arbitrarily long finite prime patterns in a varying family must not be read as a construction from a fixed input in a fixed search interval.

Finally, no finite collection of successful $cP^2 + 1$ lifts proves that the procedure continues indefinitely with a bounded multiplier. Dirichlet's theorem on an unrestricted arithmetic progression does not by itself put a prime in the particular finite range where the selected known factor is large enough for the desired certificate. The examples show that rigorous reuse works, and the formulas suggest further experiments; neither point settles an infinitude conjecture for these specific families.

13 Reproducibility, provenance, and the complete verification record

The single TEX source contains every numerical datum needed to reproduce the certificate verification: symbolic constructions of all 32 baseline integers, small witness bases and Lucas parameters, the two norm representations, the three affine elliptic points, exact digit counts, and decimal SHA-256 identifiers. The long decimal expansions of the candidate integers are unnecessary because their formulas are exact. The source compiles without external figures, a bibliography file, or an input data file.

The verifier in appendix A uses only the Python standard library and performs no network request or probable-prime acceptance. It first checks formula reconstruction and metadata, then verifies the seeds and walks the certificate dependency graph. All small factors are checked by trial division. A named large factor is available only after its own certificate has passed. Errors raise exceptions explicitly; the decisive conditions are not implemented using assertions that disappear under Python's optimization flag.

A final execution of the exact consolidated listing completed successfully with status `ALL_32_PRIMES_CERTIFIED`. It checked all 32 nodes in approximately 138.746 seconds with Python standard-library arithmetic, including p_4 and the new Collatz pair. This is one observed verification run; it is distinct from the historical search timings and from earlier per-packet verification runs.

The original two computational packets were also audited for integrity: their 31 and 57 manifest entries, respectively, matched the stored file hashes. Their 32 decimal files represented 30 distinct integers because A and C were repeated inputs. The extra Collatz pair is supplied directly in the consolidated source. Hashes identify the exact intended integers; they are not themselves primality evidence.

For historical attribution, the cyclotomic seed p_0 is public prior work, the four chain coefficients were supplied in the earlier conversation, and the first and second construction experiments were explicitly executed and independently checked. The final Collatz pair was searched and certified during manuscript preparation. No claim that any of these specific integers was previously unknown is needed for the mathematical conclusions.

How to compile and verify

Compile the one source with `pdflatex` twice, or use:

```
latexmk -pdf Certified_Prime_Lifting.tex
```

To recover the verifier, copy the Python listing in appendix A into a file, or extract its marked listing from the TEX source as follows:

```
from pathlib import Path
import re
s = Path("Certified_Prime_Lifting.tex").read_text()
```

```
pat = (r"\\begin\{lstlisting\}\[label=lst:verifier\]\n"
       r"(.*?)\\end\{lstlisting\}")
m = re.search(pat, s, re.S)
if m is None:
    raise ValueError("Verifier listing was not found")
Path("verify_prime_lifting.py").write_text(m.group(1))
```

Run `python3 verify_prime_lifting.py` with Python 3.11 or later. Success prints `ALL_32_PRIMES_CERTIFIED`.

14 Revision ledger and the continuing computation

This register revision retains the 32 certified integers of the original inventory and the eighteen explicitly specified algorithmic modules, labelled A01–A18, of the preceding expanded edition. It updates the large-prime inventory and its reproducible certificates. The counts describe different objects: the baseline has 32 distinct prime integers and 21 rows of candidate designs; it did not contain 32 independent primality algorithms. An additional algorithm need not output a new prime: some modules construct candidates, some exclude composites, some certify a supplied candidate, and some optimize or audit the computation. Each module states its output contract.

14.1 Complete register of certified primes with at least 18,000 digits

The preceding algorithm edition already included the two extensions $p_5 = 22330p_4^2 + 1$ and $p_6 = 5936p_5 + 1$. This update consolidates every certified integer of at least 18,000 decimal digits in the retained project record, incorporates the later 62988-coefficient certificate, and completes the certificates of two previously recorded probable-prime hits. It does not perform a new coefficient search.

For a self-contained specification, define the common ancestral chain by

$$\begin{aligned} p_0 &= 14^{512} - 14^{256} + 1, \\ p_i &= k_i p_{i-1}^2 + 1 \quad (1 \le i \le 5), \\ (k_1, k_2, k_3, k_4, k_5) &= (438, 13696, 5494, 13258, 22330), \end{aligned} \tag{14}$$

and define the four linear leaves by

$$r_c = cp_5 + 1, \qquad c \in \{5936, 38456, 59790, 62988\}. \tag{15}$$

We retain the older name $p_6 = r_{5936}$ and write $r = r_{62988}$ when referring to the largest certified integer. All four leaves have parent p_5. In particular, r is not obtained by applying the coefficient 62988 to p_6.

Table 4: Complete large-record register in this revision, in increasing numerical order. Every row is certified prime. The two newly certified rows use candidates found in the earlier search.

ID	Exact integer	Digits	Bits	Certificate provenance
p_5	$22330p_4^2 + 1$	18,882	62,722	Already in the preceding edition.
p_6	$5936p_5 + 1$	18,885	62,735	Already in the preceding edition.
r_{38456}	$38456p_5 + 1$	18,886	62,737	Earlier PRP hit; full certificate completed in this revision.
r_{59790}	$59790p_5 + 1$	18,886	62,738	Earlier PRP hit; full certificate completed in this revision.
r_{62988}	$62988p_5 + 1$	18,886	62,738	Certified in the intervening record upgrade; now incorporated.

The strict ordering is

$$p_5 < p_6 < r_{38456} < r_{59790} < r_{62988}.$$

The final three integers have the same decimal length but different values. Thus three entries have 18,886 digits, and r_{62988} remains the numerical maximum. This is a maximum within the recorded project computations; historical first discovery and a world record are not asserted.

14.1.1 Complete proof obligations

For p_5, the factored part of $p_5 - 1$ is $F = p_4^2$. The exact checks are

$$p_4^4 > p_5, \qquad 2^{p_5-1} \equiv 1 \pmod{p_5}, \qquad \gcd(2^{22330p_4} - 1, p_5) = 1. \tag{16}$$

For every coefficient c in (15), use $F = p_5$ and check

$$p_5^2 > r_c, \qquad 2^{r_c-1} \equiv 1 \pmod{r_c}, \qquad \gcd(2^c - 1, r_c) = 1. \tag{17}$$

All these inequalities and residues were evaluated exactly. Each full Fermat residue was 1 and each reduced-power gcd was 1.

Proposition 14.1 (Certified large-record register)**.** *The five integers in Table 4 are distinct primes. Their complete dependency graph, excluding the elementary primes* 2 *and* 7*, contains ten nodes:* $p_0, \ldots, p_5$ *and the four leaves* r_c*.*

Certificate proof. With $x = 14^{256}$, the seed is $p_0 = x^2 - x + 1$ and its factored part is $x = 2^{256}7^{256}$. Its square exceeds p_0. Bases 11 for the prime factor 2 and 2 for 7 satisfy (1). At each quadratic step, base 2, the already certified factor p_{i-1}^2, and the reduced exponent $k_i p_{i-1}$ satisfy the same criterion. This proves $p_0, \ldots, p_5$ in dependency order. The verified conditions (17) then certify all four siblings by theorem 2.1. The formula and positive distinct coefficients prove the stated ordering and distinctness. Appendix G contains the complete standalone verifier, including elementary checks of 2 and 7, all ancestors, all four leaves, and all five large-integer fingerprints. □

The ordinary quadratic gcd exponent in (16) is $22330p_4$, not 22330. For a linear leaf, the ordinary Pocklington exponent really is the small coefficient c. The large full Fermat exponent remains necessary in both cases. A probable-prime report by itself is not used as the certificate.

14.1.2 Status transitions and verification record

The preserved earlier linear-search ledger contains successful probable-prime screens at coefficients 5936, 38456, 59790, and 62988. At that time, only the 5936 output was promoted to a full certificate. A later upgrade completed the 62988 certificate. For this register revision, the remaining two saved hits were checked with both exact conditions in (17); both passed. Their integers were already known to the experiment, while their complete certificates are new to this revision. No claim of historical novelty for either integer follows.

The verification was executed in two stages. A standalone eight-node run reconstructed and certified $p_0, \ldots, p_5, p_6, r_{62988}$. Its recorded GMP-backed program time was 55.163488 seconds and process wall time was 55.261099 seconds. The two remaining leaves were then certified in 28.257082 seconds of process wall time, using the freshly certified, hash-matched p_5 without repeating its parent proofs. Their individual certificate checks took 14.081102 and 14.080691 seconds. The arithmetic and proof parameters of these successful stages are consolidated in the ten-node standalone verifier in Appendix G; its source identity and execution scope are stated there. These times are verification observations, not new search times or a comparison of algorithms.

The designated main inventory now has $32 + 5 = 37$ distinct certified integers: the retained 32-integer baseline plus these five large entries. The ancestors $p_0, \ldots, p_4$ are already in the baseline and are not added again. The small, sometimes overlapping demonstrations of A01–A18 remain separate from this designated main-inventory count. The 21 original candidate-design rows and eighteen additional module labels are also unchanged counts of different objects.

14.1.3 Canonical decimal identities

The following SHA-256 values are calculated over the full decimal ASCII expansion, without a sign, separator, space, leading zero, or trailing newline. The formulas specify the integers; these hashes detect accidental substitution and do not replace the primality proofs.

p_5

```
7407dab961f68eb15854c74700a8007bf30d89410a6beeb1d2fc7d365f4b7ce8
```

$p_6 = r_{5936}$

```
1fd5cb8c8470b74c24c163501a2775693b19707db62c65e6b074a1e56ff7cbd3
```

r_{38456}

```
00d10910b2a4d068db8b0cc164956d1983cc62a1d3950a0a0d25548402b51d36
```

r_{59790}

```
a9652db4b6bef070ce335537f7634a68e43e689d768d2d4cbeef3f536a0d28fa
```

$r = r_{62988}$

```
ca4c8532ec289a68d0fe9769f87f3797aa560f9f1a141866da17b917c11c1081
```

14.2 The interrupted 37,775-digit search

The next bounded search used $N = 2mp_6^2 + 1$ for $1 \le m \le 10^6$. Sieving by primes through 10^8 left 61,123 parameters; extending the sieve through 10^9 left 54,270. At interruption, 2,813 distinct completed Fermat-composite results were preserved in the journal, and no prime had been certified within that interrupted quadratic search. Of these rejections, 2,683 belong to the final survivor set; the other 130 had subsequently been removed by the deeper sieve. Thus 51,587 members of that set remained without a preserved completed rejection.

The last live counter was 2,820, but its seven-entry excess was not present in the durable journal. The journal count is used here. This discrepancy does not promote any unrecorded computation to evidence. The stopped search is not resumed by this revision. The implemented prototypes below are the bounded experiments of the preceding algorithm edition, with their own inputs and records.

15 A common contract for the eighteen algorithms

Definition 15.1 (Certificate-aware algorithm)**.** A certificate-aware algorithm has four explicit components: an input domain, a finite procedure or a stated finite search window, an output type, and a verification rule for that output. Arithmetic modules return exact residues, and planning modules return plans with stated model costs. Any claim about the primality of an integer is tagged as *candidate*, *proved composite*, *proved prime*, or *inconclusive*. An exhausted window is a statement about that window, not a proof that the corresponding infinite family has no prime.

A composite output includes an exact proper factor or a theorem-backed contradiction such as a failed Fermat congruence. A prime output includes all necessary factorizations, witnesses, size inequalities, and predecessor certificates. A failed gcd check, a missing witness, a parameter-search failure, or an unavailable elliptic inverse is not silently relabelled as a proof of compositeness. If an inverse failure supplies a gcd strictly between 1 and N, that gcd itself is valid compositeness evidence.

Definition 15.2 (Prime-divisor constraint)**.** A prime-divisor constraint for N is a proved statement that every prime divisor ℓ of N lies in a specified set $\mathcal{D}$. Order certificates, common-discriminant Lucas certificates, and valid elliptic point calculations provide examples. Candidate-generation restrictions, such as a decimal digit pattern, are a different type of condition and do not by themselves restrict the prime divisors.

Proposition 15.3 (Sound composition of divisor constraints)**.** *Let $N > 1$ be an integer. Suppose independently checked certificates establish $\ell \in \mathcal{D}_i$ for every prime $\ell \mid N$, for each $i = 1, \dots, t$. If no integer in*

$$[2, \lfloor \sqrt{N} \rfloor] \cap \bigcap_{i=1}^{t} \mathcal{D}_i$$

divides N, then N is prime.

Proof. A composite $N > 1$ has a prime divisor at most $\sqrt{N}$. That divisor would lie in every $\mathcal{D}_i$ and contradict the final exclusion. □

This elementary rule is the semantic foundation for combining certificates: the constraints may be intersected only after their premises have been verified. The intersection can be powerful when it has few representatives, but enumerating a large intersection is not automatically efficient.

15.1 Scope of the additional algorithms

A01–A18 are additions to this manuscript with explicit algorithms, correctness arguments, and reproducible demonstrations. Their ingredients include established order tests, residue automata, Chinese remaindering, product trees, exact integer reduction, finite optimization, and elliptic groups. New notation or a new composition does not establish historical priority. We distinguish a classical criterion, a constructed specialization, an implementation rule, and an experimental scheduling policy. None of these labels promises a faster asymptotic general-purpose primality test.

The purpose of the revision is to make stronger compositions concrete and falsifiable. An exact operation-count improvement is reported as such. Measured timing claims require matched inputs and repeated measurements. An exact finite-population optimum is not extrapolated to an unseen prime distribution, and a toy demonstration is not represented as a record-scale benchmark.

15.2 Catalogue of the eighteen additions

The identifiers in Table 5 are stable throughout the text and the executable appendices. They count procedures with different input–output contracts, not eighteen interchangeable filters. The complete hypotheses are stated in the corresponding subsections; in particular, the quadratic and cubic constructions require a certified odd prime p and a positive even coefficient a.

Table 5: Eighteen additional algorithms and their principal contracts.

ID	Procedure	Output or decisive obligation
A01	Extended quadratic Fermat closure	$N = ap^2 + 1$, $a < 4p - 2$; one full Fermat congruence and short nonunity.

ID	Procedure	Output or decisive obligation
A02	Cubic trace exclusion	$N = ap^3 + 1$; a coefficient-sized list of exact discriminant checks.
A03	Joint positive and negative lifting	Certify both $kp - 1$ and $kp + 1$ using a shared seed and separate proofs.
A04	Inverse construction of digit blocks	Prescribed prefix and suffix, with an adjustable middle block and known factor of $N - 1$.
A05	Prime-power cyclotomic divisor mining	Exact modular divisibility plus a universal lower bound on prime divisors.
A06	Short-index negative quadratic closure	$N = ap^2 - 1$, nonsquare $a < 4p + 2$; one long Lucas value and $\gcd(U_a, N)$.
A07	Sparse digit-language join	Meet-in-the-middle generation satisfying both a language and a certificate congruence.
A08	Polynomial parameter root compiler	Exact forbidden residue classes, audited by finite differences.
A09	Two-tree batch sieve	Route a small factor to each rejected candidate; retain unresolved cases.
A10	Signed cofactor reduction	Canonical modular products for $k2^n \pm 1$ with at most one correction.
A11	Common-order CRT packet	One combined order certificate for several component integers.
A12	Exact finite-population screening plan	Minimum cost over ordered subsets in an explicitly specified finite model.
A13	Prime-power witness fusion	Project separate witnesses to coprime primary orders and multiply them.
A14	Batched unit verification	One gcd on the successful path, with factor localization on failure.
A15	Elliptic order fusion	Project points to primary components and realize the lcm of known orders.
A16	Residue–Hasse window intersection	Combine universal arithmetic constraints with exact elliptic intervals.
A17	Shared certificate planning	Minimum-cost verified-set path; shared predecessors are counted once.
A18	Hereditary divisor certification	An exact divisor inherits a verified lower bound on all its prime factors.

15.3 A logical restriction on composition

A universal divisor constraint and an existential one are different types of evidence. For example, the Fermat pair used in A01 and A02 initially shows that *at least one* prime divisor is 1 modulo p. The coefficient argument then finishes the primality proof. The pair by itself does not prove that *every* prime divisor lies in that class, and must not be supplied as a universal residue input to A16 or A18.

In contrast, the gcd conditions in the Pocklington order principle exclude insufficient order at every prime divisor. Valid common-parameter Lucas certificates and strongly nonzero elliptic multiples also provide universal statements. A composition must retain this quantifier information, as well as the numerical parameters. A short formula with the wrong quantifier is not a certificate.

16 Six construction algorithms and their exact acceptance rules

The six algorithms in this section add explicit acceptance rules and search procedures to the earlier inventory. They are not six claims that a completely new theory of primality has been discovered. A01, A02, and A06 give elementary coefficient-sensitive refinements of order arguments; A03 combines two different certificate mechanisms in a simultaneous search; A04 solves a prescribed-block construction problem before testing primality; and A05 turns a small divisor of a potentially enormous cyclotomic value into a prime certificate. Their mathematical claims are proved below. Historical priority for the refinements and combinations has not been established.

A prime supplied as an input always comes with an already checked proof. The five larger numerical experiments use

$$P_\star = 2^{127} - 1 = 170141183460469231731687303715884105727. \tag{18}$$

The verification listing first proves the exponent 127 prime by trial division and repeats all 125 Lucas–Lehmer steps, obtaining residue zero. This is a familiar 39-digit seed, not a newly discovered input. The small seeds $3, 5, 11$ used in examples and controls are checked directly.

The output convention is deliberately three-valued. A valid certificate returns `CERTIFIED`; a displayed factor or a failed necessary Fermat condition returns `COMPOSITE`; a failed sufficient certificate returns `INCONCLUSIVE`. Exhausting a finite search window is a fourth search outcome, `NO_CERTIFICATE_IN_WINDOW`, and does not establish that the window contains no primes unless every entry was separately proved composite.

16.1 A01: parity-strengthened quadratic order closure

Input and purpose. The input is a certified odd prime p, a positive even coefficient a, and a witness base $1 < b < N$, where $N = ap^2 + 1$. The goal is to replace the long reduced-exponent gcd of a conventional certificate by the short condition $b^a \not\equiv 1 \pmod N$. For the range $a < p$, this principle appears in [18, Theorem 4.4]. The following parity argument enlarges the coefficient range to $a < 4p - 2$.

Lemma 16.1 (Quadratic positive coefficient barrier)**.** *Let p be an odd prime, let $a > 0$ be even, and set $N = ap^2 + 1$. If N is composite and has a prime divisor congruent to 1 modulo p, then $a \geq 4p - 2$.*

Proof. Choose such a divisor $\ell = up + 1$, and write its complementary divisor as $N/\ell = vp + 1$, using $N \equiv 1 \pmod p$. Both divisors are odd and greater than one, so $u, v \geq 2$ are even. Comparison with $ap^2 + 1$ gives

$$a = uv + \frac{u + v}{p}, \qquad p \mid u + v.$$

The sum is even, hence $u + v = 2pt$ for an integer $t \geq 1$. Since $(u - 2)(v - 2) \geq 0$, we have $uv \geq 2(u + v) - 4$. Therefore

$$a \geq (4p + 2)t - 4 \geq 4p - 2.$$

The complementary divisor need not be prime; the argument includes repeated prime factors and products with more than two prime factors. □

Theorem 16.2 (A01 acceptance rule)**.** *Let p be a certified odd prime and let a be positive and even with $a < 4p - 2$. Put $N = ap^2 + 1$. If*

$$b^{N-1} \equiv 1 \pmod N, \qquad b^a \not\equiv 1 \pmod N, \tag{19}$$

then N is prime. In particular, if $2^a < N$, then

$$N \text{ is prime} \iff 2^{N-1} \equiv 1 \pmod{N}.$$

Proof. The first congruence makes b a unit modulo N. Its multiplicative order h divides ap^2 but does not divide a, so $p \mid h \mid \phi(N)$. Also $p \nmid N$. In the prime-power formula for $\phi(N)$, this forces at least one prime divisor ℓ of N to satisfy $p \mid \ell - 1$. If N were composite, lemma 16.1 would contradict $a < 4p-2$. For the final statement, $1 < 2^a < N$ makes the second congruence automatic; the converse is Fermat's little theorem. □

Executable procedure. Check the input proof and the strict coefficient bound. Compute $f = b^{N-1} \bmod N$. A value $f \neq 1$ proves compositeness because $1 < b < N$. If $f = 1$, either use the exact integer inequality $b^a < N$ or compute $s = b^a \bmod N$. Return `CERTIFIED` when $s \neq 1$, and `INCONCLUSIVE` when $s = 1$. For base two and odd N, the inexpensive test $a < \mathrm{bits}(N)$ is equivalent to $2^a < N$.

The acceptance rule uses one full modular power and at most one short power. There is no unconditional promise that a chosen base will certify every prime in the larger interval. This is not a general claim that a Fermat pass proves primality outside the displayed family.

Sharpness and examples. The boundary admits the factorization

$$(4p - 2)p^2 + 1 = (2p + 1)(2p^2 - 2p + 1).$$

At $p = 3$, $a = 10$, and $b = 9$, the composite $91 = 7 \cdot 13$ satisfies $9^{90} \equiv 1 \pmod{91}$ and $9^{10} \equiv 9 \not\equiv 1 \pmod{91}$. Thus equality cannot be included in theorem 16.2. The short test is also essential: $N = 55 = 6 \cdot 3^2 + 1$ and $b = 54$ have both powers equal to one, so the verifier correctly returns `INCONCLUSIVE`.

The small prime $151 = 6 \cdot 5^2 + 1$ illustrates the enlarged range $a \geq p$. The executed larger search returned

$$N_{01} = 180P_\star^2 + 1, \qquad 79 \text{ decimal digits}, \qquad b = 2. \tag{20}$$

Its full residue is 1, and its short residue is exactly $2^{180} < N_{01}$. Eighteen candidates survived the small-prime sieve before this certificate was found. This is an observed finite search count, not a distributional claim.

16.2 A02: cubic closure by a bounded trace scan

Input and purpose. Let p be a certified odd prime, $a > 0$ even, and $N = ap^3 + 1$. The order implication used in A01 still supplies a prime factor that is 1 modulo p, but the quadratic coefficient barrier no longer excludes all composite values. A02 explicitly enumerates the small set of possible factor traces. Unlike the earlier cubic closure in proposition 3.4, it does not begin by proving that every prime factor is 1 modulo a large fully factored F.

Proposition 16.3 (Coefficient-sized cubic obstruction list)**.** *Assume*

$$b^{N-1} \equiv 1 \pmod{N}, \qquad b^a \not\equiv 1 \pmod{N}.$$

Set

$$T = \left\lfloor \frac{ap + 4}{2p + 1} \right\rfloor.$$

Then N is composite if and only if, for some even integer t with $2 \leq t \leq T$, the integer

$$\Delta_t = (tp)^2 - 4(ap - t) \tag{21}$$

is a nonnegative square z^2 and both

$$u = \frac{tp - z}{2}, \qquad v = \frac{tp + z}{2}$$

are positive even integers. In that case $N = (up+1)(vp+1)$ is an explicit proper factorization.

Proof. As in the proof of A01, the two power conditions imply $p \mid \phi(N)$ and hence the existence of a prime factor $\ell \equiv 1 \pmod{p}$. If N is composite, write $N = (up+1)(vp+1)$ with even $u, v \geq 2$. Expanding and dividing by p gives

$$ap^2 = uvp + (u+v).$$

Thus $u + v = tp$ with $t > 0$ even, and $uv = ap - t$. The inequality $uv \geq 2(u+v) - 4$ gives $(2p+1)t \leq ap + 4$, which is the asserted finite bound. The two coefficients are exactly the roots of $X^2 - tpX + (ap - t)$, giving (21). Conversely, roots with the stated signs and parity reconstruct two integers exceeding one whose product is N. □

Executable procedure and cost. First run the same full and short power checks as in A01. When they pass, enumerate $t = 2, 4, \ldots, 2\lfloor T/2 \rfloor$. Compute the integer square root of each nonnegative Δ_t, and check the reconstructed factors exactly. A factor gives `COMPOSITE`; absence of all such factors gives `CERTIFIED`. The closure takes at most $\lfloor T/2 \rfloor$ integer square tests, approximately $a/4$, rather than trial division up to $\sqrt{N}$. These square tests can become expensive when a grows; no general speed advantage over a well-implemented Pocklington certificate is inferred. The coefficient controls the extra work.

Executed example and adversarial control. The search returned

$$N_{02} = 82P_\star^3 + 1, \qquad 117 \text{ decimal digits}, \qquad b = 2. \tag{22}$$

Here $T = 40$. The full power is 1, the short power is $2^{82} < N_{02}$, and none of the twenty even traces supplies a factor. Ten candidates survived the preliminary sieve before this certificate was obtained.

The trace scan is a logical obligation, not a cosmetic extra check. At $p = 3$, $a = 8$, $N = 217$, and $b = 191$, both power conditions pass, but $t = 4$ gives $\Delta_t = 64$, $u = 2$, $v = 10$, and $217 = 7 \cdot 31$. The executable verifier includes this example to detect an accidental promotion of a mere order condition to a prime certificate.

16.3 A03: simultaneous opposite-sign certificates

Construction problem. Given a certified odd prime p, seek an even $k > 0$ for which both

$$N_- = kp - 1, \qquad N_+ = kp + 1$$

are prime. This produces a certified pair at distance two, with the same parent prime supplying a factor of $N_- + 1$ and $N_+ - 1$. It is a simultaneous finite search, not a proof of infinitely many twin primes and not a promise of a pair for every fixed parent or search window.

Proposition 16.4 (Joint certificate contract)**.** *Suppose $N_+ < (2p-1)^2$. If a Pocklington witness for the factor p proves N_+ and common-parameter Lucas conditions for the factor p prove N_-, then both numbers are prime. Explicitly, sufficient conditions are*

$$b^{N_+ - 1} \equiv 1 \pmod{N_+}, \qquad \gcd(b^k - 1, N_+) = 1,$$
$$U_{N_- + 1}(\alpha, \beta) \equiv 0 \pmod{N_-}, \qquad \gcd(U_k(\alpha, \beta), N_-) = 1,$$

together with $\gcd(N_-, 2\beta(\alpha^2 - 4\beta)) = 1$.

Proof. The two known factors are immediate from the definitions. The shared size bound implies the parity-strengthened bounds in theorems 2.1 and 2.3. Apply those theorems separately. □

Algorithm. Choose a finite interval for k. Reject a parameter whenever either $kp-1$ or $kp+1$ has a displayed proper small factor. For $p \neq 3$ and both candidates exceeding 3, divisibility by three forces any surviving even k to be a multiple of six. On the survivors, attempt both exact certificates and emit the pair only after both have passed. The input certificate and small-parameter sieve can be shared; modular powers with the two different moduli are still separate computations.

Executed 42-digit pair. With $p = P_\star$ and $k = 1104$, the output is

$$N_- = 187835866540358031831782783302336052722607,$$
$$N_+ = 187835866540358031831782783302336052722609.$$

For N_+, base $b = 2$ gives full residue 1 and reduced gcd 1. For N_-, $(\alpha, \beta) = (3, 1)$ gives discriminant 5, full Lucas residue 0, and reduced gcd 1. The parameter was reached after examining 184 multiples of six, of which 24 pairs survived trial division by the odd primes below 101. Neither integer is asserted to be historically new. This joint output is different from the earlier Collatz-linked pair: its specified relation is the exact difference $N_+ - N_- = 2$.

16.4 A04: inverse construction with prescribed digit blocks

Construction problem. Fix a radix $B \geq 2$, a nonzero prefix A, a suffix $0 \leq C < B^m$ of width m, and a free middle block $0 \leq X < B^t$. The candidate is

$$N = AB^{m+t} + XB^m + C. \tag{23}$$

The leading zeros of the middle and suffix blocks, when present, count toward their specified widths. The task is to enforce these digit constraints while also making a known factor large enough to certify N.

Proposition 16.5 (Certificate-compatible block splice)**.** *Let $F > 1$ be fully factored and satisfy $\gcd(B, F) = 1$. Set*

$$K = AB^{m+t} + C, \qquad r \equiv (1-K)(B^m)^{-1} \pmod{F}, \qquad 0 \leq r < F.$$

The middle blocks giving $F \mid N-1$ are exactly

$$X = r + jF \quad \textit{with} \quad 0 \leq r + jF < B^t.$$

Restricting further to

$$X \leq H := \min\left(B^t - 1, \left\lfloor \frac{F^2 - 1 - K}{B^m} \right\rfloor\right) \tag{24}$$

guarantees $N < F^2$. For every odd $N > 1$ in this restricted progression, the ordinary Pocklington witness conditions for F are therefore sufficient to certify primality.

Proof. The congruence $K + XB^m \equiv 1 \pmod{F}$ has the unique residue class displayed because B^m is invertible modulo F. The digit restriction is exactly the stated interval for X. The second upper bound is equivalent to $K + XB^m \leq F^2 - 1$. The final implication is theorem 2.1. □

Algorithm and failure states. Compute the one modular inverse, the residue r, and the clipped endpoint H. If $H < r$, the admissible interval is empty. Otherwise enumerate $j = 0, \ldots, \lfloor (H - r)/F \rfloor$, or a user-supplied finite prefix of this range. Reject even candidates, candidates no larger than the designated input prime, and candidates with a displayed small factor. Attempt a complete certificate on the survivors. A prescribed suffix that forces divisibility by two or five in radix ten can make every candidate composite; there is no existence guarantee for arbitrary blocks.

The construction is not independent primality testing of the digit blocks. It solves the global congruence before generating them. It also differs from merely appending zeros and one to a prime: both visible end blocks can be fixed while the middle block is selected by an inverse congruence. The number of admissible middle blocks is determined exactly by the progression and interval; certificate attempts remain bounded by the chosen search budget.

Executed 101-digit example. Take $B = 10$, $A = 314159$, $C = 27183$, $m = 5$, $t = 90$, and $F = P_\star^2$. In the notation above, the successful middle block is $X = r + 195F$. Equations (23)–(24), together with the exact formula for r, specify the entire integer without an external data file. It is a 101-digit prime beginning with `314159` and ending with `27183`. Base two gives full Pocklington residue 1 and the required gcd 1 for the prime $P_\star$. Fifty-nine candidates survived the preliminary sieve through this successful parameter. Its decimal SHA-256 identifier is

`136c7dd47d894a5a130e9d29463fba3cfc5407e1f7e27dcb9de347738cdf2ca0.`

The hash identifies the reconstructed integer; the modular conditions prove its primality.

16.5 A05: cyclotomic divisor mining with a certified size threshold

Input and purpose. Let p be a certified odd prime, $t \geq 1$, and $b \geq 2$. The symbolic source

$$C = \Phi_{p^t}(b) = 1 + b^{p^{t-1}} + \cdots + b^{(p-1)p^{t-1}}$$

may have far more digits than a prospective divisor R. A05 uses the restricted prime factors of this source to turn a verified small divisor into a proof of primality. It extends the earlier factor-extraction observation by specifying a prime-power index, an essential exceptional-prime guard, an explicit acceptance threshold, and a procedure that need not expand C.

Theorem 16.6 (Cyclotomic divisor threshold)**.** *Suppose* $1 < R \mid \Phi_{p^t}(b)$, $\gcd(R, p) = 1$, *and*

$$R < (2p^t + 1)^2. \tag{25}$$

Then R is prime. In particular, every accepted R exceeds the input p.

Proof. Put $m = p^t$. The value C is odd, whether b is odd or even. For any prime $\ell \mid R$, the constant term of C shows that $\ell \nmid b$. The identity $(b^{m/p} - 1)C = b^m - 1$ implies that the order of b modulo ℓ divides m. If it were a proper divisor of m, it would divide m/p, and reduction of the geometric sum would give $C \equiv p \pmod{\ell}$. That would force $\ell = p$, contrary to $\gcd(R, p) = 1$. Thus $\mathrm{ord}_\ell(b) = p^t$ and $p^t \mid \ell - 1$. Since ℓ and p^t are odd,

$$\ell \equiv 1 \pmod{2p^t}, \qquad \ell \geq 2p^t + 1.$$

A composite R has at least two prime factors counted with multiplicity, which would imply $R \geq (2p^t + 1)^2$. This contradicts (25). □

Division-free evaluation of the source. To verify $R \mid C$, first compute $x = b^{p^{t-1}} \bmod R$. For $S_j(x) = 1 + x + \cdots + x^{j-1}$, maintain the pair $(x^j, S_j(x))$ modulo R using

$$x^{j+k} = x^j x^k, \qquad S_{j+k}(x) = S_j(x) + x^j S_k(x).$$

Binary doubling evaluates $S_p(x)$ using $O(\log p)$ modular multiplications, without division by $x - 1$. A zero result proves the exact divisibility $R \mid C$. This avoids the invalid assumption that $x - 1$ has an inverse modulo a still-unproved candidate. It also avoids constructing the potentially enormous integer C.

Algorithm. Under a finite candidate budget, enumerate prospective $R = 2p^t j + 1$ satisfying (25), or accept a supplied prospective divisor from another calculation. Check the exceptional-prime guard and the modular geometric sum. A passing R is immediately a certified prime by the theorem. A failed divisibility check rejects it as a divisor of this source, not necessarily as a prime. If no candidate succeeds, the algorithm makes no claim that the source has a divisor below the threshold. Its verification cost is polynomial in the bit lengths of the modular input and exponent, but the number of candidates required to find a divisor is not bounded by this theorem.

Executed divisor certificates. For $(b, p, t) = (2, 5, 2)$, direct integer reconstruction gives

$$\Phi_{25}(2) = 1082401 = 601 \cdot 1801.$$

Both divisors are coprime to 5 and smaller than $51^2 = 2601$. Thus both 601 and 1801 are prime by the source-order argument. For $(b, p, t) = (2, 11, 2)$ the same procedure certifies 727 as a divisor of $\Phi_{121}(2)$, because $727 < 243^2$. The implementation first checks the modular geometric sums and then cross-checks these small source values by direct integer evaluation; separate trial division confirms the three small outputs.

The exceptional-prime guard cannot be deleted. Indeed,

$$\Phi_9(4) = 4161 = 3 \cdot 19 \cdot 73, \qquad 57 = 3 \cdot 19 < 19^2,$$

so 57 is a composite divisor below the proposed threshold, excluded exactly by $\gcd(57, 3) \neq 1$. Nor does the primality of all small divisors imply primality of the entire cyclotomic value: 1082401 itself is composite.

16.6 A06: short-index Lucas closure for negative quadratic lifts

Input and purpose. For $N = ap^2 - 1$, the conventional $N + 1$ certificate for the known prime power p^2 examines a reduced Lucas index ap. A06 instead uses the much shorter index a, at the cost of an explicit nonsquare condition on a and a coefficient bound. This is the negative-sign companion to the positive coefficient barrier in A01.

Lemma 16.7 (Quadratic negative coefficient barrier)**.** *Let p be an odd prime, let $a > 0$ be even, and let $N = ap^2 - 1$. If N is composite and has a prime divisor congruent to 1 or -1 modulo p, then either a is a perfect square or $a \geq 4p + 2$.*

Proof. Choose a prime divisor $\ell = up + \varepsilon$, where $\varepsilon \in \{1, -1\}$. Since $N \equiv -1 \pmod{p}$, the complementary divisor is $vp - \varepsilon$. Both are odd and greater than one, so $u, v \geq 2$ are even. Expanding gives

$$a = uv + \varepsilon \frac{v - u}{p}, \qquad p \mid v - u.$$

If $u = v$, then $a = u^2$. Otherwise, write $|u - v| = 2ph$ for an integer $h \geq 1$. With $w = \min(u, v) \geq 2$,

$$uv = w(w + 2ph) \geq 4 + 4ph.$$

It follows that $a \geq uv - 2h \geq 4 + (4p - 2)h \geq 4p + 2$. Again, the complementary divisor need not be prime. □

Theorem 16.8 (A06 acceptance rule)**.** *Let p be a certified odd prime. Let $a > 0$ be even and not a square, with $a < 4p + 2$, and set $N = ap^2 - 1$. Choose Lucas parameters (α, β), put $D = \alpha^2 - 4\beta$, and suppose*

$$\gcd(N, 2\beta D) = 1, \qquad U_{N+1}(\alpha, \beta) \equiv 0 \pmod{N}, \qquad \gcd(U_a(\alpha, \beta), N) = 1. \tag{26}$$

Then N is prime.

Proof. For a prime divisor ℓ of N, the rank-of-apparition argument in theorem 2.3 applies. The order of the ratio of the two roots of $X^2 - \alpha X + \beta$ divides ap^2 and does not divide a. It is therefore divisible by p, and consequently

$$p \mid \ell - \left(\frac{D}{\ell}\right), \qquad \ell \equiv 1 \text{ or } -1 \pmod{p}.$$

If N were composite, lemma 16.7 would force either a square coefficient or $a \geq 4p + 2$, contrary to the hypotheses. □

Algorithm and cost. Check the parent proof, parity, nonsquare condition, and strict coefficient bound. Check nondegeneracy, compute U_a, and take its gcd with N. A proper gcd proves compositeness; a gcd equal to N leaves the attempt inconclusive. When the gcd is one, compute the full Lucas residue and accept only if it is zero. A bounded search may try several parameter pairs; the Jacobi condition $(D/N) = -1$ is useful for selecting them but is not an additional hypothesis of the theorem.

The two indices are a and ap^2, rather than ap and ap^2. This is a shorter explicit proof obligation. A well-designed implementation can share index calculations, however, so the changed indices do not by themselves establish a universal speedup or a factor-of-two improvement. The number of Lucas doubling operations remains logarithmic in N for each fixed certificate attempt.

Executed example. The search returned

$$N_{06} = 678P_\star^2 - 1, \qquad 80 \text{ decimal digits}, \qquad (\alpha, \beta) = (4, 1). \tag{27}$$

Here $D = 12$, the full Lucas residue is zero, and $\gcd(U_{678}(4, 1), N_{06}) = 1$. The coefficient is even and nonsquare, and the size condition is immediate from $678 < 4P_\star + 2$. Eighty-four candidates survived the small-prime sieve through this output.

Both additional hypotheses have genuine counterexamples. If the nonsquare condition is omitted, $p = 3$, $a = 4$, and $N = 35$ pass all the Lucas conditions with $(\alpha, \beta) = (1, 1)$: $U_{36} = 0$ and $U_4 = -1$. If equality is allowed in the coefficient bound, $p = 3$, $a = 14$, and $N = 125$ also pass with the same parameters: $U_{126} = 0$ and $U_{14} = 1$. Thus neither deletion is justified by the modular conditions alone.

16.7 Finite verification of the six constructions

The executable standard-library verifier reconstructs every displayed large integer from (18), proves that seed, and checks the complete acceptance conditions. The six algorithms supply nine distinct output integers in these experiments: the four single outputs of A01, A02, A04, and A06 together with the two outputs of A03 and the three small outputs of A05, counted correctly as $1 + 1 + 2 + 1 + 3 + 1 = 9$. The common seed is a separate input. These outputs are a demonstration inventory, not nine assertions of first discovery.

The finite controls compare A01 against exact trial division on 4,144 witness attempts, with 918 accepting attempts; A02 on 672 attempts, with 160 acceptances; and A06 on 1,981 attempts, with 244 acceptances. All accepted integers in these small ranges are prime. Acceptances are counted per witness and are not counts of distinct primes. The generic matrix implementation of Lucas sequences is checked against 1,953 direct recurrence values, and the division-free geometric-sum implementation is checked against 744 directly summed examples. Seven hypothesis-sensitive negative controls include the composite examples 91, 55, 217, 35, 125, 57, and 1082401 described above. These finite checks support implementation reliability; the proofs, rather than the checked ranges, establish the general theorems.

As an additional check on the larger examples, the listing certifies A01 and A02 again with the classical Pocklington theorem, using the known factors $P_\star^2$ and $P_\star^3$ respectively. It certifies A06 again with the classical Lucas prime-power criterion, this time checking the longer reduced index $678P_\star$. These are different certificate implications applied to the same integers, not claims of independent hardware or formally verified arithmetic libraries.

The complete executed listing returned `A01_A06_ALL_EXACT_CHECKS_PASSED`. Its observed running time was below 0.1 seconds in this environment, including the small controls and the bounded example searches. These are small examples with fast standard-library arithmetic, not a benchmark of record-size searches and not evidence that a 117-digit result generalizes to an instantaneous prime of millions of digits.

17 Six exact arithmetic and search modules

Algorithms A07–A12 add a computational layer to the certificate constructions. They have different contracts: A07 and A08 enumerate candidates, A09 produces small-factor rejection certificates, A10 performs exact modular arithmetic, A11 certifies a whole packet of integers, and A12 optimizes a stated finite screening problem. Calling all six primality tests would obscure their different outputs. The ingredients include classical meet-in-the-middle search, polynomial congruences, product trees, special-modulus reduction, Chinese remaindering, and dynamic programming. We claim neither historical priority for those ingredients nor a measured improvement over optimized GMP or PRST. The contribution here is their explicit integration with the known-prime lifting contracts, with proofs and executable checks.

ID	Additional algorithm	Exact output
A07	Certificate-constrained digit-language join	Complete bounded candidate list
A08	Quadratic residue compiler with finite-difference audit	Parameter exclusions and factors
A09	Two-tree batch sieve with factor routing	Composite / small prime / unresolved
A10	Signed cofactor reduction with one correction	Canonical modular product
A11	CRT packet of common prime-power orders	Simultaneous primality certificate
A12	Exact-cost finite-population screening planner	Optimal plan in a specified model

17.1 A07: a certificate-constrained digit-language join

Problem and input. Digit constraints can describe an alphabet, a fixed prefix, forbidden adjacent pairs, or a regular language recognized by a finite automaton. It is wasteful to enumerate such strings, turn them into arbitrary integers, and only afterwards discover that their neighbors have no useful known factor. Instead, impose the certificate congruence during generation.

Fix a base $B \geq 2$, a length d, permitted digits at each position, and a finite automaton $\mathcal{A}$ accepting the desired words. Leading zeroes are excluded when a fixed digit length is intended. Let $F > 1$ be fully factored. The target set consists of the accepted integers N satisfying

$$N \equiv 1 \pmod{F}, \qquad \max \mathcal{Q} < N < F^2, \qquad \gcd\left(N, \prod_{q \in \mathcal{Q}} q\right) = 1, \tag{28}$$

where $\mathcal{Q}$ is a finite set of proved small primes. Oddness is enforced either by the digit language or by including 2 in $\mathcal{Q}$. The bound makes the elementary Pocklington certificate available, but does not itself imply primality.

Sparse split. Split a word into a prefix A of length $d - t$ and a suffix C of length t. Its integer value is $N = AB^t + C$. For each permitted suffix, store

$$(C \bmod F,\ C,\ T_C), \qquad T_C = \{s : \mathcal{A} \text{ accepts } C \text{ when started in state } s\}.$$

The records are hashed by the first component. A prefix has a value A and a resulting automaton state s_A. It only needs the bucket

$$C \bmod F = (1 - AB^t) \bmod F, \tag{29}$$

and within that bucket it retains the suffixes for which $s_A \in T_C$. Finally apply the interval and small-prime conditions in (28). No array with one entry for every residue modulo the potentially enormous integer F is allocated.

Proposition 17.1 (Completeness of the digit join)**.** *The sparse join produces exactly the integers specified by* (28)*, each once for fixed-length digit words. Every accepted output which subsequently passes the Pocklington conditions for F is prime.*

Proof. Every permitted word has a unique prefix–suffix split. Acceptance of the concatenation is equivalent to $s_A \in T_C$ by the definition of the automaton transition. The congruence $AB^t + C \equiv 1 \pmod{F}$ is equivalent to (29). Thus every required word reaches exactly its own matching bucket, and an incorrect word fails one of the explicit tests. The last assertion follows from $F^2 > N$ and theorem 2.1. □

Procedure and cost. Enumerate the two half-word sets, build the suffix buckets, join the matching residues, filter the emitted integers, and attempt a certificate. With U_L and U_R half-words and H records examined in matching buckets, the combinatorial work is $O(U_L + U_R + H)$ hash operations, in addition to digit-transition and integer-arithmetic costs. A suffix acceptance set can be stored as a bitset over the automaton states. The bit lengths of the residues still matter. In a balanced unrestricted alphabet of size a, the half-word counts are about $a^{d/2}$, but the collision count H can be as large as $U_L U_R$. This is an output-sensitive exponential search, not a polynomial-time method for every digit constraint. The split-and-join idea is classical [19]; adding the certificate congruence avoids a full F-state residue automaton.

Executed example. Use decimal digits $\{1, 3, 7, 9\}$, length four, and forbid equal adjacent digits. Set $F = 107$ and $\mathcal{Q} = \{3, 5, 7, 11\}$; all these candidates are already odd. There are 256 unrestricted words and 108 words in the language. Splitting after two digits produces 12 admissible prefixes and 12 admissible suffixes. Exactly two residue-bucket hits remain:

$$1713, \qquad 7919.$$

The first is divisible by 3. For the second,

$$7919 - 1 = 74 \cdot 107, \qquad 107^2 > 7919, \qquad 2^{7918} \equiv 1 \pmod{7919},$$

$$2^{74} \equiv 6451 \pmod{7919}, \qquad \gcd(6450, 7919) = 1.$$

Thus 7919 is certified. Direct enumeration of all 256 words gives exactly the same candidate set; trial division independently confirms the small numerical example. The example demonstrates constrained generation, not a previously unknown prime.

17.2 A08: a quadratic residue compiler with a finite-difference audit

Input and purpose. The linear coefficient sieve for kP^2+1 extends to polynomial parameter families, but a root of a polynomial modulo q must be identified exactly. Take a fully factored odd $F > 1$ and

$$N(t) = F(2t^2 + 2t + 2) + 1, \qquad 0 \le t \le T, \qquad 2T^2 + 2T + 2 < F. \tag{30}$$

All candidates are odd, $F \mid N(t) - 1$, and $N(t) < F^2$. For every small prime q, compile the complete set of forbidden parameter classes $R_q = \{t \bmod q : N(t) \equiv 0 \pmod q\}$. Each rejected parameter is accompanied by the actual factor q.

Proposition 17.2 (Two-class compiler)**.** *For the family* (30)*, R_2 is empty. If $q \mid F$ then R_q is empty. For any other odd prime q, put*

$$\Delta_q = -3 - 2F^{-1} \pmod q.$$

If Δ_q is a nonsquare, R_q is empty. If $s^2 = \Delta_q$ modulo q, then

$$R_q = \left\{ \frac{-1+s}{2}, \frac{-1-s}{2} \right\} \pmod q, \tag{31}$$

with duplicates removed. In particular there are at most two forbidden classes.

Proof. The cases $q = 2$ and $q \mid F$ follow directly from (30). In the remaining cases,

$$2N(t) = F\big((2t+1)^2 + 3\big) + 2.$$

Both F and 2 are invertible modulo q, so vanishing of $N(t)$ is equivalent to $(2t+1)^2 = \Delta_q$. Solving this square equation gives (31), including the double-root case. □

Exact reference algorithm. Compute $F \bmod q$, handle the two exceptional cases, find a modular square root by a prime-field routine such as Tonelli–Shanks, and mark $t \equiv r \pmod q$ for each $r \in R_q$. Verify a returned square root by squaring it. For prime q, the Legendre-symbol computation also decides whether a root exists. Modular square-root routines are standard [22]; this compiler specializes their output to the certificate-preserving parameter family.

An independent way to audit the root table is useful. For any integer polynomial f of degree e, let $\delta_j(t) = \Delta^j f(t) \pmod q$, where $\Delta f(t) = f(t+1) - f(t)$. The identities

$$\delta_j(t+1) = \delta_j(t) + \delta_{j+1}(t) \pmod q \quad (0 \le j < e), \qquad \delta_e(t+1) = \delta_e(t) \tag{32}$$

generate successive values using additions. Initialize the differences from $f(0), \dots, f(e)$ and update from low to high index, so that each right-hand side still uses the old value of δ_{j+1}. Enumerating $t = 0, \dots, q-1$ then gives the complete root list. This identity is over the integers; it does not divide by $e!$ and remains valid when $q \le e$.

Cost and boundary of usefulness. For the quadratic specialization, root discovery requires small-field arithmetic and at most two progressions per prime. Marking a parameter window of length W costs $O(\sum_q |R_q|(W/q+1))$ elementary marks, in addition to root discovery. The reference finite-difference audit uses eq additions per prime after initialization. It is suitable for bounded tests and small moduli; it must not replace the square-root compiler by enumeration of every residue when q is enormous. For a general polynomial or a short interval one may instead generate only the required consecutive values. No assertion about the number of prime outputs is made.

Executed example. Set $F = 1009$ and $T = 20$. The 21 candidates have polynomial coefficients

$$N(t) = 2019 + 2018t + 2018t^2.$$

The root compiler and the independent difference computation agree for every prime $q \le 31$. The union of the forbidden classes leaves exactly $t = 4$ and $t = 10$. Both pass the exact certificate:

t	$N(t)$	$(N(t)-1)/1009$	reduced base-2 residue	gcd
4	42379	42	41835	1
10	223999	222	182964	1

In both rows the full Fermat residue is 1, and the displayed gcd is $\gcd(\text{reduced residue}-1, N(t))$. Since $1009^2 > N(t)$, these are primality proofs. The difference audit uses 320 modular additions, excluding its initialization; additional tests include degree-six polynomials modulo 2 and 3. Those checks specifically exercise characteristics in which division by a polynomial degree factorial would be invalid.

17.3 A09: two-tree batch sieving with explicit factor routing

Input and output. Let $N_1, \dots, N_s > 1$ be candidates and let $\mathcal{Q} = \{q_1, \dots, q_h\}$ be distinct proved small primes. Put $W = \prod_j q_j$. The output for each N_i is one of: a proper factor, the fact that N_i itself equals a known wheel prime, or the status UNRESOLVED. The candidates need not be pairwise coprime.

Build one balanced product tree on the candidates and a second on the wheel primes. Starting from $W \bmod \prod_i N_i$, pass remainders down the candidate tree. At leaf i this gives $w_i = W \bmod N_i$. Compute

$$g_i = \gcd(w_i, N_i).$$

If $g_i = 1$, no wheel prime divides that candidate. Otherwise descend the wheel tree, retaining nodes whose product has nontrivial gcd with N_i, until a prime leaf $q \mid N_i$ is reached. Output q as a compositeness certificate only when $q < N_i$.

Proposition 17.3 (Sound factor routing)**.** *The two-tree sieve rejects exactly the candidates having a proper divisor in $\mathcal{Q}$. It never rejects a prime which itself belongs to $\mathcal{Q}$.*

Proof. At every edge of the candidate tree, the child product divides the parent product. Reducing a parent remainder modulo that child therefore preserves the remainder of W. Hence $g_i = \gcd(W, N_i)$. A prime divisor common to W and N_i belongs to at least one child of any wheel-tree node containing it, so descent reaches a leaf dividing N_i. If that leaf is smaller than N_i, it is a proper divisor. If it equals N_i, the supplied primality proof for the wheel leaf identifies a small prime, not a composite. Conversely every proper wheel divisor gives $g_i > 1$ and is found by descent. □

Why routing is part of the algorithm. The equality $g_i = N_i$ does not by itself decide compositeness. It can occur both for a wheel prime and for a squarefree product of wheel primes. Returning composite whenever the leaf gcd exceeds 1 would incorrectly reject the former. Returning unresolved whenever it equals N_i would discard useful factors in the latter. The explicit descent resolves this ambiguity and supplies an independently checkable integer divisor.

Relation to prior methods and cost. Product and remainder trees, and batch gcd computations, are established techniques [23, 21]. Here they are used for a known wheel product rather than interpreted as a new primality criterion. If the total candidate bit length is S, each balanced product-tree level has total bit length $O(S)$ and there are $O(\log s)$ levels. Actual time includes the reduction of the wheel product, remainders down the tree, leaf gcds, and wheel-tree descent; the bit length of W also counts. Batching can share work but creates large intermediate products. A production implementation should cap the total product size and use several batches. The reference experiment establishes exact outputs and makes no speed claim against an optimized segmented sieve.

Executed example. For primes at most 43 the wheel product is

$$W = 13082761331670030.$$

The batch deliberately includes both equality cases and a composite with no wheel factor:

N_i	$\gcd(W, N_i)$	routed leaf	exact output
1713	3	3	composite
7919	1	–	unresolved
341	341	11	composite
49	7	7	composite
17	17	17	known small prime
10403	1	–	unresolved
42379	1	–	unresolved
223999	1	–	unresolved

The surviving integer $10403 = 101 \cdot 103$ verifies concretely that a wheel survivor cannot be promoted to a prime. The three other nontrivial survivors have independent certificates elsewhere in this section; the sieve itself does not use that knowledge.

17.4 A10: signed cofactor reduction with one correction

This arithmetic module isolates a useful contract for moduli

$$N = k2^n + \varepsilon, \qquad k \geq 1, \quad n \geq 1, \quad \varepsilon \in \{1, -1\}, \quad N \geq 3.$$

It does not assume N prime, and it does not require the Proth inequality. When k fits a machine word, it replaces reduction of a full product by division by the large N with a bit split and division by the fixed small cofactor k. Special-modulus arithmetic and invariant-divisor techniques are classical [20]; the exact signed formula and its input range are proved explicitly below.

Proposition 17.4 (One-correction cofactor reduction)**.** *Let $R = 2^n$ and suppose*

$$0 \le x \le (N-1)^2.$$

Write

$$x = aR + b, \quad 0 \le b < R, \qquad a = kt + s, \quad 0 \le s < k.$$

Define

$$y = b + sR - \varepsilon t. \tag{33}$$

For $\varepsilon = 1$, the canonical remainder $x \bmod N$ is y if $y \ge 0$ and $y + N$ otherwise. For $\varepsilon = -1$, it is y if $y < N$ and $y - N$ otherwise. At most one correction is required.

Proof. The identity

$$x - y = t(kR + \varepsilon) = tN$$

proves congruence. Moreover $t = \lfloor x/(kR) \rfloor$. If $\varepsilon = 1$, then $kR = N - 1$, so $0 \le t \le N - 1$ and $0 \le b + sR \le kR - 1 = N - 2$. Consequently $-N < y < N$. If $\varepsilon = -1$, then $kR = N + 1$, so $0 \le t < N - 1$ and $0 \le b + sR \le kR - 1 = N$. Thus $0 \le y < 2N$. The stated single correction gives the unique integer in $[0, N)$ congruent to x. □

Procedure. For canonical input residues $0 \le u, v < N$, set $x = uv$ and perform

```
high = x >> n
low = x & ((1 << n) - 1)
t, s = divmod(high, k)
y = low + (s << n) - epsilon*t
if epsilon == 1 and y < 0:
    y += N
elif epsilon == -1 and y >= N:
    y -= N
return y
```

The product bound is automatic for these inputs, so this routine can replace the reduction inside repeated squaring. There is no Montgomery-domain conversion in this particular kernel. If k is a machine word, division of the multiword `high` by k is a word-divisor operation; fixed reciprocal techniques may be used [20]. Its linear pass over the limbs and the bit operations do not remove the cost of forming uv. If k is itself a large integer, the small-divisor advantage disappears.

The range restriction is essential. It is tempting to replace the hypothesis by $x < N^2$, but the stated one-correction formula then fails for the plus sign. For $N = 5$, $k = 1$, $R = 4$, and $x = 24 < N^2$, it gives $y = -6$, and one addition of N leaves -1 instead of the correct remainder 4. The executable reference rejects such an out-of-contract input. This does not affect products of canonical residues, for which $x \le 16$ in this example.

Executed example and tests. For $R = 256$, $k = 5$, and $x = 1000 \cdot 1200$, the common decomposition is

$$a = 4687, \quad b = 128, \quad t = 937, \quad s = 2.$$

At $N = 1281$, (33) gives -297, corrected to 984. At $N = 1279$, it gives 1577, corrected to 298. Both equal direct integer remainders. The reference script checks all 193871 canonical operand pairs for $1 \le k \le 9$, $1 \le n \le 4$, and both signs whenever $N \ge 3$. It also verifies 768 fixed-seed random products at radix sizes $64, 256, 1024, 4096$ bits, and 128 complete modular powers against Python's independent built-in `pow`. The random product choices exercise implementation arithmetic; they are not probabilistic primality evidence. No elapsed-time superiority is claimed.

17.5 A11: a CRT packet of common prime-power orders

Objective. Several candidates built from the same certified prime have a common order requirement. This permits an exact packet certificate: a verifier can prove all component integers prime through powers modulo their product. The larger modulus is a real cost, so the purpose is compositional verification and a common arithmetic contract, not an automatic speedup.

Theorem 17.5 (Common-order packet)**.** *Let p be a certified odd prime and let $N_1, \dots, N_s$ be integers with*

$$1 < N_i < p^4.$$

Put $M = \prod_i N_i$. If an integer Z satisfies

$$Y = Z^p \bmod M, \qquad Y^p \equiv 1 \pmod{M}, \qquad \gcd(Y - 1, M) = 1, \tag{34}$$

then every N_i is prime.

Proof. Let ℓ be any prime divisor of an N_i. The full power condition shows that Z is a unit modulo ℓ whose order divides p^2. The gcd condition says $Z^p \not\equiv 1 \pmod{\ell}$. Since p is prime, the order is exactly p^2, so $p^2 \mid \ell - 1$ and $\ell \ge p^2 + 1$. A composite $N_i < p^4$ would have a prime divisor at most $\sqrt{N_i} < p^2$, a contradiction. □

The theorem does not require the verifier to factor the $N_i - 1$ or individually run their complete certificates. It also does not assert that the product M is prime. In fact, for a packet with at least two components it is deliberately composite.

Building a packet from the lifting family. A convenient construction has

$$N_i = k_i p^2 + 1, \qquad N_i < p^4.$$

Choose provisional bases $2 \le a_i < N_i$ and compute $z_i = a_i^{k_i} \bmod N_i$. If the candidates are pairwise coprime, construct the CRT representative

$$Z \equiv z_i \pmod{N_i} \quad \text{for every } i, \qquad 0 \le Z < M.$$

The two powers in (34) then simultaneously encode the local order checks. Pairwise coprimality is required by this CRT construction, although it is not a hypothesis of the verification theorem. A collision must be handled by gcd and factor extraction; it cannot be ignored. Standard implementations of CRT and product-tree reduction are described in [21].

If the packet fails, it is not valid to call every component composite. Reduce the computed residues modulo the individual N_i. A failed full Fermat condition for a base strictly between 1

and N_i proves that component composite. A proper gcd supplies a factor. A reduced gcd equal to N_i can merely mean that the chosen base has insufficient order, requiring a new witness. Valid components may be retained and repacked.

Executed certificate. Take $p = 11$, bases $a_i = 2$, and

$$(k_1, k_2, k_3, k_4) = (6, 12, 18, 22).$$

The component integers and their proposed p^2-order roots are

k_i	$N_i = k_i 11^2 + 1$	$z_i = 2^{k_i} \bmod N_i$
6	727	64
12	1453	1190
18	2179	664
22	2663	79

CRT gives the fully explicit certificate

$$M = 6129547598087, \qquad Z = 4290773793366,$$

$$Y = Z^{11} \bmod M = 4047432967480, \qquad Y^{11} \bmod M = 1, \qquad \gcd(Y - 1, M) = 1.$$

Together with 11 prime and $N_i < 11^4$, this proves all four component integers prime. Direct trial division independently checks these small examples. Negative controls replace one component by 243, replace the seed by 9, use the witnesses 0 and 1, and violate the strict size bound; all are rejected.

Operation counts and the cost of packing. For a positive exponent e, left-to-right binary powering initialized at the leading bit uses $\mathrm{bits}(e) - 1$ squarings and $\mathrm{popcount}(e) - 1$ multiplications. Two powers with exponent 11 therefore use six squarings and four multiplications. Four separate order checks use 24 squarings and 16 multiplications, but their moduli have only $10, 11, 12, 12$ bits; the packet modulus has 43 bits. Under the illustrative squared-bit weight for schoolbook squaring, the two costs are

$$6(10^2 + 11^2 + 12^2 + 12^2) = 3054, \qquad 6 \cdot 43^2 = 11094.$$

Thus fewer operations are not evidence of faster arithmetic. More generally, if $L_i = \mathrm{bits}(N_i)$ and $\mathcal{M}(L)$ denotes multiplication cost, the powering terms scale as

$$O\left(\log p \sum_i \mathcal{M}(L_i)\right) \quad \text{and} \quad O\left(\log p\, \mathcal{M}\left(\sum_i L_i\right)\right),$$

respectively, with CRT setup and gcds additional. Packing gives one algebraic verification object, not a free reduction of total bit complexity.

17.6 A12: an exact-cost finite-population screening planner

Why a deterministic formulation is useful. An assertion that one candidate is most likely prime requires a probability model and evidence supporting it. A different optimization is available without such an assertion. For a specified finite set of candidates, residue classes determine exactly which candidates a given sound small-factor screen rejects. One can optimize the screening plan for that finite population under an explicit cost model.

Let the candidate index set be U, let $R_j \subseteq U$ be the exactly known rejection set of screen j, and let $c_j \geq 0$ be its prescribed cost per surviving candidate. Let $C \geq 0$ be the terminal test cost per survivor. The terminal procedure may still return composite or unresolved; it is not assumed to accept every survivor. The objective is to process the whole finite population, not to minimize the random time to the first prime.

For a subset A of screens already applied, define

$$S_A = U \setminus \bigcup_{j \in A} R_j.$$

The recurrence

$$V(A) = \min \left\{ C|S_A|,\ \min_{j \notin A} (c_j|S_A| + V(A \cup \{j\})) \right\} \tag{35}$$

includes the option of stopping the filters immediately. Retain the minimizing decision at each subset to reconstruct an order. A screen with no useful additional rejection can be omitted.

Proposition 17.6 (Optimality in the stated finite model)**.** *The value $V(\varnothing)$ is the minimum cost among every ordered subset of the available screens followed by the terminal procedure.*

Proof. After a set A of screens, the survivor set is S_A, independently of their previous order. A plan either terminates now at cost $C|S_A|$ or chooses some next screen j, pays $c_j|S_A|$, and faces exactly the subproblem indexed by $A \cup \{j\}$. Induction on the number of unused screens proves (35) and its optimality. □

Procedure, complexity, and accounting. Compute the exact rejection sets from residue classes, store them as bitsets, evaluate the 2^h subsets for h screens, and recover the minimizing path. The planner is exponential in the number of available screens and is intended for a small menu of screen groups, not for one decision variable per prime below a billion. It uses no independence assumption between screens. Candidate-dependent costs can be substituted by replacing $c_j|S_A|$ and $C|S_A|$ with sums of the corresponding prescribed costs over S_A.

The rejection sets are already known at planning time. If obtaining them costs as much as running the screens, that expense is a real setup cost; it must be added when evaluating a production implementation. In the present experiment the cost objective deliberately describes only the subsequent execution. No held-out or future-population speed claim follows from optimizing these exact finite data. The optimal-substructure argument is standard dynamic programming, applied here to certificate-search filters.

Executed example. Use the 21 parameters of (30) with $F = 1009$ and $t = 0, \ldots, 20$. Consider the following prescribed abstract costs:

screen prime q	17	3	13	5	7
cost per survivor	5	1	4	1	2
initial rejection count	3	14	2	8	6

Let the terminal cost be 40. Applying all screens in the displayed order costs 279 units. The optimal plan applies the screens $3, 5, 7$, leaves parameters $4, 7, 10$, and costs

$$21 + 7 + 8 + 3 \cdot 40 = 156$$

units. There are 32 subset states. Enumerating all 326 ordered subsets independently confirms that no plan has lower cost in this model. These are exact modeled costs, not measured seconds, and the survivor $t = 7$ is not prime: the omitted screen 11 would eliminate it in A08. A smaller menu can be optimal under its own costs without becoming a complete primality test.

17.7 Reproducibility and what these six experiments establish

The standard-library reference script in Appendix C implements every module and raises an exception if any required identity fails. It contains the complete small inputs, digit sets, wheel primes, CRT packet, cost vectors, and fixed pseudorandom seed for arithmetic stress tests. No large integer is accepted on a probable-prime pass. The new prime claims in this section are backed by explicit order certificates; trial division is an additional check on the small examples.

The decisive reference output is

`ALL_SIX_ARITHMETIC_ALGORITHMS_VERIFIED`.

Its meaning is finite and specific: the digit join agrees with exhaustive enumeration, the quadratic root compiler agrees with finite differences, every routed divisor is exact, the signed reduction satisfies its range contract on the stated tests, the packet satisfies its order equations, and the screening planner agrees with exhaustive optimization. It does not establish universal performance rankings, a new giant-prime record, or an unconditional supply of prime values in any bounded family.

18 Certificate synthesis, hybrid bounds, and proof reuse

Algorithms A13–A18 operate on evidence as well as on integers. An order certificate can establish a statement about every prime divisor of an integer without establishing that the integer itself is prime. Keeping these two conclusions separate permits useful compositions: several weak certificates can give a stronger one, and a composite auxiliary integer can sometimes certify one of its own divisors. Every prime factor used in an order must already be certified, and all modular computations appearing in a certificate must actually be checked.

The algebraic ingredients are classical: multiplicative orders, primary decomposition in finite abelian groups, Hasse's bound, products and gcds, and shortest paths. The additions below specify concrete transformations, exact closure rules, and reproducible demonstrations. They are additions to this manuscript's algorithm catalogue; historical priority for these combinations is not asserted. In particular, they are not six independent replacements for the classical primality criteria.

18.1 A13: prime-power witness projection and fusion

Purpose and input. A conventional $N-1$ certificate may use a different witness for each prime factor of its known part. This algorithm combines those witnesses into one element whose order is a fully factored integer. Input an odd $N > 1$, a nonempty set S of certified primes dividing $E = N-1$, and witnesses a_q satisfying

$$a_q^E \equiv 1 \pmod{N}, \qquad \gcd(a_q^{E/q} - 1, N) = 1 \quad (q \in S).$$

The cofactor of E outside S need not be factored. Repeated exact division by each supplied q determines $e_q = v_q(E)$.

Proposition 18.1 (A fused factored-order witness)**.** *Set*

$$Q_q = q^{e_q}, \qquad H = \prod_{q \in S} Q_q, \qquad z_q = a_q^{E/Q_q} \bmod N, \qquad z = \prod_{q \in S} z_q \bmod N.$$

For every prime $\ell \mid N$, the reduction of z has order exactly H. Consequently

$$z^H \equiv 1 \pmod{N}, \qquad \gcd(z^{H/q} - 1, N) = 1 \quad (q \in S).$$

Every prime divisor of N is at least

$$\lambda(H) = \begin{cases} H+1, & H \text{ even}, \\ 2H+1, & H \text{ odd}, \end{cases}$$

and $\lambda(H)^2 > N$ proves that N is prime.

Proof. Fix $\ell \mid N$. The full-power and gcd conditions show that $h_q = \mathrm{ord}_\ell(a_q)$ divides E and has q-valuation exactly e_q. Raising a_q to E/Q_q removes every other primary part of h_q, so $\mathrm{ord}_\ell(z_q) = Q_q$. The orders Q_q are pairwise coprime. If $z^u = 1$, raising this identity to H/Q_q kills every component except z_q; since H/Q_q is coprime to Q_q, it follows that $Q_q \mid u$. Also $z^H = 1$ modulo ℓ. Thus $\mathrm{ord}_\ell(z) = H$. The stronger annihilation congruence modulo N follows directly from $z_q^{Q_q} = a_q^E \equiv 1 \pmod N$, since $Q_q \mid H$; it is not inferred merely by checking prime divisors. The reduced powers are nonidentity modulo every $\ell \mid N$, which gives the stated gcds. Finally, $H \mid \ell - 1$ and oddness of N give the bound $\lambda(H)$. □

Algorithm. Check all input witnesses and supplied prime factors; compute the Q_q and projections; multiply the projections modulo N; output (N, H, z) with the factorization of H. A separate verifier can discard the original witnesses and check the displayed H-power and gcd conditions directly. It accepts primality only when the size bound also holds. Otherwise the output remains a valid partial order certificate.

Exact examples. For $N = 31$, use $a_2 = 3$ and $a_3 = 21$. Multiplying these witnesses directly gives $3 \cdot 21 \equiv 1 \pmod{31}$ and loses all useful order. Projection instead gives

$$H = 6, \quad z_2 = 3^{15} \equiv 30, \quad z_3 = 21^{10} \equiv 5, \quad z \equiv 26 \pmod{31}.$$

The element 26 has order 6 modulo every prime divisor, and $7^2 > 31$ closes the certificate. A second example is

$$N = 1009, \quad (Q_2, Q_3) = (16, 9), \quad (a_2, a_3) = (11, 2),$$
$$(z_2, z_3) = (179, 759), \qquad (H, z) = (144, 655).$$

Both reduced gcds are 1 and $145^2 > 1009$.

Cost and limits. After the input checks, construction uses $|S|$ modular powers with exponents E/Q_q and $|S| - 1$ modular multiplications. The compact certificate stores one witness instead of one per prime. Verification uses exponent H and its factor quotients; intermediate powers may be shared. A smaller numerical exponent is not a proportional speedup, because exponentiation cost depends on its bit length. There is no universal running-time advantage, and this procedure does not manufacture valid input witnesses. The bounded verifier independently factors the small moduli in 86 nonempty test cases below 400 and checks the resulting orders at each prime divisor, including cases where the final size bound does not hold.

18.2 A14: batched unit checks with exact failure localization

Purpose and input. Pocklington reduced powers and elliptic denominators produce a collection of conditions $\gcd(u_i, N) = 1$ for one fixed modulus N. The values u_i must already be the correctly computed quantities required by the proof. This algorithm batches their unit checks; it does not batch or omit the underlying modular powers. Product-tree methods are classical tools in batch arithmetic; see, for example, the related constructions in [23].

Proposition 18.2 (Unit aggregation and diagnosis)**.** *For $N > 1$ and a finite nonempty collection $u_1, \dots, u_s$,*

$$\gcd\left(\prod_{i=1}^{s} u_i, N\right) = 1 \quad \Longleftrightarrow \quad \gcd(u_i, N) = 1 \quad \textit{for every } i.$$

All products can be reduced modulo N. A product tree permits exact localization of every failed leaf by descending only into nodes whose product has gcd greater than 1 with N.

Proof. A prime divisor of N divides the product precisely when it divides at least one factor. Hence a gcd-1 subtree contains no failed leaf. Recursively splitting every other subtree reaches precisely the indices with nonunit u_i. Reduction modulo N does not change any of these gcds. □

Algorithm and output labels. Build a balanced tree of products modulo N. Compute the root gcd. If it is 1, return `ALL_UNITS`. Otherwise descend into the affected subtrees and compute leaf gcds. A leaf gcd d with $1 < d < N$ is a proper factor and proves compositeness. A leaf gcd equal to N only invalidates that unit obligation: it can arise from an unsuitable witness even when N is prime. If no proper factor is exposed, return `NONUNIT_WITHOUT_PROPER_FACTOR`, not a compositeness assertion.

Exact examples. For the compact witness $(N, H, z) = (1009, 144, 655)$ in A13, the reduced quantities are 1007 and 373. One gcd of their modular product proves both are units. In contrast, for

$$N = 91, \qquad (u_1, u_2, u_3, u_4) = (2, 7, 13, 5),$$

the root gcd is the uninformative value 91; tree descent exposes the proper factors 7 and 13. For the prime modulus 101, the collection $(0, 2, 3, 4)$ has a nonunit leaf but yields no proper factor. This last case tests the essential distinction between a failed certificate and a composite integer.

Cost and limits. The successful path uses $s-1$ modular multiplications and exactly one gcd, replacing s separate gcds. Full failure localization uses at most $2s-1$ gcds, so the unsuccessful path need not be faster. No wall-clock speedup is claimed from these operation counts. The verifier compares all diagnosed leaves with separate gcds in 592 bounded batches and checks the three output states above. This is distinct from trial-dividing many candidate integers: all guards here belong to a single proof modulus.

18.3 A15: elliptic order fusion by primary projection

Purpose and input. Several points on the same elliptic curve may each have a certified order too small for the uniform Hasse threshold. Their combined order information can be much stronger. Input $N > 1$ with $\gcd(N, 6) = 1$, one curve E with discriminant coprime to N, and points P_i whose fully factored orders f_i have been certified modulo every prime divisor of N. Explicitly, the input checks are

$$[f_i]P_i = O, \qquad [f_i/q]P_i \text{ strongly nonzero for every } q \mid f_i.$$

All point arithmetic must be valid at every such prime divisor. Affine implementations therefore check every denominator for invertibility and reject unresolved exceptional cases. These are the same essential safeguards as in the Goldwasser–Kilian framework [5, 6].

Proposition 18.3 (Fusion of certified elliptic orders)**.** *Put $H = \operatorname{lcm}(f_1, \dots, f_t)$. For each $q \mid H$, select an index $i(q)$ with $v_q(f_{i(q)}) = e_q = v_q(H)$ and form*

$$Z_q = [f_{i(q)}/q^{e_q}]P_{i(q)}, \qquad R = \sum_{q|H} Z_q.$$

If the displayed arithmetic has been validated modulo every prime divisor of N, then R has order exactly H over each such residue field. Thus $H > (N^{1/4}+1)^2$ proves N prime. If the f_i are pairwise coprime, the simpler sum $R = P_1 + \cdots + P_t$ has order $\prod_i f_i$.

Proof. The input conditions make the reduction of P_i have exact order f_i. Consequently Z_q has exact order q^{e_q}. In the abelian group $E(\mathbb{F}_\ell)$, elements of pairwise coprime orders have a sum whose order is their product: applying H/q^{e_q} to an annihilating multiple isolates the q-primary component. Hence R has order H. Now $H \mid \#E(\mathbb{F}_\ell)$ for every $\ell \mid N$, and Hasse's bound gives $H \le (\sqrt{\ell}+1)^2$. A prime divisor at most $\sqrt{N}$ would contradict the stated threshold. The coprime-order special case uses the same group argument without primary projections. □

If all input certificates are retained, one may use $H \mid \#E(\mathbb{F}_\ell)$ directly without forming R. Constructing R is useful for emitting a compact, independently checkable single-point certificate. The final verifier can check $[H]R = O$ and the required strongly nonzero reduced multiples without trusting the creator's point construction. Orders from different curves must not be merged by this LCM rule.

Exact example and cancellation controls. For

$$N = 101, \qquad E : y^2 = x^3 - 4x - 2,$$

the points $P = (22, 16)$ and $Q = (40, 39)$ have certified orders 15 and 7, respectively. Their direct sum is $(69, 53)$ and has order 105. Neither input order meets the uniform Hasse threshold, since $(101^{1/4}+1)^2 > 16 > 15$. The combined order does; the stronger integral check $(\lfloor\sqrt{105}\rfloor - 1)^4 = 6561 > 101$ also holds. The generic primary-projection implementation produces the different point $(18, 1)$, again of exact order 105.

Overlapping orders require the projections. Using $T = (69, 53)$, the points $[7]T = (63, 11)$ and $[3]T = (65, 42)$ have orders 15 and 35, but their unmodified sum is $[10]T$ and has order 21, losing the factor 5. The projection algorithm instead returns $(75, 91)$ of order 105. The verifier also checks the extreme cancellation $P + (-P) = O$ and confirms that selecting one maximal primary component avoids this cancellation. On the composite modulus 35, adding the valid points $(0, 1)$ and $(5, 14)$ on $y^2 = x^3 + 1$ encounters denominator gcd 5; the guard rejects the operation and exposes the factor.

Cost and limits. Verification of the inputs uses their ordinary factored-order scalar multiplications. Construction adds one scalar multiplication per distinct prime of H and the corresponding point additions. Compact output verification depends on H and its distinct prime factors rather than on the entire construction history. This can strengthen available evidence from a largest individual order to their LCM, but it does not make the input points or their orders free to find. The examples establish correct fusion, not an ECPP running-time improvement.

18.4 A16: compiling residue constraints with exact Hasse windows

Purpose and input. The uniform elliptic bound discards all information about possible small prime divisors except their size. A known point order also localizes a small prime divisor

to short intervals. These intervals can be intersected with residue information from checked $N-1$ or $N+1$ certificates. This extends the earlier two-sided CRT closure by adding an elliptic constraint.

Input $N > 1$ with $\gcd(N, 6) = 1$, validated order witnesses implying

$$\ell \bmod M \in \mathcal{R} \quad \text{for every prime } \ell \mid N,$$

and a validated elliptic point of fully factored order H over every prime divisor. For example, an odd multiplicative part F supplies $M = 2F$, $\mathcal{R} = \{1\}$. A Lucas certificate supplies both possible signs as in theorem 2.3; distinct sign choices must be retained when merging constraints by the generalized Chinese remainder theorem. The order and residue premises are proof obligations, not user-supplied assertions accepted without verification.

Proposition 18.4 (Integer Hasse-window closure)**.** *Let*

$$S = \lfloor \sqrt{N} \rfloor, \quad U = S + 1 + \operatorname{isqrt}(4S), \quad T = \lfloor U/H \rfloor,$$

and for $1 \le t \le T$ *define*

$$I_t = \left[tH + 1 - \operatorname{isqrt}(4tH),\ tH + 1 + \operatorname{isqrt}(4tH) \right] \cap [2, S].$$

Every prime divisor of N *at most* S *lies in*

$$\mathcal{D} = \left\{ d \in \mathbb{Z} :\ d \in \bigcup_{t=1}^{T} I_t,\ d \bmod M \in \mathcal{R} \right\}.$$

Under the verified input premises, N *is prime if and only if none of the integers in* $\mathcal{D}$ *divides* N*. In particular,* $\mathcal{D} = \varnothing$ *is a primality certificate requiring no further trial divisibility checks.*

Proof. For $\ell \mid N$ with $\ell \le S$, the point-order condition and Lagrange's theorem give $\#E(\mathbb{F}_\ell) = tH$ for some positive integer t. Hasse's upper bound gives $tH \le U$, hence $t \le T$. The two-sided Hasse bound is equivalent to

$$(\ell + 1 - tH)^2 \le 4\ell \quad \Longleftrightarrow \quad (\ell - tH - 1)^2 \le 4tH.$$

Since all endpoints other than the square root are integers, this is precisely membership in I_t. The checked arithmetic witnesses give the residue condition. If N is composite, it has a prime divisor in $\mathcal{D}$; conversely, every divisor in $\mathcal{D}$ is between 2 and $\sqrt{N}$, hence is proper. This proves the equivalence. □

Algorithm. Verify the point, its prime factors, all unit guards, and the arithmetic witnesses. Construct the residue classes rather than trusting an asserted list. Generate the integer intervals I_t, intersect them with each residue progression, remove duplicates, and test the resulting divisors. A discovered divisor returns `COMPOSITE`; complete exclusion returns `PRIME`. Exhausting a finite computation budget before completing the set returns `INCONCLUSIVE`. The bounded implementation executes one multiplicative and one elliptic input certificate; the general statement permits any independently verified source of the residue restriction.

Corollary 18.5 (A constant-size final closure regime)**.** *If* $2H > U$ *and* $M > 2\operatorname{isqrt}(4H)$*, there is at most one integer to test per residue class in* $\mathcal{R}$*.*

Proof. The first inequality gives $T \le 1$. The untruncated interval I_1 has endpoint difference $2\operatorname{isqrt}(4H) < M$, so each residue class modulo M occurs at most once. □

This corollary concerns the final closure after the certificates are provided. It does not bound the cost of finding them. If H is comparable to $\sqrt{N}$, a modulus of size comparable to $N^{1/4}$ can leave only boundedly many final divisibility checks per residue class; it need not itself exceed $\sqrt{N}$. For smaller orders H, the number T can be large and direct window enumeration can be impractical.

A genuinely joint example. Take $N = 211$. The multiplicative witness $a = 2$ for $F = 5$ satisfies

$$2^{210} \equiv 1 \pmod{211}, \quad 2^{42} \equiv 107 \pmod{211}, \quad \gcd(106, 211) = 1.$$

Thus every prime divisor is 1 modulo 10, but the bound $11^2 > 211$ fails. On

$$E : y^2 = x^3 - 2x + 2, \qquad Q = (23, 27),$$

the verified affine point has order 19. This order also fails the uniform elliptic threshold: with $S = 14$, one has $U = 22 \geq 19$. The window computation, however, gives

$$T = 1, \quad I_1 = [20 - 8, 20 + 8] \cap [2, 14] = [12, 14].$$

No integer in $[12, 14]$ is 1 modulo 10. The combined certificate therefore proves 211 prime with zero final divisibility checks. The claim of jointness refers to these two uniform size tests; it is not a claim that no other elementary proof of this small prime exists. The verifier additionally compares the interval formula with direct Hasse-interval divisibility for all orders $2 \leq H < 120$ and hypothetical divisors $2 \leq d \leq 100$, without floating-point arithmetic.

18.5 A17: exact proof-network planning with shared ancestors

Purpose and input. Caching a previously proved prime avoids repeated verification. Choosing which certificates to use is a separate problem when each integer has several possible proofs and some proofs share ancestors. Input a finite catalogue of proof options. An option e has a conclusion $v(e)$, a set of required already proved nodes $D(e)$, a concrete certificate, and a nonnegative assigned cost $c(e)$. Fix independently verified anchors A and a target node. Costs describe a specified additive model; they are not automatically predictions of wall time.

Proposition 18.6 (Optimal verified-set planning)**.** *Use subsets V of the catalogue's nodes as states, starting with A. For each option with $D(e) \subseteq V$ and $v(e) \notin V$, add the edge*

$$V \longrightarrow V \cup \{v(e)\}, \qquad \text{cost } c(e).$$

A minimum-cost path to a state containing the target has minimum total cost among valid derivations in the supplied catalogue, counting each accepted conclusion once. No unanchored cycle can be used to justify itself.

Proof. A path gives an order of derivation in which every prerequisite is already established. Conversely, a valid derivation can be placed in a topological order. Repeatedly proving an established conclusion is unnecessary in the nonnegative additive model, so deleting such repetitions leaves a path of no greater cost. The minimum path therefore solves the stated optimization problem. A cyclic collection with no available first step does not create a transition from the anchored initial state. □

Algorithm and verification. Run Dijkstra's algorithm [24] on these subset states, recording a predecessor option for each improved distance. Reconstruct the path, then verify each actual certificate in path order before admitting its conclusion. A failed certificate invalidates that plan and must be removed or repaired before replanning. The optimality claim assumes a valid catalogue and its fixed cost model; the acceptance check does not assume that an unverified catalogue is correct.

A shared-ancestor example. The anchors consist of 2. The catalogue contains valid Pocklington proofs

$$2 \longrightarrow 3, \qquad 3 \longrightarrow 7, \qquad 3 \longrightarrow 13, \qquad \{7, 13\} \longrightarrow 2003.$$

The first three use base 2 and known parts $2, 3, 3$, respectively; the last uses base 3 and $F = 7 \cdot 13 = 91 \mid 2002$. Give these four options costs $7, 1, 1, 1$. Also permit direct small-number trial proofs of 7 and 13, each assigned cost 6.

Optimizing the two branches independently chooses the direct proofs and spends $6+6+1 = 13$ units. The state-based planner instead proves 3 once and spends $7 + 1 + 1 + 1 = 10$ units. These are deliberately assigned model costs, not measured timings for these tiny primes. All selected certificates are verified, and exhaustive enumeration of all 64 subsets of the six options confirms the optimum 10. A separate unanchored two-node cycle is rejected.

Cost and limits. For v nodes and h options there are at most 2^v states and $h2^v$ transitions. A standard heap implementation takes $O(h2^v \log(2^v))$ planning time apart from certificate arithmetic. This is exact planning for a small portfolio, not a polynomial-time solution for arbitrary large proof networks. If real execution costs depend on cache contents or memory pressure beyond the verified-node set, the state and cost model must be enlarged before claiming optimality.

18.6 A18: inheriting a prime certificate through an extracted divisor

Purpose and input. A candidate that cannot be certified as a whole can still contain a new certified prime. The useful input is not merely a failed primality test: it is a *verified* statement that every prime divisor of an auxiliary integer N is at least L. Such a statement can come from a valid partial order certificate, even when N is composite. An exact divisor D must also be obtained, for example by a proper gcd exposed during certificate construction.

Proposition 18.7 (Hereditary divisor certification)**.** *Suppose every prime divisor of $N > 1$ is at least $L \geq 2$. If*

$$1 < D \mid N, \qquad D < L^2,$$

then D is prime. The lower-bound evidence for N and the exact divisibility and size checks suffice: no new modular exponentiation with modulus D is required. If $L > P$ for a designated input prime P, then the harvested prime is larger than P.

Proof. Every prime divisor of D also divides N, hence is at least L. If D were composite, two of its prime factors counted with multiplicity would give $D \geq L^2$. This contradicts the strict size condition. The same inheritance gives $D \geq L > P$. □

Algorithm. First verify the partial order or other lower-bound certificate on N. Record the resulting L as a proved predicate, not as an unchecked parameter. Extract or receive a proposed divisor and check it by exact division. Accept its primality only if it lies strictly

between 1 and L^2. For a factorization into several extracted divisors, reuse the same verified lower bound for every divisor that meets this condition. Divisors outside the interval remain unresolved and require other certificates. The prime status of N itself is never an input assumption.

Exact example. Start from the certified prime $P = 3$ and the auxiliary lift

$$N = 154P^2 + 1 = 1387.$$

Base 2 supplies the valid partial $F = 9$ certificate

$$2^{1386} \equiv 1 \pmod{1387}, \qquad 2^{462} \equiv 1322 \pmod{1387}, \qquad \gcd(1321, 1387) = 1.$$

Thus every prime divisor is 1 modulo 9 and, by oddness, is at least 19. The whole-modulus size bound fails since $1387 > 19^2$. A further reduced power gives

$$2^{693} \equiv 512 \pmod{1387}, \qquad \gcd(511, 1387) = 73,$$

and exact division yields $1387 = 73 \cdot 19$. Both divisors are below $19^2 = 361$, so both 73 and 19 are proved prime by inheritance. The two child conclusions use zero additional modular powers after the parent order evidence and factor extraction have been checked.

Cost and limits. The extra child verification consists of exact division and size comparisons; the expensive order checks are shared. Finding a useful divisor can nevertheless be difficult. Most ordinary composites rejected by a Fermat test do not come with the required partial order certificate, so this algorithm does not automatically recover primes from all failed search candidates. In particular, the previously logged Fermat-failing large candidates cannot be retroactively accepted by this argument without new evidence. The restricted quadratic range of A01 already excludes composite base-2 passes under its hypotheses; the present example uses a larger coefficient outside that range. The verifier rejects the boundary divisor $D = L^2$ and proposed integers that do not divide N.

Executed evidence for A13–A18. The complete standard-library implementation is included in appendix D. Its successful status is `ALL_SIX_CERTIFICATE_ALGORITHMS_VERIFIED`. The tests cover exact examples, independent small-number factorizations, 592 product-gcd comparisons, overlapping elliptic orders, invalid denominators, 118 integer Hasse-window comparisons, all 64 proof-option subsets in the planning example, and strict inheritance boundaries. These bounded computations check the stated algorithms and their failure semantics. They are not a record-size search or a benchmark establishing an asymptotic or universal speed advantage.

19 Integrated experiments and independent checks

19.1 What was actually executed

All eighteen modules have executed reference implementations. The construction experiment starts by proving $P = 2^{127} - 1$ with the Lucas–Lehmer recurrence, including a trial-division proof that 127 is prime. Its principal outputs are listed in Table 6. They are exact certified integers, not probable-prime passes. No statement is made that these integers were previously unknown.

Table 6: Executed construction outputs for the eighteen-module revision.

Module	Exact specification	Decimal digits
A01	$180P^2 + 1$	79
A02	$82P^3 + 1$	117
A03	$1104P - 1$ and $1104P + 1$	42 each
A04	Prefix 314159, suffix 27183, middle width 90; exact progression parameter $j = 195$	101
A05	$601, 1801 \mid \Phi_{25}(2)$; $727 \mid \Phi_{121}(2)$	3, 4, 3
A06	$678P^2 - 1$	80

For A04, the integer and all its leading zeroes inside the middle block are fixed by the formula and constants in the construction section and Appendix B. A04 does not claim that any arbitrarily chosen prefix and suffix admit a prime in every bounded middle interval.

The arithmetic and certificate modules have complementary demonstrations. A07 certifies 7919; A08 certifies 42379 and 223999; A11 certifies the packet $727, 1453, 2179, 2663$. A15 certifies 101 by fused elliptic order 105. A16 certifies 211 by intersecting two individually insufficient uniform bounds. A18 certifies 19 and 73 as exact divisors of the composite auxiliary integer 1387. Other modules operate on these same examples rather than each claiming a different prime. These overlapping outputs are not added together as a discovery count.

19.2 An alternate certificate for the existing large chain

The quadratic members $p_1, \dots, p_5$ of the continuing chain have coefficients

$$438, \quad 13696, \quad 5494, \quad 13258, \quad 22330.$$

Each coefficient is positive, even, and smaller than $4p_{i-1} - 2$. Their existing full base-2 Fermat congruences, together with $2^{a_i} \not\equiv 1 \pmod{p_i}$, therefore give A01 certificates. The retained seven-node verifier checks the short nonunity directly, in addition to independently checking the original Pocklington gcd.

For the 18,882-digit endpoint

$$p_5 = 22330p_4^2 + 1,$$

even that short modular power is unnecessary in a minimal A01 certificate: the exact size comparison $1 < 2^{22330} < p_5$ proves nonunity. The required long calculation is still $2^{p_5-1} \bmod p_5 = 1$. Thus A01 provides a shorter logical certificate for a previously certified large integer; it neither discovers a new integer nor eliminates the large modular calculation. If the original Pocklington gcd for the same base is already available, it too implies short nonunity, since $2^{a_i} \equiv 1$ would imply $2^{a_i p_{i-1}} \equiv 1$.

The stopped family $N = 2mp_6^2 + 1$, $1 \le m \le 10^6$, also satisfies the A01 coefficient bound. A future full Fermat pass together with short nonunity would therefore be a proof in this family. Every durable completed test in the interrupted run was a Fermat failure, so this observation does not change that run's outcome.

19.3 Independent oracles and deliberately invalid inputs

The reference programs do more than reproduce their own formulas. They compare different algorithms or use exact small-instance oracles. The principal checks are recorded in Table 7. An acceptance count below counts certificate attempts, possibly including several witnesses for the same integer.

Table 7: Finite verification evidence and its precise scope.

Component	Executed check
Original inventory	In the preceding edition, the unchanged standalone listing certified all 32 original integers again.
Retained seven-node chain	In the preceding edition, seven exact nodes, through 18,885 digits, passed Pocklington and decimal-identity checks. A01 also verified the five quadratic steps.
Current large-prime register	Ten nodes through 18,886 digits, including five at or above 18,000 digits. An eight-node exact run and two subsequent leaf certifications checked every Pocklington obligation; two prior PRP hits now have full certificates.
A01, A02, A06	Construction verifier compared respectively 4,144, 672, and 1,981 witness attempts with trial division.
Lucas and cyclotomic arithmetic	1,953 matrix-versus-recurrence checks and 744 modular geometric-sum checks.
Independent boundary audit	Separately implemented checks: 8,272 positive and 517 negative attempts; 1,364 and 98 valid certificates, respectively; zero false prime certificates.
A07, A08	Digit join versus exhaustive words; quadratic root compiler versus finite differences, including small characteristic.
A09, A14	Exact proper-factor routing; 592 independent batches compared with the gcd of each leaf.
A10	193,871 exhaustive canonical products, 768 fixed-seed stress products through 4,096-bit radix, and 128 modular powers.
A11, A13	Explicit four-number packet and rejected malformed packets; 86 small odd moduli audited by independent factorization.
A15, A16	Invalid curves, wrong orders, nonunit denominators, and point-cancellation controls; 118 exact Hasse-window comparisons.
A12, A17	All 326 ordered screen subsets and all 64 certificate-edge subsets in their respective finite examples.
A18	Exact inherited divisors; nondivisors and equality at the strict square boundary rejected.

The mathematical proofs establish the general implications. Finite tests check that the implementations match those implications on stated inputs; they are not substitutes for proofs. Particularly useful controls are the composite integers 91, 35, and 125, which satisfy tempting weakened versions of A01 or A06. The signed reduction counterexample $N = 5$, $x = 24$ demonstrates why its product-domain restriction cannot be silently enlarged.

The older elapsed times in the baseline sections remain historical observations. The repeated baseline standard-library verification took 135.055 seconds in the preceding revision. The retained seven-node verification used GMP through the optional Python interface; its exact observed duration is reported in the execution ledger. These different workloads and arithmetic backends are not a comparative benchmark. The reference prototypes completed quickly in their recorded environment, but no record-scale speed claim is extrapolated from them.

20 How the modules can be combined

20.1 Three concrete workflows

Known-prime quadratic search. Choose a certified p and a finite even-coefficient window. Compile small-factor exclusions using A08, or apply A09 when many explicit candidates are already available. A12 may choose a subset of screen groups under an explicit cost model.

For a surviving $ap^2 + 1$ inside A01's range, a full Fermat pass and short nonunity finish the proof. A02 or the baseline order criteria are alternatives when the chosen candidate family has different structure. The constructor need not invoke every module.

Human-specified digit pattern. Use A04 for fixed end blocks or A07 for a more general digit language. Both enforce an appropriate known factor of $N - 1$ while constructing candidates. Apply the small-factor screens, then an exact order certificate. A13 can combine independently supplied primary witnesses; A14 can verify their nonvanishing obligations through one successful product gcd. The digit pattern shapes the output, while the order certificate proves its primality.

Partial arithmetic and elliptic evidence. Verify a universal arithmetic residue constraint and one or more elliptic point orders. A15 combines useful primary components into a larger known order; A16 turns that order into exact Hasse windows and intersects them with the arithmetic classes. If a proper factor is discovered during the computation, A18 can certify a sufficiently small divisor using a previously verified universal lower bound. A17 plans dependencies among already available finite certificate proposals. It does not manufacture the missing witnesses or predict their search cost.

20.2 The meaningful sense of a new composition

There are three distinct mathematical operations here. Candidate construction imposes constraints on N itself. Order fusion increases a proved order in a finite group. Constraint intersection narrows the set of possible prime divisors of N. Their combination is useful because they act on different objects, but their types and hypotheses must be preserved.

For example, A16's $N = 211$ demonstration begins with $\ell \equiv 1 \pmod{10}$ for every prime divisor and with an elliptic point of exact order 19. A hypothetical divisor $\ell \le \lfloor \sqrt{211} \rfloor = 14$ must lie in $[12, 14]$. This interval contains no integer congruent to 1 modulo 10, so the intersection is empty. Neither uniform size bound alone suffices. The new verification object is the combination of the two checked constraints, not a guess that the candidate is unusually likely prime.

These procedures are implemented additions to this manuscript. The coefficient inequalities, finite closures, and composition rules are proved in full. Their historical novelty is not settled by a search for matching terminology, and no priority claim is made. In particular, classical Pocklington, Lucas, cyclotomic-order, CRT, dynamic-programming, and elliptic principles remain credited to their established sources.

21 Efficiency, digit blocks, and the Lucas–Lehmer comparison

Let $L = \mathrm{bits}(N)$, and let $\mathcal{M}(L)$ denote the cost of multiplying L-bit integers. A full modular exponent whose exponent has $O(L)$ bits requires $O(L)$ modular arithmetic steps by binary powering. With a suitable reduction implementation, the corresponding cost is conventionally expressed as $O(L\mathcal{M}(L))$, with gcd, setup, and certificate-factor costs accounted for separately. A01 can remove a long reduced-power obligation in its specific coefficient range; it does not reduce the remaining full exponent to $O(\log L)$ steps.

For a Mersenne candidate $2^r - 1$ with prime exponent r, Lucas–Lehmer uses $r - 2$ iterations of $s \leftarrow s^2 - 2$ modulo that candidate. The modulus permits particularly efficient special reduction, and implementations use fast multiplication [26]. The present paper contains

no matched benchmark establishing that its general lifting certificates equal or beat that specialized test. Having the same broad arithmetic order of growth does not establish equal constants, memory use, or optimized throughput.

Large integers can indeed be processed in blocks. GMP represents them using limbs, and its large multiplication algorithms split inputs, transform or evaluate pieces, multiply, and recombine [25]. A10 adds a proved reduction identity for a particular modulus shape. These are ways to compute exact global arithmetic. They do not reduce primality to independent primality decisions on decimal chunks: $2323 = 23 \cdot 101$ has two prime two-digit chunks, whereas the prime 1009 has chunks 10 and 09, neither prime.

There is also a difference between reducing the number of arithmetic operations and reducing total work. A11 changes four small-modulus verifications into one larger-modulus packet. Its demonstration reduces the count of squarings from 24 to 6, while the illustrative squared-bit cost increases from 3054 to 11094. A12's 279 to 156 improvement is in assigned model units, not measured seconds. A17 optimizes a finite certificate proposal graph and is exponential in the number of nodes. These limitations are part of the algorithms' stated contracts.

The practical next implementation step is therefore a matched benchmark for a chosen candidate family and certificate route, including failed candidates, sieve setup, arithmetic width, and independent verification. This revision supplies exact prototypes and proofs for that work. It does not claim that applying more filters, doubling the candidate's digit length after each failure, or renaming classical operations yields an automatic supply of giant primes.

22 Reproducing the expanded manuscript

The complete deliverable consists of one PDF and this one TEX file. Compilation needs ordinary LaTeX packages and no external figure, bibliography database, numerical data file, or included source file. Run `pdflatex Certified_Prime_Lifting.tex` at least twice, or use `latexmk -pdf Certified_Prime_Lifting.tex`.

22.1 Extracting the exact programs

Appendix A retains the original verifier byte for byte. Appendices B, C, and D contain the three six-module verifiers; Appendix E verifies the continuing large chain; Appendix F supplies the independently implemented boundary audit. Appendix G adds the ten-node large-prime register verifier. Every listing has its own imports, constants, and entry point. Extract them into separate directories because two programs write an adjacent `results.json`. Save and run the following extractor beside this TEX file:

```
from pathlib import Path
import re

source = Path("Certified_Prime_Lifting.tex").read_text(encoding="utf-8")
items = [
    ('verifier', 'baseline', 'verify_baseline.py'),
    ('constructions', 'constructions', 'verify_constructions.py'),
    ('arithmetic', 'arithmetic', 'verify_arithmetic.py'),
    ('certificates', 'certificates', 'verify_certificates.py'),
    ('later-chain', 'later_chain', 'verify_later_chain.py'),
    ('boundary-audit', 'boundary_audit', 'independent_boundary_audit.py'),
    ('large-register', 'large_register', 'verify_large_register.py'),
]
for label, folder, filename in items:
    pattern = (r"\\begin\{lstlisting\}\[label=lst:" + re.escape(label)
               + r"\]\n(.*?)\\end\{lstlisting\}")
    match = re.search(pattern, source, re.S)
    if match is None:
```

```
        raise ValueError("Missing listing: " + label)
    target = Path("verifiers") / folder / filename
    target.parent.mkdir(parents=True, exist_ok=True)
    target.write_text(match.group(1), encoding="utf-8")
    print(target)
```

The three retained module verifiers and the boundary audit use only Python's standard library. Run, for example,

```
python3 verifiers/constructions/verify_constructions.py
python3 verifiers/arithmetic/verify_arithmetic.py
python3 verifiers/certificates/verify_certificates.py
python3 verifiers/boundary_audit/independent_boundary_audit.py
```

Their required final status strings are, respectively,

```
A01_A06_ALL_EXACT_CHECKS_PASSED
ALL_SIX_ARITHMETIC_ALGORITHMS_VERIFIED
ALL_SIX_CERTIFICATE_ALGORITHMS_VERIFIED
INDEPENDENT_BOUNDARIES_VERIFIED
```

For the larger inherited inventories, run

```
python3 verifiers/baseline/verify_baseline.py
python3 verifiers/later_chain/verify_later_chain.py
```

The respective success states are

```
ALL_32_PRIMES_CERTIFIED
ALL_SEVEN_PRIMES_CERTIFIED
```

The latter program defaults to standard Python integers. If `gmpy2` is separately installed, the optional `-gmp` flag accelerates the same exact checks. The mathematical certificate does not depend on that optional backend.

For the complete current large-prime register, run

```
python3 verifiers/large_register/verify_large_register.py
```

Its success status is `ALL_TEN_PRIMES_CERTIFIED`. It verifies the shared ancestors once, then the four linear siblings of p_5. This program automatically uses `gmpy2` if installed and otherwise uses exact standard-library integers; it does not require the old seven-node verifier or an external checkpoint file.

22.2 Execution ledger and source identities

The reference environment used Python 3.12.14 on x86-64 Linux. In the preceding algorithm edition, the unchanged baseline was rerun successfully in 135.055 seconds. The seven-node chain, including the additional A01 checks, was rerun successfully with the optional GMP backend in 38.024 seconds. These are observations from different workloads, not a timing comparison between algorithms. All three final module listings and the independent boundary listing were extracted from the integrated TEX and executed successfully in that edition.

For this register update, the eight-node core and two additional leaves passed the exact checks recorded in Section 14.1. The final ten-node listing consolidates those executed obligations and retains the core arithmetic unchanged. Its syntax, node parameters, and all five fingerprints

were checked; no single full ten-node execution time is claimed. All six earlier program bodies and their hashes are preserved byte for byte.

The SHA-256 values below identify the exact UTF-8 program text, including its final newline. They detect accidental edits and support reproducing this revision; they do not replace the mathematical verification.

baseline

```
2dbc720444cefb09c060df3f6c4fd16c359abfc2b9df5a65a28574d7de05a68a
```

constructions

```
13a39c5f4470f508f4d012c15b3041c61c1b04d9ddad6a4d9a5d820f75f9db7b
```

arithmetic

```
0e92435d4089f559e87aac9b7c16cfdd09b0210f71b7c41497db96650c4f2154
```

certificates

```
ac90cdca390824689358c7a79bd264ac83b4d2e8042820f7acef995bd506f144
```

later_chain

```
8ac7bdfe891112e210eb5b366ada19ec2aa6c42cc3d574e1ae10849508f59509
```

boundary_audit

```
15f0a3e548dc45759610250742f9c8aa91314bfd623dd6d1d82d0fbee57d7435
```

large_register

```
4d104340ec995054a160d90aa3a4260eafd8e07080f8de47e5d602119206ad25
```

References

[1] H. C. Pocklington, *The determination of the prime or composite nature of large numbers by Fermat's theorem*, Proceedings of the Cambridge Philosophical Society **18** (1914), 29–30.

[2] J. Brillhart, D. H. Lehmer, and J. L. Selfridge, *New primality criteria and factorizations of* $2^m \pm 1$, Mathematics of Computation **29** (1975), no. 130, 620–647. https://doi.org/10.1090/S0025-5718-1975-0384673-1.

[3] M. A. Morrison, *A note on primality testing using Lucas sequences*, Mathematics of Computation **29** (1975), no. 129, 181–182.

[4] U. M. Maurer, *Fast generation of prime numbers and secure public-key cryptographic parameters*, Journal of Cryptology **8** (1995), no. 3, 123–155. https://crypto.ethz.ch/publications/Maurer95a.html.

[5] S. Goldwasser and J. Kilian, *Primality testing using elliptic curves*, Journal of the ACM **46** (1999), no. 4, 450–472. https://doi.org/10.1145/320211.320213.

[6] A. V. Sutherland, *18.783 Elliptic Curves, Lecture 11: Primality proving*, Massachusetts Institute of Technology, 7 March 2022. https://math.mit.edu/classes/18.783/2022/LectureSlides11.pdf.

[7] R. Bröker and P. Stevenhagen, *Constructing elliptic curves of prime order*, in *Computational Arithmetic Geometry*, Contemporary Mathematics **463**, American Mathematical Society, 2008, 17–28. https://arxiv.org/abs/0712.2022.

[8] C. Caldwell, *Primality proving:* $N-1$ *tests;* $N+1$ *tests and the Lucas–Lehmer test*, PrimePages exposition. Accessed 5 November 2026. https://t5k.org/prove/prove3_1.html; https://t5k.org/prove/prove3_2.html.

[9] GIMPS, *List of known Mersenne prime numbers*. Source snapshot consulted 5 November 2026. https://www.mersenne.org/primes/.

[10] Y. Gallot et al., $b^{2^n} - b^{2^{n-1}} + 1$, PrimeGrid discussion thread 6062, beginning 27 December 2014. https://www.primegrid.com/forum_thread.php?id=6062.

[11] PrimePages, $10223 \cdot 2^{31172165} + 1$, database entry 122473, proven status and decimal digit count. Accessed 5 November 2026. https://t5k.org/primes/page.php?id=122473.

[12] PrimePages, *The largest known primes*, list snapshot dated 5 November 2026, 14:37:39 UTC. https://t5k.org/primes/lists/all.txt.

[13] PRST project, *Primality tests for numbers of special forms*, software repository. The bounded record-size experiment used the binary labelled 14.0.1888. https://github.com/patnashev/prst.

[14] J. Maynard, *Dense clusters of primes in subsets*, Compositio Mathematica **152** (2016), no. 7, 1517–1554. https://arxiv.org/abs/1405.2593.

[15] Keio, *Recurrences, Absolute Differences, and Collatz Orbits: Algebra, Prime Patterns, and Zero Distributions*, manuscript `[072] unified_collatz_recurrences.tex`, in *Verifiable Mathematical Reasoning Corpus*, commit `5936298ed4bacb7fb1423b889f02278fa521ea16`. Relevant sections: "Pascal–Franel weights and a certified ten-prime trajectory" and "Primality-certificate barriers in Mersenne inverse orbits." Pinned repository snapshot. This manuscript is cited for the explicit constructions audited here, not as blanket validation of the repository.

[16] K. Saito, *Mills' constant is irrational*, Mathematika **71** (2025), no. 3, e70027. https://doi.org/10.1112/mtk.70027; https://arxiv.org/abs/2404.19461v2.

[17] K. Saito, *Transcendency of variants of Mills' constant*, arXiv:2508.16068v3, 7 December 2025. The version is fixed because the theorem statements changed across revisions. https://arxiv.org/abs/2508.16068v3.

[18] Ariko Stephen Philemon, *Primality of numbers of the form* $ap^k + 1$, arXiv:2005.02327v5, 10 April 2021. https://arxiv.org/abs/2005.02327v5. Theorem 4.4 supplies the earlier $a < p$ quadratic special case discussed here; the coefficient barriers in this revision are proved independently.

[19] E. Horowitz and S. Sahni, *Computing partitions with applications to the knapsack problem*, Journal of the ACM **21** (1974), no. 2, 277–292. https://doi.org/10.1145/321812.321823. Author-hosted text: https://www.cise.ufl.edu/~sahni/papers/computingPartitions.pdf.

[20] T. Granlund and P. L. Montgomery, *Division by invariant integers using multiplication*, Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation (1994). https://doi.org/10.1145/178243.178249. https://gmplib.org/~tege/divcnst-pldi94.pdf.

[21] FLINT developers, *Multimodular reduction and Chinese remaindering*, official documentation, accessed 6 November 2026. https://flintlib.org/doc/mpn_extras/multi_crt.html.

[22] FLINT developers, *fq_sqrt: square roots in finite fields*, official documentation, accessed 6 November 2026. https://flintlib.org/doc/fq/fq_sqrt.html.

[23] FactHacks, *Batch gcd*, primary project documentation accompanying the 29C3 talk *FactHacks: RSA factorization in the real world*. https://facthacks.cr.yp.to/batchgcd.html. See also *Batch trial division*, https://facthacks.cr.yp.to/batchtrial.html.

[24] E. W. Dijkstra, *A note on two problems in connexion with graphs*, Numerische Mathematik **1** (1959), 269–271. https://doi.org/10.1007/BF01386390.

[25] GNU MP developers, *GNU MP Manual: Nomenclature and Types; FFT Multiplication*, official documentation, accessed 6 November 2026. https://gmplib.org/manual/Nomenclature-and-Types; https://gmplib.org/manual/FFT-Multiplication.

[26] GIMPS, *Mathematics and Research Strategy*, official project documentation, last updated 16 October 2024; accessed 6 November 2026. https://www.mersenne.org/various/math.php.

A Complete standalone certificate verifier

The following single program constructs and certifies all 32 integers. Large literals are confined to the norm representations, elliptic coordinates, and metadata hashes. Their inclusion makes the TEX source self-contained; there are no downloads or external certificate files. For readability in print, long program lines may wrap visually; extraction from the TEX source preserves the original Python line breaks.

```
"""Complete constants and symbolic constructions for the 32-prime corpus.
No N decimal expansions, external data files, or primality tests are used.
Q coordinates and norm representations are the only large literals.
Factor tuples are (prime-id-or-small-integer, exponent, witness base).
This file constructs integers; a separate verifier checks primality.
"""
from math import prod

MERSENNE_EXPONENTS = (521, 1279, 4423)
PROTH_SEED = (401713, 3300, 3)
CHAIN_COEFFICIENTS = (438, 13696, 5494, 13258)
COLLATZ_R = 98723
PROTH_OUTPUT = ("mersenne_to_proth", "M1279", 2158, 3)
# (name, factors, lower bound: F+1, or 2F+1 for odd F)
N_MINUS_ONE = (('p0', ((2, 256, 11), (7, 256, 2)), 'plus_one'),
 ('p1', (('p0', 2, 2),), 'plus_one'),
 ('p2', (('p1', 2, 2),), 'plus_one'),
 ('p3', (('p2', 2, 2),), 'plus_one'),
 ('p4', (('p3', 2, 2),), 'plus_one'),
 ('power4', (('M1279', 4, 2),), 'plus_one'),
 ('mixed', (('M4423', 1, 2), ('Proth1000', 1, 2)), 'plus_one'),
 ('near_square', (('M4423', 1, 2),), 'odd'),
 ('linear_2214_p3', (('p3', 1, 2),), 'plus_one'),
 ('Phi3_M521_r260', ((2, 2, 2), (5, 1, 3), (13, 1, 2), ('M521', 1, 2)), 'plus_one'),
 ('Phi3_M521_r78299', ((13, 1, 2), (19, 1, 2), (317, 1, 2), ('M521', 1, 2)), 'plus_one'),
 ('Phi7_M521_r78299',
  ((13, 1, 2), (19, 1, 2), (317, 1, 2), ('M521', 1, 2), ('Phi3_M521_r78299', 1, 2)),
  'plus_one'),
 ('Phi3_M1279_r4683', ((3, 1, 2), (7, 1, 5), (223, 1, 2), ('M1279', 1, 2)), 'plus_one'),
 ('Phi3_M1279_r48909',
  ((3, 1, 2), (7, 1, 2), (17, 1, 2), (137, 1, 2), ('M1279', 1, 2)),
  'plus_one'),
 ('Phi7_M1279_r48909',
  ((3, 1, 5),
   (7, 1, 2),
   (17, 1, 2),
   (137, 1, 2),
   ('M1279', 1, 2),
   ('Phi3_M1279_r48909', 1, 2)),
  'plus_one'))

N_MINUS_ONE += (("Collatz_L0", (("M1279",1,2),), "plus_one"),
                ("Collatz_L1", (("M1279",1,2),), "plus_one"))

# (name, factors, delta, d): N=F^3-delta*F^2-d*F+1
CUBIC = (('cube_m1279', (('M1279', 1, 2),), 0, 1971),
 ('cube_m4423', (('M4423', 1, 2),), 0, 15751),
 ('cube_proth', (('Proth1000', 1, 2),), 2, 1315),
 ('cube_product', (('M1279', 1, 2), ('Proth1000', 1, 2)), 0, 6529))

# (name, seed, multiplicity, Lucas A, Lucas Q, criterion)
LUCAS = (("lucas_m1279", "M1279", 2, 3, 1, "rank"),
         ("negative_cube_Proth1000", "Proth1000", 1, 9, 1, "negative_cube"))
# (name, F-id, G-id, j offset, Fermat base, Lucas A, Lucas Q)
CRT = ("TwoSidedCRT", "M1279", "Proth1000", 963, 2, 5, 1)
# (Mersenne exponent, multiplier, whether Phi_7 is also certified)
CYCLOTOMIC = ((521,260,False),(521,78299,True),
              (1279,4683,False),(1279,48909,True))

CM_C_A = int(
    "3106317625883058783437871637738128657175216387920917302288777637890344 95"
```

```
    "954192907319663768591665987731375923749696423700820569088044808879886822"
    "258964235435536062255948339117868667989870666141079852134124567709538406"
    "111829961059967601471937794099279664347282513320846348431308128221727386"
    "304456250215307011466999782784002159857277076988608387962314279762358437"
    "670945365766441423864654270209319245536986659580841360980358909041299475"
    "779934503584136796454512583491287663549076082784022726337039640491 20"
)
CM_C_B = int(
    "646394448418340813587086397455518241093626160973699044592409376946072812"
    "927542095303134986263719079210603866810486549745106402316447944647298652"
    "273501513907108885381875336005647835807391259856661179973876073735053139"
    "548478658554567266954915667679810463290320134325911326318563117849384230"
    "803896882501919744227857224518960422919908511746619077843495351690238831"
    "032042640196794609549495307918566284724809952407098795998911706752193668"
    "331941455270508752519677908091088521995723297651867704108688255436 7"
)
CM_M_A = int(
    "125556301470467014213504119369214331566866510773858216486523063427163211"
    "730579729456184100200351588736361996066221109587475994042391413331307591"
    "488986155618273122883086403005781950490219566090 2"
)
CM_M_B = int(
    "171575966758849216072637266161917841026461861951504617224221626925195888"
    "939795367706262827187804862499320222886436892319532435202804868666865774"
    "863780267956499892756686705339702316917623105961 9"
)
CM1006_QX = int(
    "673328134373210046801457714014713279690189557668434309245922982534583515"
    "518159955843849693092589324214765684678235330557550525951029113487797472"
    "196288128368778431073692784112749595006186913490078007588504681706912446"
    "418675148430876068556139380879014814699613260676910273795841331548120786"
    "714646884099019442644671804074540609948429838015740516018093319758648102"
    "487808272682646570058786465803905579113640157932982668119697766822014200"
    "787584782907388197983483305308865883094283217040531813846955673280461999"
    "193689761980489930703563809886838460170213392325252669699327746831297726"
    "525858239266521888258778815306775417997899734778913812868273864954841532"
    "715556439956437201214505846689194679547303918549642110545341905703372353"
    "494214164696999257898989140686838100928179988046540138545993346187657388"
    "150002305622123517073683259161470309562935686280479929943675553884761917"
    "550888736676924031296152313975089887877975306914337333514367707611561032"
    "681279920125987675102110465767990382766642915982521159943931 4951999606"
)
CM1006_QY = int(
    "295791267571015429399469841243025383457072761688802342862952596919286469"
    "638058812827729877323766676884829956394387139145544258435255240879250721"
    "561464817943649508278360054484248576537898356358549201494701916930496028"
    "494849275294493178497330039739734043145259689049222217345889656286031738"
    "589274684125611613988099130701869179880900506558570203627692684361803010"
    "644485615962123336772323563635516818534383619904642309448548278185411614"
    "741003751218094683567875455994619298970591757245688914985223908555365385"
    "191002341200848191744848529850397073202431827243246170146490498303281303"
    "302191106984361571820378666192831359056899116184768546128375004664325306"
    "968536635670478982981186658495835983465316179159592959814113806340668351"
    "938351911982581859637616142991972505184218472236304654236874288510854233"
    "465376310771296059472653935104672004428066494745951036411407358257415199"
    "362625082227105501881474747111492858205509802319100847367937472511625204"
    "219994565444297743999297022108180659686620854928828568858935293587013 2"
)

CM1993_QX = int(
    "672120230134223320956150163200363857064374771257409266849368813848297679"
    "034741202886841462132176236840867912598995394451725726996937213675418436"
    "570148051371769361857073007986498823694298158084469098721463990653817048"
    "733634626074342596924250892494078144297296753095978210007913212230665841"
    "357919237578285944025243864996731956910473047000710999509736544045105848"
    "756677133390378390825082359935691094230286336613531916938185170467344044"
    "801745940518857225083938406178966907437461486694552583611601083862174362"
    "139160726089288571738099772783043032800334941466983644687852340975025700"
    "905852511815436687924445748219185339376674037493793778650645930596484894"
    "404510948356944666924606941590703553276859837799050782132772527636402356"
    "157683727854792153173940527810658572724321401111588543812199484504463181"
    "712088496729161295293930814169289853167090564260899935963893644172260214"
```

```
    "8350364236393547970569049738611785624234033118371561747679806920223344 15"
    "4143034912620381013023091053598280017207518454900238990525270537328651 51"
    "8146941110265562290951575532862575379111614154178058673897286278074096 28"
    "0974854803605503003718261823889232329063460939038689123669705495909434 01"
    "4707478714922338492221035518017259919995954971588313835871641203490767 04"
    "5823161322667666713499446282684708227489734470846658453587083918436808 15"
    "8193329153214216658308857149874041016780986750352651252419223614935519 95"
    "1225016086262192708025434728265240759863268155099466776500496041882143 91"
    "8321368167176351260079258194026772540915857439646380367364389180629597 42"
    "6020083519292842530967743171191427010665571400159392563130825624517026 34"
    "8796023436824278953842563351545708821750902819081931074360440693989126 46"
    "6926545113667469510992093367611469544452253331048051108101819584616983 88"
    "9272456607007939966867829399269888703907288882962916532874600164110700 57"
    "0193726608106629436726279861830168373450455021255546557357213906774140 09"
    "3955047079779197225835398773825718071431855537039770753063288990245877 38"
    "7766160535769902414020454847179289661516201156 04"
)
CM1993_QY = int(
    "2109770685485498405027322037668992180060416947665838233075296631061924 9"
    "9029364878117631531976297922466557473531573085579073333682405946298611 03"
    "4496064389108928539302184343893286179340261700804462643399254729533711 13"
    "4645664757736760547324766489412041112951868604460232997019748395796843 07"
    "6144219840354016271949916903264914820191055831759287148343522478823704 23"
    "6346752216816882823928893146488722472132075411833464357743988396287353 81"
    "8594773620878960696461383809725032450858354568444728796210950681686923 25"
    "2018866906521509278133638659265534014448611114697999733609301307835403 8"
    "6466653834141170983304465175196342989047040649586597100778151873093641 40"
    "2834714067842174850325345318667495396212421062734923395933059122571446 88"
    "6223475272807504453779327710787824414369496533034140898842133317905589 27"
    "3606692770035100760131232300649795510946708580066372045568942926029841 43"
    "1410835870539991728804904459497225089534951620607562964083119639528067 59"
    "9143729277283966995578941953668703359298738781326826399164692664735637 82"
    "9789879266926807731077760110311154822946044251034980951300115540533364 56"
    "4931591824255918702318056153053942853410049051117015255267692626437083 70"
    "0785352573399598406654684982498171916543275334043240022797149175737417 64"
    "6950132052321254829514746465518643128553356821624648624038414331544456 51"
    "4906630005641023553543351322008542002770866853863753550004414012809939 31"
    "4344413840090041967286999355931906115298145423996362299557603746762940 62"
    "3539640907503400684097969875632647466586382926124301140081406957561071 00"
    "2739890839442326290528947231028451972776567738641798010622708802889429 46"
    "4872975705614126504415775166380294112983569754282402082764546134680565 99"
    "8881990942697099001748665285478963135760528343085348344389180353280070 86"
    "5664291938954745882424492482095646268981278101634161509570862190064912 31"
    "9779909789605845735463276357745239969828336850865552477394754913470907 08"
    "9868399954249052194499647494652927948925340354571406190724800369383972 40"
    "9952364996447502654000786342409929570746083266 05"
)

CM770_QX = int(
    "1045834384130076481052171574756958883051367946933776977783643283601561 20"
    "0044722823797276447798504505385198821595021642635908602005335273660514 82"
    "3967439277168369051981582677772639153765319656434266863916260931107815 96"
    "7553824627282421272714435768607848568680962291019721400644764086574459 72"
    "5489663237238799407290692332012957475148120666095159414886070435157298 66"
    "0041769401783863658015952567856283151437331572172189973890465901522661 30"
    "4850502608255627981241043005531461343862736482043013172465719095265880 93"
    "5250424760144138230364039330904847490767925001969623394609831324331912 91"
    "6896051992732170397971158504864930569252499813870588317288961301234299 61"
    "1059187175995256925274424736559907186361268429182886965925819288232953 16"
    "6453640513411997207330518380914905464101346274 4195"
)
CM770_QY = int(
    "1273908530159576501599786982770016693244762883510165614892909897530081 04"
    "3963186035730603619761581172834479856413195677978896251702039515061015 67"
    "7179473465001861971040375852327362044786266629664727934359595973131782 99"
    "9213074596039324008045120564441811426934220994171705801075814570065950 32"
    "5188279912421333854913200676548980126446165760712904143059402703429729 60"
    "6683958148840726045920977768025256731104858996089461594732019872359223 91"
    "0497443638512182758237811441866139749618706679055150733490128824559508 18"
    "9129004254057663992694928676082815926435269666168555453278306553293519 10"
    "3205321780608926991786963269036367389730695165079925766294153697179599 48"
    "2535895339461138156588868806912310332208445516342066020150347254314785 36"
```

```
    "05782308494379427759187594771855977973191210724248"
)

# (name, seed-id, norm D, a, b, t, curve A, curve B, affine point)
CM = (
 ("elliptic_reverse_cm", "Proth1000", 1, CM_C_A, CM_C_B,
  3018, -4, 0, (CM1006_QX, CM1006_QY)),
 ("elliptic_reverse_cm_1650", "Proth1000", 1, CM_C_A, CM_C_B,
  2**1650+94, -4, 0, (CM1993_QX, CM1993_QY)),
 ("elliptic_reverse_cm_minus3_638", "M1279", 3, CM_M_A, CM_M_B,
  2**638+794, 0, 1, (CM770_QX, CM770_QY)),
)

def factor_value(factors, numbers):
    return prod((numbers[q] if isinstance(q,str) else q)**e
                for q,e,_base in factors)

def build_numbers():
    n = {"M%d"%e: 2**e-1 for e in MERSENNE_EXPONENTS}
    n["Proth1000"] = PROTH_SEED[0]*2**PROTH_SEED[1]+1
    n["p0"] = 14**512-14**256+1
    for i,k in enumerate(CHAIN_COEFFICIENTS,1):
        n["p%d"%i] = k*n["p%d"%(i-1)]**2+1
    for name,factors,delta,d in CUBIC:
        F = factor_value(factors,n)
        n[name] = F**3-delta*F**2-d*F+1
    n["power4"] = 2*1241*n["M1279"]**4+1
    n["mixed"] = 2*361*n["M4423"]*n["Proth1000"]+1
    n["near_square"] = 2*n["M4423"]*(n["M4423"]-868)+1
    n["linear_2214_p3"] = 2214*n["p3"]+1
    name,seed,e,_a = PROTH_OUTPUT
    n[name] = n[seed]*2**e+1
    n["lucas_m1279"] = 2*388*n["M1279"]**2-1
    F = n["Proth1000"]
    n["negative_cube_Proth1000"] = F**3-2*F**2-9053*F-1
    name,Fid,Gid,offset,_a,_A,_Q = CRT
    F,G = n[Fid],n[Gid]
    modulus = 2*F*G
    rho = 1+2*F*((-pow(F,-1,G))%G)
    n[name] = rho+(G//F+1+offset)*modulus
    for e,r,paired in CYCLOTOMIC:
        b = r*n["M%d"%e]
        n["Phi3_M%d_r%d"%(e,r)] = b*b+b+1
        if paired:
            n["Phi7_M%d_r%d"%(e,r)] = sum(b**j for j in range(7))
    for name,seed,D,a,b,t,A,B,Q in CM:
        assert a*a+D*b*b == n[seed]
        n[name] = n[seed]*t*t+2*a*t+1
    n["Collatz_L0"] = 6*COLLATZ_R*n["M1279"]+1
    n["Collatz_L1"] = 8*COLLATZ_R*n["M1279"]+1
    return n

# Digits and SHA-256 of ASCII decimal digits WITHOUT a newline.
EXPECTED = {'M1279': (386, 'aaa42d44bf59eb3b901c42a4f1b4404b4d483d07bcf4dfb920cb1518da372239'),
 'M4423': (1332, '5c6c10ad2e971f207b1a7b31e1bfc3b1f608fb99f01f8f3d7bcda27474174bd9'),
 'Proth1000': (1000, '830f39a37d5887cd387e6d7faca5c0b495434a11f35bc424f5e1d458adc2aa87'),
 'TwoSidedCRT': (1999, 'b5b20a0e1e1e9879b276ac949818757395becc9c93b4ce502f1aeae528e53dde'),
 'cube_m1279': (1156, '00cf7903f0a9859d9d30d59ff6736e6c32877864ec3395a868689f32e5822345'),
 'cube_m4423': (3995, 'a4d0e01af571dc1bff0af971e4333e2b69ca7b5291ba50e467795d8f8771d5fe'),
 'cube_product': (4153, '75633ee2ef9d19c089b5c56999ac741d16ee0bc6e53ff45f056613f72fbeb294'),
 'cube_proth': (2998, '368200f424894a48b345556771bb59680ac5d558483646739332e955b4976080'),
 'lucas_m1279': (773, 'a8c8e34a06e557738eda7d13eec4a0d921874032d0711f205fed8efed5647c67'),
 'mersenne_to_proth': (1035, 'a012ccd657c3e9fa00c45848ba312b9ce2c5eeed97ea6ccc3efc7de2e2a26164'),
 'mixed': (2334, 'bc694e73d89eb6473bfbfd80e5f2239cc38c6494a735b942929eab97c2be2856'),
 'near_square': (2664, '49f9a2da9fdd46a784988ce3274abd0706901f7d43f1b3598d7000d8755a9dc2'),
 'power4': (1544, '9ae48b1c2900b09afb20cd70383851f5a620dc44846f24cc727714920c918a78'),
 'M521': (157, '06514590f94c1d4595274279dfb1568077269f22e4258a5c9f928fe07e832e0e'),
 'p0': (587, 'a338e3a2cccacc978def8f28b08c248d39d5cd1542a47f8f0cf627d03febfb0c'),
 'p1': (1177, 'c852e5af4a1166124ffff6084fb1d986da411fe5b210aace6739009180e12c3f'),
 'p2': (2357, 'f1c08bab7b1e51ec1eef6dc053d1d98a2a13eb74e323c7273ac65c2dd2c58785'),
 'p3': (4718, 'c3e84997a4b2a949282840d14981c52bd98d66982c022a38528038779cd32566'),
 'p4': (9439, '82ddc0fab24f40e0c8c777c2f2e2138fec8d9bc505e21eb414858111ca910089'),
```

```
 'elliptic_reverse_cm': (1006, 'ad37b747ef780d2277a07dcc27f3c46306ed93468ba94405a4739a40a75ae16a'),
 'elliptic_reverse_cm_1650': (1993, '2c65661c212fed01711609fdb236ec578ce7393fc575d2798ca010c8b46c4c18'),
 'elliptic_reverse_cm_minus3_638': (770, '1
    dec1d9b2886ded668571100864e4cd4a01d226c9641f1dddfc1ee324bac6290'),
 'negative_cube_Proth1000': (2998, '003683f716c33e8388e84922b33ec5c417d2b0866eebc25da205ac8028cede92'),
 'linear_2214_p3': (4721, '0d509027ec97653dc3dda75e7cfc2de75f32365004d625dd5510beff7788e9bc'),
 'Phi3_M521_r78299': (324, '3d6ea87798e2ac072eab8dcf6eab4e2adcd88eccebf7b8db128fe2503bb4c52d'),
 'Phi7_M521_r78299': (971, '293ca4a619d7f565236f87e8f156411a5f8f757fcbef1eb3190b43804ab49850'),
 'Phi3_M521_r260': (319, '43f721ffb5e37bc3393bc3bad8841e51cfc90a4b444be7febb9fa2f6fff155d7'),
 'Phi3_M1279_r48909': (780, '9b30f8ed0708b9a32807f0113b0988832f5ac750bec62d19668abd793a61c356'),
 'Phi7_M1279_r48909': (2339, 'c168e7e14a3bbc74c1a83cf5a03c1d728e410d109b930d395754a07d43f3cac1'),
 'Phi3_M1279_r4683': (778, 'db4eb5d98b75f8fc9ed4d912be961ed5cf6c48396075269e3f1f89265eb147b0')}

EXPECTED.update({
 "Collatz_L0": (391, "720bb661d9d67e6f31922febbffe8a88410a12492a9f12d20c9f932435c565bb"),
 "Collatz_L1": (391, "3f2baf960fdd26c8277fbaf35e045906de1a38b86e80cafa2c80145383e0b3b4"),
})


# Complete standard-library verification. Run with Python 3.11 or later.
import hashlib
import json
import math
import sys
import time

sys.set_int_max_str_digits(0)


def require(ok, message):
    if not ok:
        raise ValueError(message)


def small_prime(n):
    return n >= 2 and all(n % d for d in range(2, math.isqrt(n)+1))


def mm(x, y, n):
    a,b,c,d = x
    e,f,g,h = y
    return ((a*e+b*g)%n, (a*f+b*h)%n,
            (c*e+d*g)%n, (c*f+d*h)%n)


def lucas_u(k, a, b, n):
    r, m = (1,0,0,1), (a%n,(-b)%n,1,0)
    while k:
        if k & 1:
            r = mm(r,m,n)
        k >>= 1
        if k:
            m = mm(m,m,n)
    return r[2]


def ec_mul(k, point, n, ca):
    def add(p, q):
        if p is None:
            return q
        if q is None:
            return p
        x,y = p
        u,v = q
        if x == u:
            if (y+v)%n == 0:
                return None
            require(y == v, 'Exceptional affine case')
            top, bot = 3*x*x+ca, 2*y
        else:
            top, bot = v-y, u-x
        require(math.gcd(bot,n) == 1, 'Nonunit denominator')
```

```
        slope = top*pow(bot,-1,n)%n
        z = (slope*slope-x-u)%n
        return z, (slope*(x-z)-y)%n
    r = None
    for bit in bin(k)[2:]:
        r = add(r,r)
        if bit == '1':
            r = add(r,point)
    return r


def verify_all():
    start = time.perf_counter()
    numbers = build_numbers()
    require(set(numbers) == set(EXPECTED), 'Inventory mismatch')
    require(len(numbers) == len(set(numbers.values())) == 32,
            'Expected exactly 32 distinct integers')
    for name,n in numbers.items():
        s = str(n)
        require((len(s),hashlib.sha256(s.encode('ascii')).hexdigest())
                == EXPECTED[name], 'Digit or hash mismatch: '+name)
    proved, report = set(), []

    def accept(name, method, t0):
        require(name not in proved, 'Duplicate acceptance')
        proved.add(name)
        row = dict(id=name, digits=EXPECTED[name][0], method=method,
                   seconds=round(time.perf_counter()-t0,6))
        report.append(row)
        print(json.dumps(row), flush=True)

    def prime_value(ref):
        if isinstance(ref,int):
            require(ref < 1000000 and small_prime(ref), 'Small factor')
            return ref
        require(ref in proved, 'Unproved dependency: '+ref)
        return numbers[ref]

    def minus_orders(n, factors):
        require(n > 1 and n%2 == 1, 'Invalid N')
        f, qs, full = 1, set(), {}
        for ref,e,a in factors:
            q = prime_value(ref)
            require(q not in qs and e >= 1, 'Invalid factor list')
            qs.add(q)
            require((n-1)%q**e == 0, 'Factor not in N-1')
            f *= q**e
            z = pow(a,(n-1)//q,n)
            if a not in full:
                full[a] = pow(z,q,n)
            require(full[a] == 1 and math.gcd(z-1,n) == 1,
                    'N-1 order witness failed')
        return f

    def plus_orders(n, ref, e, a, b):
        p = prime_value(ref)
        f, d = p**e, a*a-4*b
        require(n > 1 and n%2 == 1 and (n+1)%f == 0, 'N+1 shape')
        require(math.gcd(n,2*b*d) == 1, 'Lucas degeneracy')
        require(lucas_u(n+1,a,b,n) == 0, 'Full Lucas residue')
        require(math.gcd(lucas_u((n+1)//p,a,b,n),n) == 1,
                'Reduced Lucas gcd')
        return f

    for e in MERSENNE_EXPONENTS:
        t0 = time.perf_counter()
        require(small_prime(e) and e%2, 'Mersenne exponent')
        name, s = 'M%d'%e, 4
        for _ in range(e-2):
            s = (s*s-2)%numbers[name]
        require(s == 0, 'Lucas-Lehmer residue')
        accept(name,'Lucas-Lehmer',t0)
```

```
t0 = time.perf_counter()
k,e,a = PROTH_SEED
n = numbers['Proth1000']
require(k%2 == 1 and 0 < k < 2**e and n == k*2**e+1,
        'Proth seed shape')
require(pow(a,(n-1)//2,n) == n-1, 'Proth seed witness')
accept('Proth1000','Proth',t0)

for name,factors,mode in N_MINUS_ONE:
    t0 = time.perf_counter()
    n = numbers[name]
    f = minus_orders(n,factors)
    require(mode in ('plus_one','odd'), 'Bound type')
    if mode == 'odd':
        require(f%2 == 1, 'Odd factor required')
    bound = 2*f+1 if mode == 'odd' else f+1
    require(bound*bound > n, 'Insufficient factored part')
    accept(name,'N-1 order and divisor bound',t0)

for name,factors,delta,d in CUBIC:
    t0 = time.perf_counter()
    n = numbers[name]
    f = minus_orders(n,factors)
    require(f >= 3 and f%2 == 1 and 0 < d < f and d%2,
            'Cubic parameter range')
    require(delta == (0 if f%4 == 3 else 2), 'Cubic delta')
    require(n == f**3-delta*f*f-d*f+1, 'Positive cubic shape')
    accept(name,'Positive cubic theorem',t0)

name,ref,e,a = PROTH_OUTPUT
t0 = time.perf_counter()
k, n = prime_value(ref), numbers[name]
require(k%2 == 1 and k < 2**e and n == k*2**e+1,
        'Proth lift shape')
require(pow(a,(n-1)//2,n) == n-1, 'Proth lift witness')
accept(name,'Proth',t0)

for name,ref,e,a,b,kind in LUCAS:
    t0 = time.perf_counter()
    n = numbers[name]
    f = plus_orders(n,ref,e,a,b)
    if kind == 'negative_cube':
        d, delta = 9053, 0 if f%4 == 3 else 2
        require(f%2 == 1 and 0 < d < f and d%2,
                'Negative cubic parameters')
        require(n == f**3-delta*f*f-d*f-1 and n > 1,
                'Negative cubic shape')
    else:
        require(kind == 'rank' and (f-1)**2 > n,
                'Lucas size bound')
    accept(name,'Lucas '+kind,t0)

t0 = time.perf_counter()
name,fi,gi,offset,a,la,lb = CRT
n, f, g = numbers[name], prime_value(fi), prime_value(gi)
require(f%2 and g%2 and math.gcd(f,g) == 1, 'CRT inputs')
require(minus_orders(n,((fi,1,a),)) == f, 'CRT minus')
require(plus_orders(n,gi,1,la,lb) == g, 'CRT plus')
modulus = 2*f*g
rho = 1+2*f*((-pow(f,-1,g))%g)
require(1 < rho < modulus and rho%(2*f) == 1
        and rho%g == g-1, 'CRT residue')
require(n < modulus**2 and (rho*rho > n or n%rho != 0),
        'CRT divisor exclusion')
accept(name,'Two-sided CRT',t0)

for name,ref,d,u,v,t,ca,cb,q in CM:
    t0 = time.perf_counter()
    p, n = prime_value(ref), numbers[name]
    require(n > 3 and n%2 == 1 and t > 0, 'Elliptic input range')
    require(u > 0 and v > 0 and u%2 == 0 and v%2 == 1,
```

```
                ’Norm parities’)
        require(u*u+d*v*v == p and n == p*t*t+2*u*t+1,
                ’Norm construction’)
        require(math.gcd(n,6*(4*ca**3+27*cb**2)) == 1,
                ’Bad curve reduction’)
        require((math.isqrt(p)-1)**4 > n, ’Hasse bound’)
        x,y = q
        require(0 <= x < n and 0 <= y < n
                and (y*y-x**3-ca*x-cb)%n == 0, ’Affine point’)
        require(ec_mul(p,q,n,ca) is None, ’Elliptic order’)
        accept(name,’Elliptic known prime order’,t0)

    q0, q1 = numbers[’Collatz_L0’], numbers[’Collatz_L1’]
    require(q1 > q0 > numbers[’M1279’] and q0%2 == q1%2 == 1,
            ’Collatz pair ordering’)
    require(3*q1+1 == 4*q0, ’Exact Collatz three-step relation’)
    require(proved == set(numbers), ’Uncertified integer remains’)
    result = dict(status=’ALL_32_PRIMES_CERTIFIED’, count=len(proved),
                  total_seconds=round(time.perf_counter()-start,6),
                  python=sys.version, checks=report)
    print(json.dumps(result,indent=2), flush=True)
    return result


if __name__ == ’__main__’:
    verify_all()
```

B Construction algorithms A01–A06

This complete program proves its common Mersenne seed, runs the bounded searches, and checks all construction certificates and controls.

```
"""A01--A06: exact certificates and bounded checks, Python standard library.

All accepted large integers depend on an independently repeated Lucas--Lehmer
certificate for 2**127-1.  Fermat passes are never accepted without the stated
structural theorem.  No internet, probable-prime acceptance, or random input.
"""
from math import gcd, isqrt
from pathlib import Path
import hashlib
import json
import time


def need(ok, message):
    if not ok:
        raise ValueError(message)


def trial_prime(n):
    if n < 2:
        return False
    if n % 2 == 0:
        return n == 2
    return all(n % d for d in range(3, isqrt(n) + 1, 2))


def mersenne_seed(e):
    need(e > 2 and trial_prime(e), "prime Mersenne exponent")
    p = (1 << e) - 1
    s = 4
    for _ in range(e - 2):
        s = (s*s - 2) % p
    need(s == 0, "Lucas--Lehmer residue")
    return p


def mm(x, y, n):
    a,b,c,d = x
    e,f,g,h = y
    return ((a*e+b*g)%n, (a*f+b*h)%n,
            (c*e+d*g)%n, (c*f+d*h)%n)


def lucas_u(k, a, b, n):
    """Lower-left entry of [[a,-b],[1,0]]**k, modulo n."""
    need(k >= 0 and n > 1, "Lucas arguments")
    z = (1,0,0,1)
    x = (a % n, (-b) % n, 1, 0)
    while k:
        if k & 1:
            z = mm(z, x, n)
        x = mm(x, x, n)
        k >>= 1
    return z[2]


def jacobi(a, n):
    need(n > 0 and n % 2, "Jacobi modulus")
    a %= n
    s = 1
    while a:
        while a % 2 == 0:
            a //= 2
            if n % 8 in (3,5):
                s = -s
```

```
        a,n = n,a
        if a % 4 == n % 4 == 3:
            s = -s
        a %= n
    return s if n == 1 else 0


def metadata(n):
    s = str(n)
    return dict(n=s, digits=len(s), bits=n.bit_length(),
                sha256=hashlib.sha256(s.encode()).hexdigest())


def fermat_pair(n, a, b):
    need(1 < b < n, "witness range")
    f = pow(b, n-1, n)
    short = pow(b, a, n)
    return f, short


def a01(p, a, b=2):
    """p must already be certified prime; parity-strengthened plus closure."""
    need(p > 2 and p % 2 and a > 0 and a % 2 == 0,
         "A01 input parity")
    need(a < 4*p-2, "A01 strict coefficient bound")
    n = a*p*p+1
    f, short = fermat_pair(n, a, b)
    if f != 1:
        return dict(status="COMPOSITE", reason="Fermat failure")
    if short == 1:
        return dict(status="INCONCLUSIVE", reason="short power is one")
    return dict(status="CERTIFIED", a=a, base=b, full_residue=f,
                short_residue=str(short), **metadata(n))


def cubic_factors(p, a):
    n = a*p**3+1
    bound = (a*p+4)//(2*p+1)
    for t in range(2, bound+1, 2):
        d = (t*p)**2-4*(a*p-t)
        if d < 0:
            continue
        z = isqrt(d)
        if z*z != d or (t*p-z) % 2:
            continue
        u,v = (t*p-z)//2, (t*p+z)//2
        if u >= 2 and v >= 2 and u % 2 == v % 2 == 0:
            r,s = u*p+1, v*p+1
            need(r*s == n, "reconstructed cubic product")
            return dict(t=t, u=u, v=v, factor=str(r), cofactor=str(s))
    return None


def a02(p, a, b=2):
    """One Fermat pair plus a finite, coefficient-sized trace exclusion."""
    need(p > 2 and p % 2 and a > 0 and a % 2 == 0,
         "A02 input parity")
    n = a*p**3+1
    f, short = fermat_pair(n, a, b)
    if f != 1:
        return dict(status="COMPOSITE", reason="Fermat failure")
    if short == 1:
        return dict(status="INCONCLUSIVE", reason="short power is one")
    factors = cubic_factors(p,a)
    if factors is not None:
        return dict(status="COMPOSITE", reason="trace factorization", **factors)
    bound = (a*p+4)//(2*p+1)
    return dict(status="CERTIFIED", a=a, base=b, full_residue=f,
                short_residue=str(short), trace_upper_bound=bound,
                traces_checked=bound//2, **metadata(n))
```

```
def pocklington(n, p, exponent=1, base=2):
    f = p**exponent
    need(n > 1 and n % 2 and (n-1) % f == 0,
         "Pocklington factor identity")
    need((2*f+1)**2 > n, "Pocklington parity bound")
    residue = pow(base, n-1, n)
    g = gcd(pow(base, (n-1)//p, n)-1, n)
    return residue == 1 and g == 1


def lucas_known_factor(n, p, a, b=1, exponent=1):
    f=p**exponent
    need((n+1) % f == 0 and (2*f-1)**2 > n,
         "Lucas known factor and size")
    d = a*a-4*b
    if gcd(n,2*b*d) != 1:
        return False
    return (lucas_u(n+1,a,b,n) == 0 and
            gcd(lucas_u((n+1)//p,a,b,n),n) == 1)


def a06(p, a, u, v=1):
    """Short-index Lucas certificate for a*p*p-1; p is already certified."""
    need(p > 2 and p % 2 and a > 0 and a % 2 == 0,
         "A06 input parity")
    need(a < 4*p+2, "A06 strict coefficient bound")
    need(isqrt(a)**2 != a, "A06 excludes square coefficients")
    n = a*p*p-1
    d = u*u-4*v
    if gcd(n,2*v*d) != 1:
        return dict(status="INCONCLUSIVE", reason="degenerate parameters")
    short = lucas_u(a,u,v,n)
    short_gcd = gcd(short,n)
    if 1 < short_gcd < n:
        return dict(status="COMPOSITE", factor=str(short_gcd))
    if short_gcd != 1:
        return dict(status="INCONCLUSIVE", reason="short rank not excluded")
    full = lucas_u(n+1,u,v,n)
    if full:
        return dict(status="INCONCLUSIVE", reason="full Lucas condition fails")
    return dict(status="CERTIFIED", a=a, lucas_A=u, lucas_B=v,
                discriminant=d, full_residue=full,
                short_residue=str(short), short_gcd=short_gcd, **metadata(n))


SMALL = [q for q in range(3,101,2) if trial_prime(q)]


def sieve(n):
    return all(n == q or n % q for q in SMALL)


def search_a01(p, stop=10000):
    checked = 0
    for a in range(2,min(stop,4*p-2),2):
        n = a*p*p+1
        if not sieve(n):
            continue
        checked += 1
        r = a01(p,a)
        if r['status'] == 'CERTIFIED':
            r['survivors_checked'] = checked
            return r
    raise RuntimeError("bounded A01 search had no certificate")


def search_a02(p, stop=10000):
    checked = 0
    for a in range(2,stop,2):
        if not sieve(a*p**3+1):
            continue
        checked += 1
```

```
        r = a02(p,a)
        if r['status'] == 'CERTIFIED':
            r['survivors_checked'] = checked
            return r
    raise RuntimeError("bounded A02 search had no certificate")


def search_a03(p, stop=50000):
    checked = 0
    for j in range(1,stop+1):
        k = 6*j
        minus,plus = k*p-1,k*p+1
        need(plus < (2*p-1)**2, "A03 common size bound")
        if not sieve(minus) or not sieve(plus):
            continue
        checked += 1
        if pow(2,plus-1,plus) != 1 or pow(2,minus-1,minus) != 1:
            continue
        if not pocklington(plus,p):
            continue
        for u in range(3,64):
            if jacobi(u*u-4,minus) == -1 and lucas_known_factor(minus,p,u):
                return dict(status="BOTH_CERTIFIED", k=k,
                            plus=metadata(plus), minus=metadata(minus),
                            plus_base=2, lucas_A=u, lucas_B=1,
                            parameters_examined=j, surviving_pairs=checked,
                            minus_full=lucas_u(minus+1,u,1,minus),
                            minus_reduced_gcd=gcd(lucas_u(k,u,1,minus),minus),
                            plus_full=pow(2,plus-1,plus),
                            plus_reduced_gcd=gcd(pow(2,k,plus)-1,plus))
    raise RuntimeError("bounded A03 search had no certified pair")


def search_a04(p, prefix=314159, suffix=27183, suffix_digits=5,
               middle_digits=90, stop=10000):
    b = 10
    f = p*p
    shift = b**suffix_digits
    offset = prefix*b**(suffix_digits+middle_digits)+suffix
    need(gcd(shift,f) == 1, "A04 invertible suffix shift")
    need(0 <= suffix < shift and prefix > 0, "A04 block widths")
    r = ((1-offset)*pow(shift,-1,f)) % f
    max_x = min(b**middle_digits-1, (f*f-1-offset)//shift)
    need(r <= max_x, "A04 nonempty certified interval")
    checked = 0
    for j in range(stop):
        x = r+j*f
        if x > max_x:
            break
        n = offset+x*shift
        if n % 2 == 0 or not sieve(n):
            continue
        checked += 1
        if pocklington(n,p,2):
            need(str(n).startswith(str(prefix)), "prefix reconstruction")
            need(str(n).endswith(str(suffix).zfill(suffix_digits)),
                 "suffix reconstruction")
            return dict(status="CERTIFIED", prefix=prefix, suffix=suffix,
                        suffix_digits=suffix_digits, middle_digits=middle_digits,
                        middle=str(x), least_middle_residue=str(r), j=j,
                        known_factor=str(f), base=2, surviving_candidates=checked,
                        full_residue=pow(2,n-1,n),
                        reduced_gcd=gcd(pow(2,(n-1)//p,n)-1,n), **metadata(n))
    raise RuntimeError("bounded A04 search had no certificate")


def cyclotomic_prime_power(b,p,t):
    need(b >= 2 and p > 2 and p % 2 and t >= 1, "A05 input range")
    x = b**(p**(t-1))
    return sum(x**j for j in range(p))
```

```
def geometric_sum_mod(x,k,n):
    """Return (x**k, 1+...+x**(k-1)) modulo n without division."""
    power,total = 1,0
    block_power,block_total = x % n,1
    while k:
        if k & 1:
            total=(total+power*block_total)%n
            power=power*block_power % n
        block_total=block_total*(1+block_power) % n
        block_power=block_power*block_power % n
        k >>= 1
    return power,total


def a05(b,p,t,r):
    """p is a certified odd prime; r is a supplied exact divisor."""
    need(b >= 2 and p > 2 and p % 2 and t >= 1, "A05 input range")
    bound = 2*p**t+1
    need(1 < r, "A05 nontrivial candidate")
    need(gcd(r,p) == 1, "A05 exceptional-prime exclusion")
    need(r < bound*bound, "A05 strict square bound")
    x=pow(b,p**(t-1),r)
    residue=geometric_sum_mod(x,p,r)[1]
    need(residue == 0, "A05 exact cyclotomic divisibility")
    out=dict(status="CERTIFIED", b=b, p=p, t=t,
             lower_factor_bound=bound, cyclotomic_residue=residue,
             calculation="modular geometric sum without division", **metadata(r))
    if p**t * b.bit_length() <= 20000:
        c=cyclotomic_prime_power(b,p,t)
        need(c % r == 0, "A05 small expanded-value cross-check")
        out.update(cyclotomic_value=str(c),cofactor=str(c//r))
    return out


def mine_a05(b,p,t):
    need(trial_prime(p), "A05 seed prime")
    m = p**t
    bound = 2*m+1
    out=[]
    for r in range(2*m+1,bound*bound,2*m):
        x=pow(b,p**(t-1),r)
        if geometric_sum_mod(x,p,r)[1] == 0 and gcd(r,p) == 1:
            out.append(a05(b,p,t,r))
    return out


def search_a06(p, stop=10000):
    checked=0
    for a in range(2,min(stop,4*p+2),2):
        if isqrt(a)**2 == a:
            continue
        n=a*p*p-1
        if not sieve(n):
            continue
        checked+=1
        if pow(2,n-1,n) != 1:
            continue
        for u in range(3,64):
            if jacobi(u*u-4,n) != -1:
                continue
            r=a06(p,a,u)
            if r['status'] == 'CERTIFIED':
                r['survivors_checked']=checked
                return r
    raise RuntimeError("bounded A06 search had no certificate")


def rejected(call):
    try:
        call()
    except ValueError:
        return True
```

```
    return False


def controls():
    # Independent Lucas recurrence checks, including negative parameters.
    matrix_checks=0
    for n in (35,77,101):
        for aa in range(-2,5):
            for bb in (-1,1,2):
                x,y=0,1
                for k in range(31):
                    need(lucas_u(k,aa,bb,n) == x % n,
                         "matrix Lucas versus direct recurrence")
                    x,y=y,aa*y-bb*x
                    matrix_checks+=1
    geometric_checks=0
    for n in (35,77,101):
        for x in range(2,10):
            for k in range(31):
                pair=geometric_sum_mod(x,k,n)
                need(pair==(pow(x,k,n),sum(x**j for j in range(k))%n),
                     "geometric sum versus direct finite sum")
                geometric_checks+=1
    # Boundary and missing-hypothesis counterexamples, not just malformed data.
    need(91 == 7*13 and pow(9,90,91)==1 and pow(9,10,91)==9,
         "A01 boundary counterexample")
    need(rejected(lambda:a01(3,10,9)), "A01 must reject equality boundary")
    need(a01(3,6,54)['status']=='INCONCLUSIVE', "A01 short nonunity required")
    need(pow(191,216,217)==1 and pow(191,8,217)!=1,
         "A02 positive Fermat pair on a composite")
    bad=a02(3,8,191)
    need(bad['status']=='COMPOSITE' and bad['reason']=='trace factorization',
         "A02 trace exclusion must detect 217")
    need(int(bad['factor'])*int(bad['cofactor'])==217, "A02 exact factors")
    need(lucas_u(36,1,1,35)==0 and gcd(lucas_u(4,1,1,35),35)==1,
         "A06 square-coefficient counterexample")
    need(rejected(lambda:a06(3,4,1)), "A06 square guard")
    need(lucas_u(126,1,1,125)==0 and gcd(lucas_u(14,1,1,125),125)==1,
         "A06 boundary counterexample")
    need(rejected(lambda:a06(3,14,1)), "A06 strict bound guard")
    need(cyclotomic_prime_power(4,3,2)==4161 and 4161 % 57 == 0,
         "A05 exceptional-prime composite example")
    need(rejected(lambda:a05(4,3,2,57)), "A05 exceptional-prime guard")
    need(rejected(lambda:a05(2,5,2,1082401)), "A05 strict size guard")

    plus_cases=minus_cases=cubic_cases=0
    plus_accepts=minus_accepts=cubic_accepts=0
    for p in (3,5,7,11,13,17,19,23,29,31):
        need(trial_prime(p), "small seed")
        for a in range(2,4*p-2,2):
            n=a*p*p+1
            prime=trial_prime(n)
            for b in range(2,min(n,16)):
                r=a01(p,a,b)
                plus_cases+=1
                if r['status']=='CERTIFIED':
                    plus_accepts+=1
                    need(prime, "A01 versus trial division")
        for a in range(2,4*p+2,2):
            if isqrt(a)**2==a:
                continue
            n=a*p*p-1
            prime=trial_prime(n)
            for u in (1,3,4,5,6,7,8):
                r=a06(p,a,u)
                minus_cases+=1
                if r['status']=='CERTIFIED':
                    minus_accepts+=1
                    need(prime, "A06 versus trial division")
        if p <= 19:
            for a in range(2,26,2):
                n=a*p**3+1
```

```
                prime=trial_prime(n)
                for b in range(2,10):
                    r=a02(p,a,b)
                    cubic_cases+=1
                    if r['status']=='CERTIFIED':
                        cubic_accepts+=1
                        need(prime, "A02 versus trial division")
    return dict(status="ALL_CONTROLS_PASSED", matrix_recurrence_checks=matrix_checks,
                geometric_sum_checks=geometric_checks,
                A01_cases=plus_cases, A01_acceptances=plus_accepts,
                A02_cases=cubic_cases, A02_acceptances=cubic_accepts,
                A06_cases=minus_cases, A06_acceptances=minus_accepts,
                essential_guard_counterexamples=7)


def main():
    started=time.perf_counter()
    p=mersenne_seed(127)
    out=dict(seed=dict(formula="2^127-1",lucas_lehmer_residue=0,**metadata(p)))
    for name,fun in [('A01',search_a01),('A02',search_a02),
                     ('A03',search_a03),('A04',search_a04),
                     ('A06',search_a06)]:
        t=time.perf_counter()
        out[name]=fun(p)
        out[name]['observed_seconds']=time.perf_counter()-t
        print(name,out[name]['status'],flush=True)
    t=time.perf_counter()
    out['A05']=dict(status="CERTIFIED_DIVISORS",
                    cases=mine_a05(2,5,2)+mine_a05(2,11,2))
    out['A05']['observed_seconds']=time.perf_counter()-t
    need([int(x['n']) for x in out['A05']['cases'][:2]]==[601,1801],
         "A05 specified divisors")
    for x in out['A05']['cases']:
        need(trial_prime(int(x['n'])), "A05 independent small trial division")
    # Different classical certificate implications for the refined rules.
    need(pocklington(int(out['A01']['n']),p,2),
         "A01 also certified by classical Pocklington")
    need(pocklington(int(out['A02']['n']),p,3),
         "A02 also certified by classical Pocklington")
    need(lucas_known_factor(int(out['A06']['n']),p,
                            out['A06']['lucas_A'],out['A06']['lucas_B'],2),
         "A06 also certified by classical Lucas prime-power criterion")
    out['classical_cross_checks']={
        'A01':'Pocklington with known factor p^2',
        'A02':'Pocklington with known factor p^3',
        'A06':'Lucas with known factor p^2 and reduced index a*p'}
    out['controls']=controls()
    out['status']='A01_A06_ALL_EXACT_CHECKS_PASSED'
    out['observed_total_seconds']=time.perf_counter()-started
    path=Path(__file__).with_name('results.json')
    path.write_text(json.dumps(out,indent=2)+'\n')
    print(json.dumps(out,indent=2))


if __name__=='__main__':
    main()
```

C Arithmetic and search algorithms A07–A12

This complete program includes the digit join, parameter sieve, two product trees, signed reduction, packet certificate, and finite planner.

```
#!/usr/bin/env python3
"""Exact, bounded reference experiments for algorithms A07--A12.

Python standard library only. No probable-prime acceptance, network access,
unbounded prime search, or external data. The structural kernel is tested on
integer products, not asserted to make a composite number prime.
"""
from __future__ import annotations
from functools import lru_cache
from itertools import permutations, product
from math import gcd, isqrt, prod
from pathlib import Path
import hashlib
import json
import random
import time


def require(condition, message):
    if not condition:
        raise ValueError(message)


def trial_prime(n):
    if n < 2:
        return False
    return all(n % d for d in range(2, isqrt(n) + 1))


def small_primes(bound):
    return [n for n in range(2, bound + 1) if trial_prime(n)]


def certificate(n, prime_factors):
    """Find and then exactly verify an elementary Pocklington certificate."""
    factor = 1
    for q, e in prime_factors.items():
        require(trial_prime(q) and e >= 1, "uncertified seed factor")
        factor *= q ** e
    require(n > 2 and n % 2 and (n-1) % factor == 0,
            "invalid factored-part input")
    require(factor * factor > n, "known factor too small")
    witnesses = {}
    for q in prime_factors:
        for a in range(2, min(n, 100)):
            if (pow(a, n-1, n) == 1 and
                    gcd(pow(a, (n-1)//q, n)-1, n) == 1):
                witnesses[str(q)] = a
                break
        if str(q) not in witnesses:
            return None
    for qs, a in witnesses.items():
        q = int(qs)
        require(pow(a, n-1, n) == 1, "full residue failed")
        require(gcd(pow(a, (n-1)//q, n)-1, n) == 1,
                "reduced gcd failed")
    return {"n": n, "F": factor, "factorization": prime_factors,
            "witnesses": witnesses}


def digits_value(word, base):
    r = 0
    for d in word:
        r = base * r + d
    return r
```

```
def word_ok(word):
    return all(a != b for a, b in zip(word, word[1:]))


def digit_mitm(digit_sets, base, factor, wheel):
    """A07: sparse join; no table indexed by every residue modulo factor."""
    split = len(digit_sets)//2
    tail = len(digit_sets) - split
    left = [w for w in product(*digit_sets[:split]) if word_ok(w)]
    right = [w for w in product(*digit_sets[split:]) if word_ok(w)]
    buckets = {}
    for w in right:
        value = digits_value(w, base)
        buckets.setdefault(value % factor, []).append((w, value))
    candidates = []
    bucket_hits = 0
    for a in left:
        prefix = digits_value(a, base)
        target = (1 - prefix * pow(base, tail, factor)) % factor
        for b, suffix in buckets.get(target, []):
            bucket_hits += 1
            if a[-1] == b[0]:
                continue
            n = prefix * base**tail + suffix
            if max(wheel) < n < factor*factor:
                candidates.append(n)
    candidates.sort()
    survivors = [n for n in candidates if all(n % q for q in wheel)]
    return {"left_words": len(left), "right_words": len(right),
            "right_buckets": len(buckets), "bucket_hits": bucket_hits,
            "congruence_candidates": candidates, "survivors": survivors}


def experiment_a07():
    alphabet = (1, 3, 7, 9)
    sets = [alphabet] * 4
    f = 107
    wheel = (3, 5, 7, 11)
    out = digit_mitm(sets, 10, f, wheel)
    brute = [digits_value(w, 10) for w in product(alphabet, repeat=4)
             if word_ok(w)]
    congr = sorted(n for n in brute if n % f == 1 and
                   max(wheel) < n < f*f)
    require(out["congruence_candidates"] == congr,
            "MITM omitted or added a word")
    require(out["survivors"] == [n for n in congr
                                 if all(n % q for q in wheel)],
            "wheel mismatch")
    certs = [certificate(n, {f: 1}) for n in out["survivors"]]
    require(all(c is not None for c in certs), "example not certified")
    require(all(trial_prime(c["n"]) for c in certs), "trial cross-check")
    out.update({"alphabet": alphabet, "length": 4,
                "language": "no equal adjacent digits", "F": f,
                "unrestricted_words": 4**4, "language_words": len(brute),
                "wheel_primes": wheel, "certificates": certs})
    return out


def polynomial_value(coefficients, t, modulus=None):
    value = 0
    for a in reversed(coefficients):
        value = value*t+a
        if modulus is not None:
            value %= modulus
    return value


def difference_heads(coefficients, modulus):
    row = [polynomial_value(coefficients, i, modulus)
           for i in range(len(coefficients))]
```

```
    heads = []
    while row:
        heads.append(row[0])
        row = [(b-a) % modulus for a, b in zip(row, row[1:])]
    return heads


def roots_by_differences(coefficients, q):
    """A08: updates must be performed from low to high index."""
    heads = difference_heads(coefficients, q)
    roots = []
    values = []
    for t in range(q):
        values.append(heads[0])
        if heads[0] == 0:
            roots.append(t)
        for j in range(len(heads)-1):
            heads[j] = (heads[j] + heads[j+1]) % q
    require(values == [polynomial_value(coefficients, t, q)
                       for t in range(q)], "finite-difference mismatch")
    return roots


def polynomial_sieve(coefficients, first, last, primes):
    factors = {}
    roots = {}
    for q in primes:
        residues = roots_by_differences(coefficients, q)
        roots[str(q)] = residues
        for r in residues:
            t = first + (r-first) % q
            for t in range(t, last+1, q):
                n = polynomial_value(coefficients, t)
                if n > q:
                    factors.setdefault(t, q)
    survivors = [t for t in range(first, last+1) if t not in factors]
    return roots, factors, survivors


def modular_sqrt(a, q):
    """Tonelli--Shanks on a prime field, with a checked returned root."""
    require(trial_prime(q), "root modulus not a proved small prime")
    a %= q
    if q == 2 or a == 0:
        return a
    if pow(a, (q-1)//2, q) != 1:
        return None
    if q % 4 == 3:
        r = pow(a, (q+1)//4, q)
    else:
        odd, v = q-1, 0
        while odd % 2 == 0:
            odd //= 2
            v += 1
        z = 2
        while pow(z, (q-1)//2, q) != q-1:
            z += 1
        c = pow(z, odd, q)
        r = pow(a, (odd+1)//2, q)
        t = pow(a, odd, q)
        m = v
        while t != 1:
            j, u = 0, t
            while u != 1 and j < m:
                u = u*u % q
                j += 1
            require(j < m, "Tonelli--Shanks invariant failed")
            b = pow(c, 1 << (m-j-1), q)
            r = r*b % q
            c = b*b % q
            t = t*c % q
            m = j
```

```
    require(r*r % q == a, "invalid square root")
    return r


def quadratic_lift_roots(factor, q):
    """Roots of factor*(2*t*t+2*t+2)+1 modulo q."""
    require(q > 1 and trial_prime(q), "invalid root-field prime")
    if q == 2 or factor % q == 0:
        return []
    delta = (-3-2*pow(factor,-1,q)) % q
    root = modular_sqrt(delta,q)
    if root is None:
        return []
    inv2 = pow(2,-1,q)
    return sorted({((-1+root)*inv2)%q,((-1-root)*inv2)%q})


def experiment_a08():
    f = 1009
    coefficients = (2*f+1, 2*f, 2*f)
    qs = small_primes(31)
    roots, factors, survivors = polynomial_sieve(coefficients, 0, 20, qs)
    compiled_roots = {str(q):quadratic_lift_roots(f,q) for q in qs}
    require(compiled_roots == roots, "quadratic root compiler mismatch")
    require(survivors == [t for t in range(21)
                          if all(polynomial_value(coefficients, t) % q
                                 for q in qs)], "sieve mismatch")
    for t, q in factors.items():
        n = polynomial_value(coefficients, t)
        require(trial_prime(q) and 1 < q < n and n % q == 0,
                "invalid rejection witness")
    certs = []
    for t in survivors:
        n = polynomial_value(coefficients, t)
        c = certificate(n, {f: 1})
        require((c is not None) == trial_prime(n), "bounded certificate check")
        if c is not None:
            c["t"] = t
            certs.append(c)
    # Degree exceeds some residue characteristics: no factorial inversion.
    extra_checks = 0
    for coefficients2 in [(3,-4,0,5,2), (0,1,0,-1), (7,)*7]:
        for q in (2,3,5,7,11):
            roots_by_differences(coefficients2, q)
            extra_checks += 1
    return {"F": f, "coefficients_low_to_high": coefficients,
            "t_interval": [0,20], "wheel_primes": qs, "roots": roots,
            "quadratic_root_compiler_matches": True,
            "rejected_parameters": {str(k):v for k,v in sorted(factors.items())},
            "surviving_parameters": survivors, "certificates": certs,
            "additional_polynomial_characteristic_checks": extra_checks,
            "finite_difference_additions": 2*sum(qs)}


def product_tree(values):
    require(values and all(n > 0 for n in values), "invalid product leaves")
    levels = [list(values)]
    while len(levels[-1]) > 1:
        a = levels[-1]
        levels.append([prod(a[j:j+2]) for j in range(0,len(a),2)])
    return levels


def remainder_tree(value, tree):
    values = [value % tree[-1][0]]
    for level in reversed(tree[:-1]):
        values = [values[j//2] % n for j,n in enumerate(level)]
    return values


def locate_factor(n, primes, qtree, level=None, index=0):
    """Descend only nodes whose product has nontrivial gcd with n."""
```

```
    if level is None:
        level = len(qtree)-1
    d = gcd(n, qtree[level][index])
    if d == 1:
        return None
    if level == 0:
        q = primes[index]
        require(n % q == 0, "bad factor route")
        return q
    for child in (2*index, 2*index+1):
        if child < len(qtree[level-1]):
            q = locate_factor(n, primes, qtree, level-1, child)
            if q is not None:
                return q
    raise ValueError("lost factor in product tree")


def batch_sieve(candidates, primes):
    require(all(trial_prime(q) for q in primes), "nonprime wheel entry")
    ntree = product_tree(candidates)
    qtree = product_tree(primes)
    residues = remainder_tree(qtree[-1][0], ntree)
    out = []
    for n, r in zip(candidates, residues):
        g = gcd(n, r)
        require(g == gcd(n, qtree[-1][0]), "batch gcd mismatch")
        if g == 1:
            out.append({"n":n,"gcd":g,"status":"UNRESOLVED"})
            continue
        q = locate_factor(n, primes, qtree)
        if q < n:
            require(n % q == 0, "improper rejection")
            out.append({"n":n,"gcd":g,"status":"COMPOSITE", "factor":q})
        else:
            require(q == n and trial_prime(q), "small prime equality")
            out.append({"n":n,"gcd":g,"status":"SMALL_PRIME", "factor":q})
    return out, len(ntree), len(qtree)


def experiment_a09():
    ns = [1713,7919,341,49,17,10403,42379,223999]
    qs = small_primes(43)
    out, nl, ql = batch_sieve(ns, qs)
    require([x["n"] for x in out if x["status"] == "COMPOSITE"] ==
            [1713,341,49], "unexpected factor routing")
    require([x["n"] for x in out if x["status"] == "SMALL_PRIME"] == [17],
            "small wheel prime misclassified")
    # A composite can survive every wheel prime.
    require(10403 == 101*103 and
            next(x for x in out if x["n"]==10403)["status"]=="UNRESOLVED",
            "survival incorrectly promoted")
    return {"candidates":ns,"wheel_primes":qs,"wheel_product":prod(qs),
            "candidate_tree_levels":nl,"wheel_tree_levels":ql,
            "leaf_gcds":out}


def cofactor_reduce(x, k, nbits, sign):
    """A10: exact one-correction reduction in the stated product range."""
    require(k >= 1 and nbits >= 1 and sign in (-1,1), "bad modulus shape")
    radix = 1 << nbits
    modulus = k*radix+sign
    require(modulus >= 3 and 0 <= x <= (modulus-1)**2, "outside proved range")
    high, low = x >> nbits, x & (radix-1)
    quotient, digit = divmod(high, k)
    value = low + (digit << nbits) - sign*quotient
    if sign == 1 and value < 0:
        value += modulus
    elif sign == -1 and value >= modulus:
        value -= modulus
    require(0 <= value < modulus, "one-correction range failed")
    return value
```

```
def cofactor_pow(a, exponent, k, nbits, sign):
    require(exponent >= 0, "negative exponent outside this algorithm")
    modulus = k*(1<<nbits)+sign
    a %= modulus
    r = 1
    for bit in bin(exponent)[2:]:
        r = cofactor_reduce(r*r,k,nbits,sign)
        if bit == '1':
            r = cofactor_reduce(r*a,k,nbits,sign)
    return r


def experiment_a10():
    exhaustive = 0
    for k in range(1,10):
        for nbits in range(1,5):
            for sign in (-1,1):
                modulus=k*(1<<nbits)+sign
                if modulus<3:
                    continue
                for u in range(modulus):
                    for v in range(modulus):
                        require(cofactor_reduce(u*v,k,nbits,sign)==u*v%modulus,
                                "exhaustive product failure")
                        exhaustive += 1
    rng=random.Random(20261106)
    stress=0
    for nbits in (64,256,1024,4096):
        for k in (1,3,17,2**31-1):
            for sign in (-1,1):
                modulus=k*(1<<nbits)+sign
                for _ in range(24):
                    u=rng.randrange(modulus);v=rng.randrange(modulus)
                    require(cofactor_reduce(u*v,k,nbits,sign)==u*v%modulus,
                            "large product failure")
                    stress+=1
                for exponent in (0,1,17,257):
                    require(cofactor_pow(3,exponent,k,nbits,sign)==
                            pow(3,exponent,modulus),"power failure")
    # Outside the range x <= (N-1)^2, a single correction can fail.
    k,nbits,sign,x=1,2,1,24
    modulus=5
    high,low=x>>nbits,x&3
    t,s=divmod(high,k)
    uncorrected=low+(s<<nbits)-t
    corrected_once=uncorrected+modulus if uncorrected<0 else uncorrected
    require(corrected_once==-1 and x%modulus==4,"boundary counterexample")
    rejected=False
    try:
        cofactor_reduce(x,k,nbits,sign)
    except ValueError:
        rejected=True
    require(rejected,"range guard missing")
    return {"exhaustive_canonical_product_checks":exhaustive,
            "stress_product_checks":stress,"max_radix_bits":4096,
            "power_checks":4*4*2*4,
            "outside_range_counterexample":{"N":5,"x":24,
                 "one_correction_result":corrected_once,"true_remainder":4},
            "runtime_speed_claim":"NONE"}


def crt(residues, moduli):
    require(len(residues)==len(moduli) and moduli,"empty CRT packet")
    z,modulus=0,1
    for residue,n in zip(residues,moduli):
        require(gcd(modulus,n)==1,"CRT inputs not pairwise coprime")
        step=((residue-z)*pow(modulus,-1,n))%n
        z+=modulus*step
        modulus*=n
    require(all(z%n==r%n for r,n in zip(residues,moduli)),"CRT mismatch")
    return z,modulus
```

```
def verify_packet(p,moduli,z):
    """A11 proof depends only on order witnesses, not local trial division."""
    if not trial_prime(p) or p<3 or not moduli:
        return False
    if any(n<=1 or n>=p**4 for n in moduli):
        return False
    modulus=prod(moduli)
    if not 0<=z<modulus:
        return False
    y=pow(z,p,modulus)
    return pow(y,p,modulus)==1 and gcd(y-1,modulus)==1


def binary_cost(exponent):
    require(exponent>=1,"positive exponent needed")
    return exponent.bit_length()-1, exponent.bit_count()-1


def experiment_a11():
    p=11
    multipliers=[6,12,18,22]
    moduli=[k*p*p+1 for k in multipliers]
    local=[pow(2,k,n) for k,n in zip(multipliers,moduli)]
    z,modulus=crt(local,moduli)
    require(verify_packet(p,moduli,z),"correct packet failed")
    require(all(trial_prime(n) for n in moduli),"independent small primality")
    y=pow(z,p,modulus)
    require(all(y%n==pow(r,p,n) for r,n in zip(local,moduli)),
            "packet residue extraction")
    bad=[243]+moduli[1:]
    # Keep the same candidate construction, including k=2 for 243.
    badlocal=[pow(2,k,n) for k,n in zip([2]+multipliers[1:],bad)]
    badz,badm=crt(badlocal,bad)
    require(not verify_packet(p,bad,badz),"composite packet accepted")
    require(not verify_packet(9,moduli,z),"composite seed accepted")
    require(not verify_packet(p,moduli,0),"zero witness accepted")
    require(not verify_packet(p,moduli,1),"identity witness accepted")
    require(not verify_packet(p,[p**4],0),"size boundary accepted")
    sq,mul=binary_cost(p)
    widths=[n.bit_length() for n in moduli]
    return {"p":p,"multipliers":multipliers,"moduli":moduli,
            "local_roots":local,"packet_modulus":modulus,"packet_witness":z,
            "intermediate_y":y,"final_power":pow(y,p,modulus),
            "final_gcd":gcd(y-1,modulus),"packet_status":"ALL_CERTIFIED",
            "component_bits":widths,"packet_bits":modulus.bit_length(),
            "individual_binary_operations":{"squares":2*sq*len(moduli),
                                            "multiplies":2*mul*len(moduli)},
            "packet_binary_operations":{"squares":2*sq,"multiplies":2*mul},
            "schoolbook_square_work_individual":2*sq*sum(b*b for b in widths),
            "schoolbook_square_work_packet":2*sq*modulus.bit_length()**2,
            "negative_controls":5,"runtime_speed_claim":"NONE"}


def optimal_screen_plan(rejections,costs,terminal_cost,count):
    """A12: finite population, deterministic outcomes, prescribed cost model."""
    all_ids=frozenset(range(count))
    s=len(rejections)
    @lru_cache(None)
    def remaining(mask):
        alive=all_ids
        for j,R in enumerate(rejections):
            if mask>>j&1:
                alive=alive-R
        return alive
    @lru_cache(None)
    def solve(mask):
        alive=remaining(mask)
        best=(terminal_cost*len(alive),())
        for j in range(s):
            if not mask>>j&1:
```

```
                tail,order=solve(mask|1<<j)
                offer=(costs[j]*len(alive)+tail,(j,)+order)
                if offer[0]<best[0]:
                    best=offer
        return best
    cost,order=solve(0)
    return cost,order,solve.cache_info().currsize


def plan_cost(order,rejections,costs,terminal_cost,count):
    alive=set(range(count))
    total=0
    for j in order:
        total+=costs[j]*len(alive)
        alive-=rejections[j]
    total+=terminal_cost*len(alive)
    return total,sorted(alive)


def experiment_a12():
    f=1009
    ns=[f*(2*t*t+2*t+2)+1 for t in range(21)]
    qs=[17,3,13,5,7]
    costs=[5,1,4,1,2]
    terminal=40
    rejections=[frozenset(t for t,n in enumerate(ns) if n%q==0) for q in qs]
    opt,order,states=optimal_screen_plan(rejections,costs,terminal,len(ns))
    best_exhaustive=None
    checked=0
    for length in range(len(qs)+1):
        for perm in permutations(range(len(qs)),length):
            cost,_=plan_cost(perm,rejections,costs,terminal,len(ns))
            best_exhaustive=cost if best_exhaustive is None else min(best_exhaustive,cost)
            checked+=1
    require(opt==best_exhaustive,"DP not globally optimal in model")
    baseline,rest=plan_cost(range(len(qs)),rejections,costs,terminal,len(ns))
    actual,rest_opt=plan_cost(order,rejections,costs,terminal,len(ns))
    require(actual==opt,"plan reconstruction cost")
    return {"population":"1009*(2*t*t+2*t+2)+1, t=0..20", "count":len(ns),
            "screen_primes":qs,"abstract_costs":costs,"terminal_cost":terminal,
            "rejection_sets":{str(q):sorted(R) for q,R in zip(qs,rejections)},
            "optimal_order":[qs[j] for j in order],"optimal_cost":opt,
            "baseline_order":qs,"baseline_cost":baseline,
            "terminal_parameters":rest_opt,"DP_states":states,
            "ordered_subsets_exhaustively_checked":checked,
            "planning_cost_in_objective":False,
            "wall_time_or_future_population_claim":"NONE"}


def main():
    start=time.perf_counter()
    results={"A07":experiment_a07(),"A08":experiment_a08(),
             "A09":experiment_a09(),"A10":experiment_a10(),
             "A11":experiment_a11(),"A12":experiment_a12()}
    results["status"]="ALL_SIX_ARITHMETIC_ALGORITHMS_VERIFIED"
    results["reference_runtime_seconds"]=time.perf_counter()-start
    results["source_sha256"]=hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
    output=Path(__file__).with_name("results.json")
    output.write_text(json.dumps(results,indent=2,sort_keys=True)+"\n")
    print(json.dumps(results,indent=2,sort_keys=True))


if __name__=="__main__":
    main()
```

D Certificate algorithms A13–A18

This complete program verifies all input evidence before applying the certificate transformations and hybrid closure.

```
"""Exact, standard-library examples for Algorithms A13--A18.

These are bounded demonstrations of certificate transformations, not a
large-prime search.  Every acceptance uses explicit integer conditions.
No probable-prime result is accepted as a proof.
"""
from math import gcd, isqrt, prod
from heapq import heappop, heappush
import json
import time


def need(ok, message):
    if not ok:
        raise ValueError(message)


def trial_prime(n):
    return n >= 2 and all(n % d for d in range(2, isqrt(n) + 1))


def factor_small(n):
    need(n >= 1, "positive factorization input")
    out, q = {}, 2
    while q * q <= n:
        while n % q == 0:
            out[q] = out.get(q, 0) + 1
            n //= q
        q += 1
    if n > 1:
        out[n] = out.get(n, 0) + 1
    return out


def factored_value(factors):
    need(bool(factors), "nonempty factorization")
    for q, e in factors.items():
        need(trial_prime(q) and isinstance(e, int) and e >= 1,
             "certified prime factors and positive exponents")
    return prod(q ** e for q, e in factors.items())


def pocklington_partial(n, factors, witnesses):
    """Return a prime-divisor lower bound; n need not be prime."""
    need(n > 1 and n % 2 == 1, "odd modulus greater than one")
    f = factored_value(factors)
    need((n - 1) % f == 0, "known part divides n-1")
    residues = []
    for q in sorted(factors):
        a = witnesses[q]
        full = pow(a, n - 1, n)
        reduced = pow(a, (n - 1) // q, n)
        d = gcd(reduced - 1, n)
        need(full == 1 and d == 1, "Pocklington order obligation")
        residues.append([q, a, full, reduced, d])
    lower = 2 * f + 1 if f % 2 else f + 1
    return {"N": n, "F": f, "prime_divisor_lower_bound": lower,
            "residues": residues, "proves_whole_N": lower * lower > n}


def verify_factored_order(n, h, z):
    factors = factor_small(h)  # only the small demonstration inputs
    need(h > 1 and (n - 1) % h == 0, "factored exponent")
    need(pow(z, h, n) == 1, "annihilation by H")
    gs = [gcd(pow(z, h // q, n) - 1, n) for q in factors]
```

```
    need(all(d == 1 for d in gs), "exact H-order over every prime divisor")
    lower = 2 * h + 1 if h % 2 else h + 1
    return {"H": h, "z": z, "gcds": gs,
            "prime_divisor_lower_bound": lower,
            "proves_whole_N": lower * lower > n}


def prime_power_fusion(n, witnesses):
    """A13: separate full Pocklington witnesses -> one H-order witness."""
    e = n - 1
    need(n > 1 and n % 2 and witnesses, "fusion input")
    h, z, components = 1, 1, []
    for q, a in sorted(witnesses.items()):
        need(trial_prime(q) and e % q == 0, "selected certified factor")
        need(pow(a, e, n) == 1, "input full power")
        need(gcd(pow(a, e // q, n) - 1, n) == 1, "input q-order")
        t = 1
        while e % (t * q) == 0:
            t *= q
        u = pow(a, e // t, n)
        h *= t
        z = z * u % n
        components.append({"q": q, "prime_power": t, "a": a,
                           "projection": u})
    record = verify_factored_order(n, h, z)
    record.update({"N": n, "components": components})
    return record


def batch_units(n, values):
    """A14: one product gcd on success; exact leaf diagnosis on failure."""
    need(n > 1 and values, "nonempty unit batch")
    values = [x % n for x in values]

    def build(lo, hi):
        if hi - lo == 1:
            return (lo, hi, values[lo], None, None)
        mid = (lo + hi) // 2
        left, right = build(lo, mid), build(mid, hi)
        return (lo, hi, left[2] * right[2] % n, left, right)

    tree = build(0, len(values))
    bad, calls, root_gcd = [], 0, None

    def walk(node):
        nonlocal calls, root_gcd
        calls += 1
        d = gcd(node[2], n)
        if calls == 1:
            root_gcd = d
        if d == 1:
            return
        if node[1] - node[0] == 1:
            bad.append([node[0], d])
            return
        walk(node[3])
        walk(node[4])

    walk(tree)
    proper = sorted({d for _, d in bad if 1 < d < n})
    status = ("ALL_UNITS" if not bad else
              "PROPER_FACTOR_FOUND" if proper else
              "NONUNIT_WITHOUT_PROPER_FACTOR")
    return {"N": n, "values": values, "root_gcd": root_gcd,
            "failed_leaves": bad, "proper_factors": proper,
            "tree_gcd_calls": calls, "status": status}


class NonunitDenominator(ValueError):
    def __init__(self, factor):
        super().__init__("elliptic denominator is not a unit")
        self.factor = factor
```

```
def on_curve(p, n, a, b):
    return p is None or (p[1] * p[1] - p[0] ** 3 - a * p[0] - b) % n == 0


def ec_add(p, q, n, a, b):
    """Affine operations with explicit conditions valid at every prime divisor."""
    need(on_curve(p, n, a, b) and on_curve(q, n, a, b), "curve membership")
    if p is None:
        return q
    if q is None:
        return p
    x, y = p
    u, v = q
    if x == u:
        if (y + v) % n == 0:
            return None
        need(y == v, "unresolved equal-x exceptional case")
        numerator, denominator = 3 * x * x + a, 2 * y
    else:
        numerator, denominator = v - y, u - x
    d = gcd(denominator, n)
    if d != 1:
        raise NonunitDenominator(d)
    slope = numerator * pow(denominator, -1, n) % n
    rx = (slope * slope - x - u) % n
    r = (rx, (slope * (x - rx) - y) % n)
    need(on_curve(r, n, a, b), "addition result on curve")
    return r


def ec_mul(k, p, n, a, b):
    need(k >= 0, "nonnegative scalar")
    r = None
    for bit in bin(k)[2:]:
        r = ec_add(r, r, n, a, b)
        if bit == "1":
            r = ec_add(r, p, n, a, b)
    return r


def ec_order_record(n, a, b, p, factors):
    need(n > 1, "modulus greater than one")
    f = factored_value(factors)
    need(gcd(n, 6) == 1, "small characteristic excluded")
    need(gcd(4 * a ** 3 + 27 * b * b, n) == 1, "smooth reductions")
    need(p is not None and on_curve(p, n, a, b), "affine input point")
    need(ec_mul(f, p, n, a, b) is None, "elliptic annihilation")
    reduced = []
    for q in sorted(factors):
        t = ec_mul(f // q, p, n, a, b)
        need(t is not None, "strongly nonzero reduced multiple")
        reduced.append([q, list(t)])
    return {"order": f, "point": list(p), "reduced_multiples": reduced}


def elliptic_fusion(n, a, b, records):
    """A15: project overlapping certified orders to primary components."""
    checked, best = [], {}
    for i, (p, factors) in enumerate(records):
        checked.append(ec_order_record(n, a, b, p, factors))
        for q, e in factors.items():
            if q not in best or e > best[q][0]:
                best[q] = (e, i)
    r, h, projections = None, 1, []
    for q, (e, i) in sorted(best.items()):
        t = q ** e
        f = checked[i]["order"]
        z = ec_mul(f // t, records[i][0], n, a, b)
        need(z is not None, "nonzero primary component")
        r = ec_add(r, z, n, a, b)
```

```
        h *= t
        projections.append([q, e, i, list(z)])
    compact = ec_order_record(n, a, b, r, {q: e for q, (e, _) in best.items()})
    # Stronger integral sufficient bound already used in the main manuscript.
    proves = (isqrt(h) - 1) ** 4 > n
    return {"N": n, "curve": [a, b], "inputs": checked,
            "projections": projections, "fused": compact,
            "proves_whole_N": proves}


def crt_merge(m, residues, modulus, allowed):
    """Generalized CRT, preserving all possible signs/classes."""
    g = gcd(m, modulus)
    new_m = m // g * modulus
    new = set()
    for r in residues:
        for s in allowed:
            if (s - r) % g:
                continue
            v = modulus // g
            t = 0 if v == 1 else ((s - r) // g * pow(m // g, -1, v)) % v
            new.add((r + m * t) % new_m)
    return new_m, sorted(new)


def hasse_windows(n, h):
    """Every prime divisor <= sqrt(n) lies in this exact union of intervals."""
    need(n > 1 and h > 1, "positive modulus and nontrivial order")
    s = isqrt(n)
    tmax = (s + 1 + isqrt(4 * s)) // h
    intervals = []
    for t in range(1, tmax + 1):
        centre, radius = t * h + 1, isqrt(4 * t * h)
        lo, hi = max(2, centre - radius), min(s, centre + radius)
        if lo <= hi:
            intervals.append([lo, hi])
    return {"sqrt_floor": s, "T": tmax, "intervals": intervals}


def constrained_divisors(n, modulus, residues, intervals, budget=100000):
    """A16 closure AFTER the residue and elliptic evidence has been checked."""
    candidates = set()
    for lo, hi in intervals:
        for r in residues:
            first = lo + (r - lo) % modulus
            for d in range(first, hi + 1, modulus):
                if len(candidates) >= budget:
                    return {"status": "INCONCLUSIVE_BUDGET"}
                candidates.add(d)
    candidates = sorted(candidates)
    factors = [d for d in candidates if n % d == 0]
    return {"candidates": candidates, "divisor_checks": len(candidates),
            "status": "COMPOSITE" if factors else "PRIME",
            "proper_divisors": factors}


def verify_hybrid_certificate(n, arithmetic_factors, arithmetic_witnesses,
                              a, b, point, elliptic_factors):
    """A16 entry point: construct every predicate from checked evidence."""
    arithmetic = pocklington_partial(n, arithmetic_factors, arithmetic_witnesses)
    elliptic = ec_order_record(n, a, b, point, elliptic_factors)
    f = arithmetic["F"]
    modulus = 2 * f if f % 2 else f
    m, residues = crt_merge(1, [0], modulus, [1])
    windows = hasse_windows(n, elliptic["order"])
    closure = constrained_divisors(n, m, residues, windows["intervals"])
    return {"N": n, "arithmetic": arithmetic, "elliptic": elliptic,
            "windows": windows, "modulus": m, "residues": residues,
            "closure": closure}


def proof_plan(nodes, anchors, target, edges):
```

```
    """A17: minimum nonnegative additive cost in the verified-set graph."""
    index = {x: i for i, x in enumerate(nodes)}
    mask = lambda values: sum(1 << index[x] for x in set(values))
    start, goal = mask(anchors), 1 << index[target]
    prepared = [(mask(e["deps"]), 1 << index[e["out"]], e) for e in edges]
    dist, parent, queue = {start: 0}, {}, [(0, start)]
    while queue:
        cost, state = heappop(queue)
        if cost != dist[state]:
            continue
        if state & goal:
            path, end = [], state
            while state != start:
                old, name = parent[state]
                path.append(name)
                state = old
            return {"cost": cost, "path": path[::-1],
                    "visited_states": len(dist), "final_mask": end}
        for required, out, edge in prepared:
            need(edge["cost"] >= 0, "nonnegative costs")
            if state & out or state & required != required:
                continue
            new, value = state | out, cost + edge["cost"]
            if value < dist.get(new, float("inf")):
                dist[new] = value
                parent[new] = state, edge["name"]
                heappush(queue, (value, new))
    return None


def verify_plan(plan, anchors, edges):
    known = set(anchors)
    need(all(trial_prime(p) for p in known), "verified anchors")
    by_name = {e["name"]: e for e in edges}
    for name in plan["path"]:
        edge = by_name[name]
        need(set(edge["deps"]) <= known, "no circular proof dependency")
        n = edge["out"]
        if edge["kind"] == "trial":
            need(trial_prime(n), "trial certificate")
        else:
            factors, a = edge["factors"], edge["a"]
            need(set(factors) <= known, "every factor already proved")
            record = pocklington_partial(n, factors, {q: a for q in factors})
            need(record["proves_whole_N"], "Pocklington size bound")
        known.add(n)
    return sorted(known)


def inherit_divisor(n, lower, d):
    """A18: lower is a VERIFIED bound on every prime divisor of n."""
    need(1 < d < n and n % d == 0, "exact proper divisor")
    need(lower >= 2 and d < lower * lower, "strict inherited size bound")
    return {"divisor": d, "cofactor": n // d,
            "inherited_lower_bound": lower, "status": "PRIME"}


def must_reject(fn, message):
    try:
        fn()
    except ValueError:
        return
    raise ValueError("negative control accepted: " + message)


def run_all():
    began = time.perf_counter()
    out = {}
    fusion31 = prime_power_fusion(31, {2: 3, 3: 21})
    fusion1009 = prime_power_fusion(1009, {2: 11, 3: 2})
    need(3 * 21 % 31 == 1, "raw witness multiplication loses all orders")
    need(fusion31["proves_whole_N"] and fusion1009["proves_whole_N"],
```

```
     "A13 advertised certificates")
fusion_checks = 0
for n in range(3, 400, 2):
    witnesses = {}
    for q in factor_small(n - 1):
        for a in range(2, min(n, 18)):
            if pow(a, n - 1, n) == 1 and gcd(pow(a, (n - 1) // q, n) - 1, n) == 1:
                witnesses[q] = a
                break
    if not witnesses:
        continue
    rec = prime_power_fusion(n, witnesses)
    for ell in factor_small(n):
        need(pow(rec["z"], rec["H"], ell) == 1, "independent annihilation")
        need(all(pow(rec["z"], rec["H"] // q, ell) != 1
                 for q in witnesses), "independent exact primary orders")
    if rec["proves_whole_N"]:
        need(trial_prime(n), "independent A13 primality cross-check")
    fusion_checks += 1
must_reject(lambda: prime_power_fusion(31, {2: 1}), "bad fusion witness")
out["A13"] = {"examples": [fusion31, fusion1009],
              "independent_small_moduli_checked": fusion_checks}

h, z = fusion1009["H"], fusion1009["z"]
positive = batch_units(1009, [pow(z, h // q, 1009) - 1 for q in [2, 3]])
split = batch_units(91, [2, 7, 13, 5])
unsuitable = batch_units(101, [0, 2, 3, 4])
need(positive["status"] == "ALL_UNITS" and positive["tree_gcd_calls"] == 1,
     "one gcd positive path")
need(split["root_gcd"] == 91 and split["proper_factors"] == [7, 13],
     "saturated product gcd localized")
need(unsuitable["status"] == "NONUNIT_WITHOUT_PROPER_FACTOR", "not a compositeness claim")
unit_checks = 0
for n in range(2, 150):
    for values in [[2, 3, 5, 7], [0, n - 1, n + 1], [n, 2 * n, n * n],
                   [n // 2, n // 3, n // 5, n // 7]]:
        rec = batch_units(n, values)
        expected = [[i, gcd(v, n)] for i, v in enumerate(values) if gcd(v, n) > 1]
        need(rec["failed_leaves"] == expected, "gcd tree agrees with every leaf")
        unit_checks += 1
out["A14"] = {"positive": positive, "factor_localization": split,
              "unsuitable_witness": unsuitable, "independent_batches": unit_checks}

n, a, b = 101, -4, -2
p, q = (22, 16), (40, 39)
ec = elliptic_fusion(n, a, b, [(p, {3: 1, 5: 1}), (q, {7: 1})])
need(ec["fused"]["order"] == 105 and ec["proves_whole_N"], "fused elliptic certificate")
raw = ec_add(p, q, n, a, b)
need(raw == (69, 53), "advertised direct coprime sum")
ec_order_record(n, a, b, raw, {3: 1, 5: 1, 7: 1})
# An overlapping-order case where a naive sum loses the factor 5.
p15 = ec_mul(7, raw, n, a, b)
p35 = ec_mul(3, raw, n, a, b)
naive = ec_add(p15, p35, n, a, b)
need(ec_mul(21, naive, n, a, b) is None, "overlap cancellation control")
overlap = elliptic_fusion(n, a, b, [(p15, {3: 1, 5: 1}),
                                    (p35, {5: 1, 7: 1})])
need(overlap["fused"]["order"] == 105, "LCM fusion avoids overlap cancellation")
negative_p = (p[0], (-p[1]) % n)
need(ec_add(p, negative_p, n, a, b) is None, "raw point-sum cancellation")
same_order = elliptic_fusion(n, a, b, [(p, {3: 1, 5: 1}),
                                       (negative_p, {3: 1, 5: 1})])
need(same_order["fused"]["order"] == 15, "primary selection prevents total cancellation")
must_reject(lambda: ec_order_record(101, 0, 0, (1, 1), {2: 1}), "singular curve")
must_reject(lambda: ec_order_record(n, a, b, p, {7: 1}), "incorrect point order")
must_reject(lambda: elliptic_fusion(-1, 0, 0, [((0, 0), {2: 1})]),
            "nonpositive elliptic modulus")
try:
    ec_add((0, 1), (5, 14), 35, 0, 1)
except NonunitDenominator as error:
    need(error.factor == 5, "proper denominator factor")
else:
```

```
        raise ValueError("nonunit elliptic addition was accepted")
    need(trial_prime(n), "independent A15 cross-check")
    out["A15"] = {"coprime_fusion": ec, "direct_sum": list(raw),
                  "overlapping_fusion": overlap,
                  "naive_overlapping_sum_order_divides": 21,
                  "opposite_equal_order_points_cancel_raw": True,
                  "nonunit_control_factor": 5}

    n = 211
    hybrid = verify_hybrid_certificate(n, {5: 1}, {5: 2},
                                       -2, 2, (23, 27), {19: 1})
    order, windows, closure = (hybrid["arithmetic"], hybrid["windows"],
                               hybrid["closure"])
    need(not order["proves_whole_N"], "arithmetic uniform bound alone insufficient")
    s = isqrt(n)
    need(19 <= s + 1 + isqrt(4 * s), "elliptic uniform bound alone insufficient")
    need(windows["intervals"] == [[12, 14]], "exact Hasse window")
    need(closure["status"] == "PRIME" and closure["divisor_checks"] == 0,
         "empty hybrid intersection")
    need(trial_prime(n), "independent A16 cross-check")
    window_checks = 0
    for h in range(2, 120):
        ntest = 10000
        ww = hasse_windows(ntest, h)
        represented = {r for lo, hi in ww["intervals"] for r in range(lo, hi + 1)}
        direct = set()
        for r in range(2, isqrt(ntest) + 1):
            radius = isqrt(4 * r)
            low, high = max(1, r + 1 - radius), r + 1 + radius
            if ((low + h - 1) // h) * h <= high:
                direct.add(r)
        need(represented == direct, "integer window equivalence cross-check")
        window_checks += 1
    must_reject(lambda: verify_hybrid_certificate(n, {5: 1}, {5: 1},
                                                  -2, 2, (23, 27), {19: 1}),
                "unverified arithmetic predicate")
    must_reject(lambda: verify_hybrid_certificate(n, {5: 1}, {5: 2},
                                                  -2, 2, (23, 28), {19: 1}),
                "unverified elliptic predicate")
    hybrid["window_equivalence_checks"] = window_checks
    out["A16"] = hybrid

    nodes = [2, 3, 7, 13, 2003]
    edges = [
        {"name": "prove_3", "deps": [2], "out": 3, "cost": 7,
         "kind": "pock", "factors": {2: 1}, "a": 2},
        {"name": "7_via_3", "deps": [3], "out": 7, "cost": 1,
         "kind": "pock", "factors": {3: 1}, "a": 2},
        {"name": "13_via_3", "deps": [3], "out": 13, "cost": 1,
         "kind": "pock", "factors": {3: 1}, "a": 2},
        {"name": "7_direct", "deps": [], "out": 7, "cost": 6, "kind": "trial"},
        {"name": "13_direct", "deps": [], "out": 13, "cost": 6, "kind": "trial"},
        {"name": "goal", "deps": [7, 13], "out": 2003, "cost": 1,
         "kind": "pock", "factors": {7: 1, 13: 1}, "a": 3},
    ]
    plan = proof_plan(nodes, [2], 2003, edges)
    need(plan is not None and plan["cost"] == 10, "optimal shared proof plan")
    known = verify_plan(plan, [2], edges)
    # Independent exhaustive enumeration of edge subsets in this finite example.
    optimum = float("inf")
    for subset in range(1 << len(edges)):
        selected = [e for i, e in enumerate(edges) if subset >> i & 1]
        closure_nodes = {2}
        while True:
            newer = closure_nodes | {e["out"] for e in selected if set(e["deps"]) <= closure_nodes}
            if newer == closure_nodes:
                break
            closure_nodes = newer
        if 2003 in closure_nodes:
            optimum = min(optimum, sum(e["cost"] for e in selected))
    need(optimum == plan["cost"], "exhaustive optimality cross-check")
    cycle = [{"name": "c1", "deps": [7], "out": 3, "cost": 0},
```

```
            {"name": "c2", "deps": [3], "out": 7, "cost": 0}]
    need(proof_plan([2, 3, 7], [2], 7, cycle) is None, "unanchored cycle rejected")
    out["A17"] = {"nodes": nodes, "anchors": [2], "catalogue": edges,
                  "plan": plan, "verified_primes": known,
                  "independent_branch_model_cost": 13,
                  "costs_are_assigned_model_units_not_wall_times": True,
                  "edge_subsets_checked": 1 << len(edges),
                  "unanchored_cycle_rejected": True}

    n = 1387
    partial = pocklington_partial(n, {3: 2}, {3: 2})
    need(not partial["proves_whole_N"], "auxiliary modulus not certified prime")
    residue = pow(2, (n - 1) // 2, n)
    d = gcd(residue - 1, n)
    need(d == 73 and n // d == 19, "exact factor extraction")
    inherited = [inherit_divisor(n, partial["prime_divisor_lower_bound"], x)
                 for x in [d, n // d]]
    need(all(trial_prime(x["divisor"]) and x["divisor"] > 3 for x in inherited),
         "independent harvested-prime cross-check")
    must_reject(lambda: inherit_divisor(6859, 19, 361), "strict inherited size boundary")
    must_reject(lambda: inherit_divisor(n, 19, 17), "nondivisor")
    out["A18"] = {"auxiliary_N": n, "seed": 3, "coefficient": 154,
                  "partial_order_certificate": partial,
                  "factor_extraction_residue": residue, "factor": d,
                  "harvested": inherited,
                  "new_modular_powers_for_inherited_divisors": 0}
    out["status"] = "ALL_SIX_CERTIFICATE_ALGORITHMS_VERIFIED"
    out["audit_runtime_seconds"] = time.perf_counter() - began
    return out


if __name__ == "__main__":
    print(json.dumps(run_all(), indent=2, sort_keys=True))
```

E Exact verification of the continuing seven-node chain

This retained program reconstructs the seven-node chain through p_6, proves its known factors in dependency order, and checks both the original Pocklington route and the alternate A01 obligations where applicable. Its code and historical success status are unchanged. The additional linear siblings belong to the current ten-node verifier in Appendix G.

```
"""Exact later seven-node chain; optional --gmp, no external data files."""
import argparse
import hashlib
import json
import math
import sys
import time

if hasattr(sys, "set_int_max_str_digits"):
    sys.set_int_max_str_digits(0)

def require(ok, message):
    if not ok:
        raise ValueError(message)

def verify(use_gmp=False):
    if use_gmp:
        import gmpy2
        integer, power, gcd = gmpy2.mpz, gmpy2.powmod, gmpy2.gcd
    else:
        integer, power, gcd = int, pow, math.gcd
    start = time.perf_counter()
    p = integer(14)**512 - integer(14)**256 + 1
    expected = [
        (587, "a338e3a2cccacc978def8f28b08c248d39d5cd1542a47f8f0cf627d03febfb0c"),
        (1177, "c852e5af4a1166124ffff6084fb1d986da411fe5b210aace6739009180e12c3f"),
        (2357, "f1c08bab7b1e51ec1eef6dc053d1d98a2a13eb74e323c7273ac65c2dd2c58785"),
        (4718, "c3e84997a4b2a949282840d14981c52bd98d66982c022a38528038779cd32566"),
        (9439, "82ddc0fab24f40e0c8c777c2f2e2138fec8d9bc505e21eb414858111ca910089"),
        (18882, "7407dab961f68eb15854c74700a8007bf30d89410a6beeb1d2fc7d365f4b7ce8"),
        (18885, "1fd5cb8c8470b74c24c163501a2775693b19707db62c65e6b074a1e56ff7cbd3"),
    ]
    nodes = [(p, [(integer(2),256,11), (integer(7),256,2)])]
    for k,e in [(438,2),(13696,2),(5494,2),(13258,2),(22330,2),(5936,1)]:
        n = k*p**e + 1
        nodes.append((n, [(p,e,2)]))
        p = n
    proved = {integer(2), integer(7)}
    rows = []
    for i,(n, factors) in enumerate(nodes):
        began = time.perf_counter()
        f = integer(1)
        for q,e,a in factors:
            require(q in proved and e > 0, "unproved factor")
            f *= q**e
        require((n-1)%f == 0 and f*f > n, "factor-size condition")
        for q,e,a in factors:
            require(power(integer(a),n-1,n) == 1, "Fermat congruence")
            z = power(integer(a),(n-1)//q,n)
            require(gcd(z-1,n) == 1, "Pocklington gcd")
        decimal = str(n)
        digest = hashlib.sha256(decimal.encode("ascii")).hexdigest()
        require((len(decimal),digest) == expected[i], "identity mismatch")
        alternate = None
        if 1 <= i <= 5:
            q = factors[0][0]
            k = (n-1)//(q*q)
            require(q > 2 and q % 2 and k > 0 and k % 2 == 0,
                    "A01 parity")
            require(k < 4*q-2, "A01 strict coefficient bound")
            short = power(integer(2), k, n)
            require(short != 1, "A01 short nonunity")
            # The full base-2 Fermat condition was verified above.
```

```
            alternate = {
                "algorithm":"A01", "coefficient":int(k),
                "short_power_is_one":False,
                "short_nonunity_from_size":bool(integer(1)<<int(k) < n),
                "candidate_bits":int(n.bit_length())}
        proved.add(n)
        rows.append({"index":i,"digits":len(decimal),"sha256":digest,
                     "alternate_quadratic_certificate":alternate,
                     "seconds":time.perf_counter()-began})
    return {"status":"ALL_SEVEN_PRIMES_CERTIFIED",
            "arithmetic":"gmpy2" if use_gmp else "Python integers",
            "seconds":time.perf_counter()-start,"nodes":rows}

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--gmp",action="store_true")
    args = parser.parse_args()
    print(json.dumps(verify(args.gmp),indent=2))
```

F Independent checks of the coefficient boundaries

This audit uses direct Lucas recurrence and exact small trial division to check the positive and negative coefficient closures independently of the construction program's matrix implementation.

```
"""Independent bounded audit of the two quadratic closure lemmas.
Small primality truth uses exhaustive trial division. Lucas values here use
the direct recurrence, independently of fast matrix/doubling implementations.
"""
from math import gcd, isqrt
import json, time

def require(ok, text):
    if not ok:
        raise ValueError(text)

def prime(n):
    return n >= 2 and all(n%d for d in range(2,isqrt(n)+1))

def lucas_pair_direct(n,a,c,b):
    old,cur=0,1
    short=0 if a==0 else None
    for j in range(1,n+2):
        if j==a:
            short=cur
        if j==n+1:
            return short,cur
        old,cur=cur,(c*cur-b*old)%n
    raise ValueError("unreachable")

def run():
    start=time.perf_counter()
    pos_cases=pos_passes=neg_cases=neg_passes=0
    for p in range(3,100,2):
        if not prime(p):
            continue
        for a in range(2,4*p-2,2):
            n=a*p*p+1
            truth=prime(n)
            for b in (2,3,5,9):
                pos_cases+=1
                passed=pow(b,n-1,n)==1 and pow(b,a,n)!=1
                require(not passed or truth,"positive false certificate")
                pos_passes+=passed
    for p in (3,5,7,11,13,17,19):
        for a in range(2,4*p+2,2):
            if isqrt(a)**2==a:
                continue
            n=a*p*p-1
            truth=prime(n)
            for c,b in ((1,1),(3,1),(4,1),(5,1),(5,2)):
                disc=c*c-4*b
                if gcd(n,2*b*disc)!=1:
                    continue
                ua,ufull=lucas_pair_direct(n,a,c,b)
                neg_cases+=1
                passed=ufull==0 and gcd(ua,n)==1
                require(not passed or truth,"negative false certificate")
                neg_passes+=passed
    require(91==7*13 and pow(9,90,91)==1 and pow(9,10,91)!=1,
            "positive boundary control")
    for a,n in ((4,35),(14,125)):
        ua,ufull=lucas_pair_direct(n,a,1,1)
        require(not prime(n) and gcd(n,-6)==1 and ufull==0
                and gcd(ua,n)==1,"negative missing-guard control")
    # A single correction is not guaranteed for arbitrary x < N^2.
    n,radix,k,x=3,2,1,8
    big,low=divmod(x,radix)
    t,s=divmod(big,k)
```

```
    y=low+s*radix-t
    corrected=y+n if y<0 else y
    require(x<n*n and corrected!=x%n,"fold domain control")
    return {"status":"INDEPENDENT_BOUNDARIES_VERIFIED",
            "positive_attempts":pos_cases,"positive_certificates":pos_passes,
            "negative_attempts":neg_cases,"negative_certificates":neg_passes,
            "false_prime_certificates":0,
            "guard_counterexamples":[91,35,125],
            "fold_out_of_domain":{"N":3,"x":8,"one_correction":corrected,
                                  "correct_remainder":x%n},
            "seconds":time.perf_counter()-start}

if __name__=="__main__":
    print(json.dumps(run(),indent=2))
```

G Complete verification of the current large-prime register

The standalone program reconstructs $p_0, \ldots, p_5$ and all four linear descendants. It proves 2 and 7, then verifies all ancestors, Pocklington conditions, and decimal fingerprints. It uses no primality oracle, search, external integer data, or checkpoint. The labels `r`, `r38456`, and `r59790` denote r_{62988}, r_{38456}, and r_{59790}, respectively.

Validation used the two stages recorded in Section 14.1: eight nodes, then two extra leaves using the freshly certified parent. The integrated listing retains the executed certificate function unchanged; the added nodes, fingerprints, ordering check, and status were checked against the saved results. No single ten-node execution is claimed. Both stages used GMP through `gmpy2`; the complete Python fallback was not run for this update.

Table 8: All witness obligations in the current large-prime graph. Every full Fermat residue and every reduced-power gcd was 1.

Node	Factored part F	Prime factor: base	Reduced exponent
p_0	$2^{256}7^{256}$	$2 : 11$	$(p_0 - 1)/2$
p_0	$2^{256}7^{256}$	$7 : 2$	$(p_0 - 1)/7$
p_1	p_0^2	$p_0 : 2$	$438p_0$
p_2	p_1^2	$p_1 : 2$	$13696p_1$
p_3	p_2^2	$p_2 : 2$	$5494p_2$
p_4	p_3^2	$p_3 : 2$	$13258p_3$
p_5	p_4^2	$p_4 : 2$	$22330p_4$
p_6	p_5	$p_5 : 2$	5936
r_{38456}	p_5	$p_5 : 2$	38456
r_{59790}	p_5	$p_5 : 2$	59790
r_{62988}	p_5	$p_5 : 2$	62988

There are eleven witness rows because the initial seed requires two distinct prime-factor checks. There are ten non-elementary integer nodes, and five of them have at least 18,000 decimal digits. The four terminal nodes share p_5 and are not one succession of linear lifts.

```
"""Reconstruct and certify ten primes; no external data or PRP oracle.

Optional gmpy2 accelerates exact arithmetic. Standard Python integers
provide a slower fallback. All four linear leaves depend on p5.
"""
import hashlib
import json
import math
import sys
import time

if hasattr(sys, "set_int_max_str_digits"):
    sys.set_int_max_str_digits(0)
try:
    from gmpy2 import mpz as Z, powmod as modpow, gcd
    backend = "gmpy2 exact integers"
except ImportError:
    Z, modpow, gcd = int, pow, math.gcd
    backend = "Python exact integers"


def need(condition, reason):
    if not condition:
        raise ArithmeticError(reason)


def verify():
    start = time.perf_counter()
    known, rows = {}, []
```

```
    for q in (2, 7):
        need(all(q % d for d in range(2, math.isqrt(q) + 1)),
             "base-prime trial division")
        known[Z(q)] = str(q)

    def certify(label, n, factors):
        tick = time.perf_counter()
        need(n > 2 and n % 2 == 1, "odd positive candidate")
        F, checks = Z(1), []
        for q, e, a in factors:
            need(q in known and e >= 1, "proved prime factor")
            F *= q**e
        need((n-1) % F == 0, "F divides n-1")
        need(F*F > n, "Pocklington size bound")
        for q, e, a in factors:
            need(1 < a < n, "witness range")
            full = modpow(Z(a), n-1, n)
            reduced = modpow(Z(a), (n-1)//q, n)
            g = gcd(reduced-1, n)
            need(full == 1 and g == 1, "Pocklington witness")
            checks.append(dict(q=known[q], e=e, witness=a,
                               full_residue=int(full), gcd=int(g)))
        known[n] = label
        decimal = str(n)
        rows.append(dict(name=label, digits=len(decimal),
                         bits=int(n.bit_length()),
                         sha256=hashlib.sha256(
                             decimal.encode("ascii")).hexdigest(),
                         F_divides_N_minus_1=True, F_squared_gt_N=True,
                         F_bits=int(F.bit_length()), checks=checks,
                         seconds=time.perf_counter()-tick))
        return n

    x = Z(14)**256
    p = certify("p0", x*x-x+1,
                [(Z(2), 256, 11), (Z(7), 256, 2)])
    for i, k in enumerate((438, 13696, 5494, 13258, 22330), 1):
        p = certify("p" + str(i), k*p*p+1, [(p, 2, 2)])
    p5 = p
    p6 = certify("p6", 5936*p5+1, [(p5, 1, 2)])
    r = certify("r", 62988*p5+1, [(p5, 1, 2)])
    r38456 = certify("r38456", 38456*p5+1, [(p5, 1, 2)])
    r59790 = certify("r59790", 59790*p5+1, [(p5, 1, 2)])
    need(p5 < p6 < r38456 < r59790 < r,
         "strict ordering of large primes")
    expected = {
        "p5": (18882, 62722,
               "7407dab961f68eb15854c74700a8007bf"
               "30d89410a6beeb1d2fc7d365f4b7ce8"),
        "p6": (18885, 62735,
               "1fd5cb8c8470b74c24c163501a2775693"
               "b19707db62c65e6b074a1e56ff7cbd3"),
        "r": (18886, 62738,
              "ca4c8532ec289a68d0fe9769f87f3797"
              "aa560f9f1a141866da17b917c11c1081"),
        "r38456": (18886, 62737,
                   "00d10910b2a4d068db8b0cc164956d198"
                   "3cc62a1d3950a0a0d25548402b51d36"),
        "r59790": (18886, 62738,
                   "a9652db4b6bef070ce335537f7634a68"
                   "e43e689d768d2d4cbeef3f536a0d28fa")}
    for row in rows:
        if row["name"] in expected:
            need((row["digits"], row["bits"], row["sha256"])
                 == expected[row["name"]], "identity fingerprint")
    return dict(status="ALL_TEN_PRIMES_CERTIFIED", backend=backend,
                seconds=time.perf_counter()-start, nodes=rows)


if __name__ == "__main__":
    print(json.dumps(verify(), indent=2))
```